파이썬 기초

Python Basic

류영표 강사
ryp1662@gmail.com



Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.

류 영 표

AI & Applied Mathematics Researcher · Lecturer

ryp1662@gmail.com



학력

- 동국대학교 일반대학원 응용수학 석사 수료 (2016-2019)
- 동국대학교 수학과 졸업 (2010-2016)

심사 및 전문가 자문 활동

- 인공지능 학습용 데이터 구축 사업 품질검증 전문가 자문위원 (2023)
- 2023 신뢰할 수 있는 인공지능 개발 안내서(안)_고도화 전문위원
- 2023 자율주행/의료/공공사회 분야 신뢰할 수 있는 인공지능 개발 안내서(안) 자문위원

연구 및 프로젝트

- 부산대학교 병원: 개인 맞춤형 당뇨병 예방·관리 인공지능 시스템 외주 프로젝트 (2022)
- 연세대학교 병원: 영상의학과 FLASK 고도화 연구 (2023, 2024)
- 산업수학 스터디 그룹 (국가수리과학연구소): 피부암/유방암 분류, 유전자 정보 분석
- 폐플라스틱 이미지 객체 검출 경진대회 3위 (2021)
- 동국대학교 수학과 연구원(2025.10 ~ / 양자 AI, XAI 연구 진행 중)

논문

- Quantum computing for the optimization of CT image reconstruction, IEEE ICTC 2022

㈜ 헬로월드랩스 전담교육팀 과장

강의 및 교육 활동

대학 강의

- 고려대학교, 인천대학교, 동양미래대학교, 목포대학교
- 이젠 아카데미: 파이썬/ADSP 과정

기업 교육

- 현대자동차 연구원
- 한화, 하나금융, 한전 KDN, IBK 기업은행, 아가방앤컴퍼니, IM증권

공공/국비 과정

- [Est Camp] AI 퀀트 캠프 1기
- 보건·바이오 AI 활용 전문가 양성과정 (1-3기)
- 국제인공지능올림피아드 한국 대표 멘토
- 충청 ICT 취창업 역량강화 프로그램 (SQLD)

아카데미 & 부트캠프

- 모두의연구소 Aiffel 퍼실리테이터 (1기)
- Upstage AI Boostcamp 멘토 (1-6기)
- 패스트캠퍼스/멀티캠퍼스 조교 및 멘토
- 공개SW개발자대회 멘토

Contents

1. 파이썬 소개
2. 파이썬의 기본
3. 파이썬의 자료형
4. 조건문
5. 반복문
6. 함수
7. 파일 읽기
8. 파이썬 심화

1. 파이썬 소개

- Why python?
- Programming Language Ranking

- What is programming?
- 코딩 교육

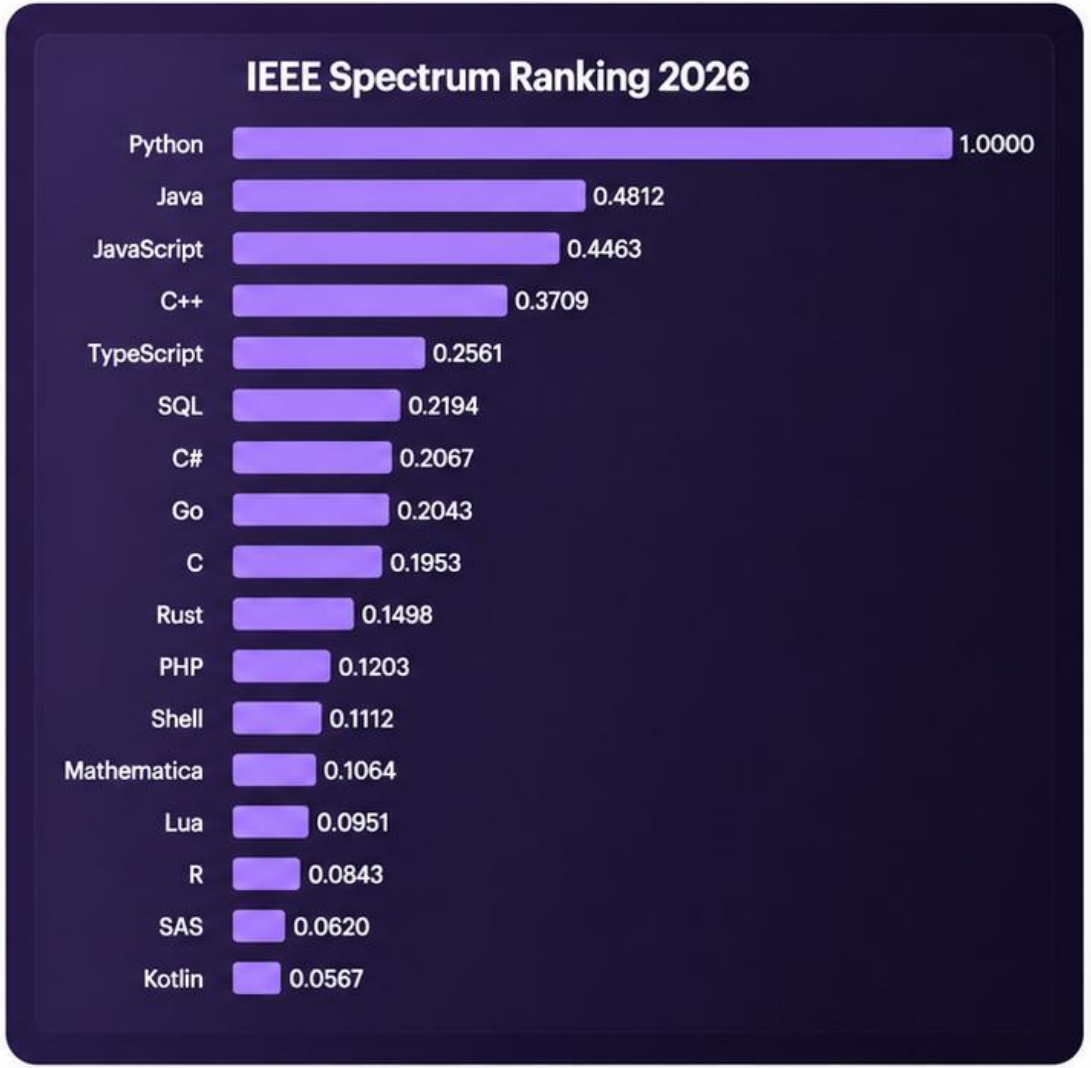
- 파이썬 이란?
- Python의 특징
- Python 2 VS python 3
- 컴파일러 VS 인터프리터
- Why 파이썬?
- 다양한 라이브러리 지원

- 파이썬 개발환경
- 개발 환경 개요
- 운영체제-Operation System
- 코드 편집기(Editor)
- ANACONDA 설치
- Jupyter Notebook 사용하기
- Google colab 사용법
- Goggle colab 런타임 방지·Python
- IDE Pycharm 이란?

Why Python

The TIOBE Ranking 2026

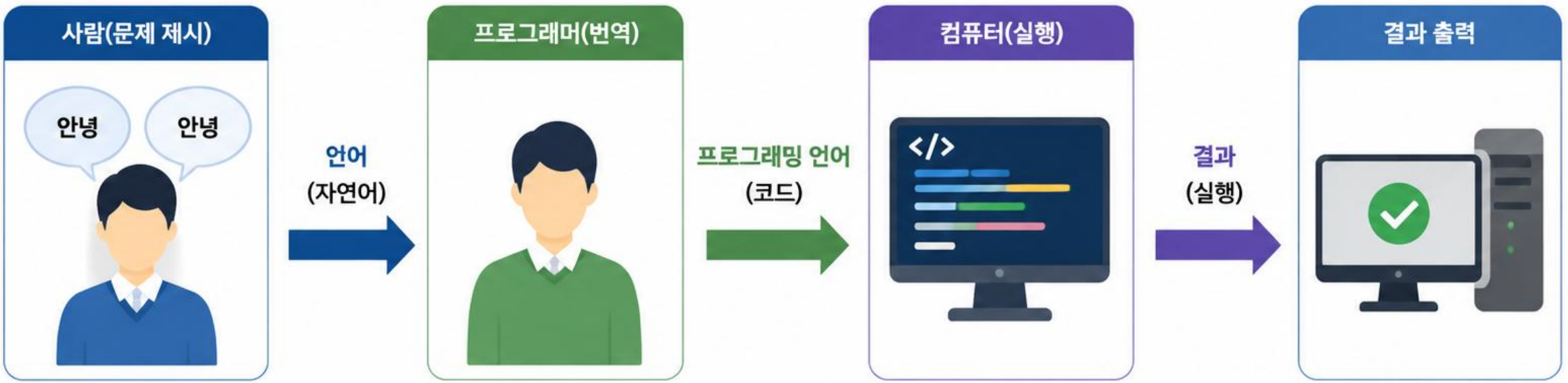
FEB 2026	FEB 2025	LANGUAGE	RATINGS	CHANGE
1	1	Python	24.31%	+8.91%
^ 2	3	C++	11.96%	+0.59%
^ 3	4	Java	10.79%	+0.13%
v 4	2	C	9.33%	-0.51%
5	5	C#	4.24%	+0.12%



What is programming?

프로그래밍(programming)이란?

사람이 이해하는 언어(자연어)를 컴퓨터가 이해하고 실행할 수 있는 언어(프로그래밍 언어)로 바꾸어,
컴퓨터가 **문제를 해결**하도록 지시하는 과정

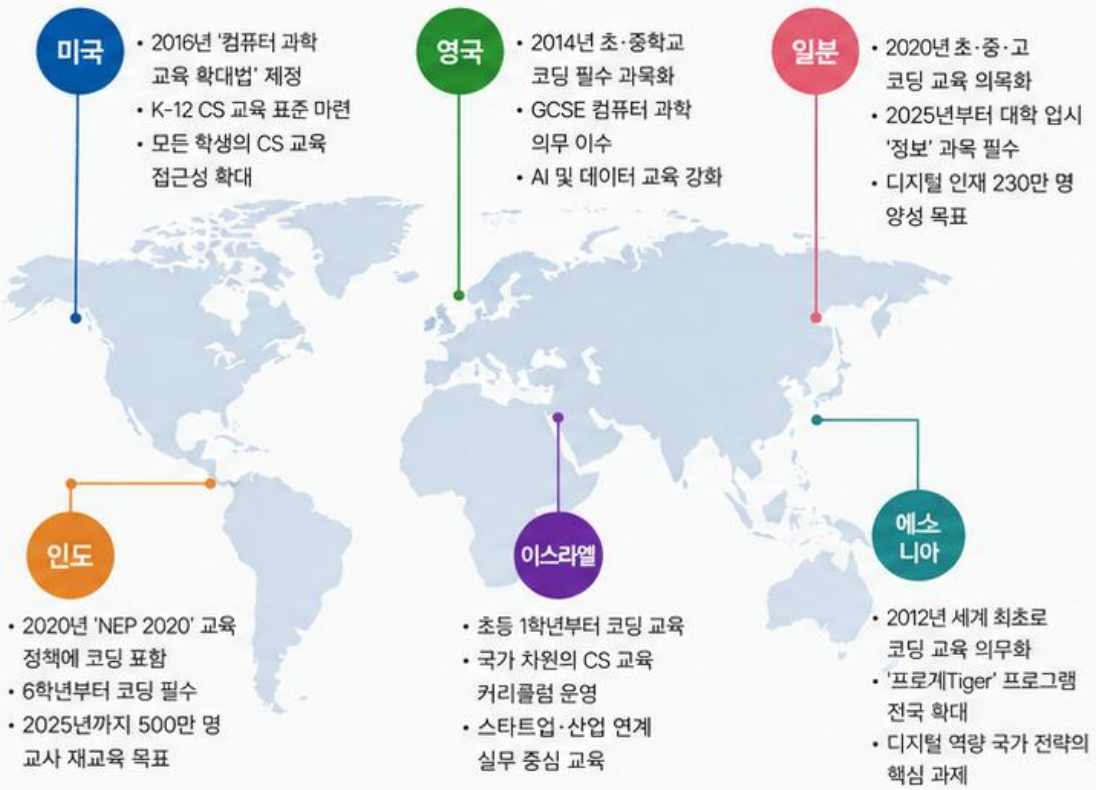


생각 / 요구 ▸ 설계 / 번역 ▸ 작성 / 코드 ▸ 실행 / 결과

★ 프로그래밍은 사람과 컴퓨터 사이의 다리(번역자) 역할을 하여, 우리의 아이디어를 컴퓨터가 수행할 수 있는 명령으로 바꿔줍니다.

코딩 교육

주요 국가 코딩 교육 현황 (2024~2025)



공통 흐름 코딩 교육을 기초 소양으로 규정하고, 초등 저학년부터 필수화하여 디지털 인재 양성과 국가 경쟁력 강화에 집중하고 있습니다.

디지털 인재 양성 주요 내용 (최신)

자료: 교육부, 각국 교육부(2024~2025)

구분	주요 내용	적용 시기
초·중·고 디지털 교육 강화	✓ 코딩 교육 의무화 및 교육과정 개선 ✓ AI·데이터·컴퓨팅 사고력 중심 교육 확대 ✓ 실습 중심 프로젝트 학습 강화	2024~2025년 지속 확대
디지털 영재 육성	✓ 소프트웨어(SW)·AI 영재 교육 확대 ✓ 영재교육원 및 특화 프로그램 확충 ✓ 국제 경진대회 및 산학 연계 강화	2024~2025년 지속 확대
대학·고등교육 혁신	✓ 학·석·박사 통합과정 확대(5년 6개월~11학기) ✓ AI·SW 융합전공 및 마이크로디그리 도입 ✓ 산학협력 및 취업 연계 강화	2023~2025년
산업·사회 연계 및 인프라	✓ 기업·학교 연계 교육(인턴십, 실무 프로젝트) ✓ 디지털 교육 플랫폼 및 학습 인프라 확충 ✓ 지역·계층 간 디지털 교육 격차 해소	2024~2025년 지속 확대
정책·제도 지원	✓ 디지털 인재 양성 종합 계획 추진 ✓ 교원 연수 및 전문성 강화 ✓ 교육 예산 및 지원 체계 강화	2024년부터 지속 추진

코딩 교육은 미래 사회의 필수 역량!
모든 학생이 디지털 세상을 이해하고 창의적으로 문제를 해결할 수 있도록 지원합니다.

파이썬 이란?

역사

- **1989년 크리스마스:** 개발 시작
  (Guido van Rossum)
- **1991년:** 첫 공식 배포

특징

쉽고 직관적인 문법
누구나 쉽게 읽고 이해할 수 있는 문법

객체지향 + 고수준 언어
효율적이고 유연한 프로그래밍 지원

다른 언어(C, C++)보다 배우기 쉬움
프로그래밍 입문에 최적화

강력한 라이브러리와 생태계

데이터 과학, 인공지능, 웹, 게임, IoT 등
다양한 분야를 지원하는 풍부한 라이브러리

Data Science | AI / ML | Web | Game | IoT | ...

객체지향 언어

- 프로그램을 객체(Object) 단위로 구성
- 재사용, 유지보수, 확장성에 유리

고수준 언어

- 하드웨어 세부사항을 몰라도 개발 가능
- 생산성 높고 코드 작성 속도 빠름

왜 많은 개발자가 선택할까?

- 빠른 개발과 유지보수
- 다양한 분야에서 활용 가능
- 커뮤니티와 자료가 풍부



귀도 반 로섬
(Guido van Rossum)

파이썬의 창시자
네덜란드 출신 프로그래머

> *"Life is too short. You need python."*
> (인생은 너무 짧으니 파이썬이 필요해.)

파이썬 이란?

- Python 이름은 영국 코미디 프로그램 *Monty Python's Flying Circus* 에서 유래
  → 재미있고 가벼운 느낌을 주는 이름을 원해 선택

- 단어 자체는 그리스 신화 속 거대한 뱀을 뜻하기도 함
  → 하지만 이는 공식 유래와는 무관



그리스 신화 속 거대한 뱀 '파이톤(Python)'



Python 로고

Python의 특징

1. 간결한 문법

볼필요한 코드가 적고,
간결하고 직관적입니다.

C++

```cpp
#include <iostream>
using namespace std;
int main() {
    cout << "Hello, gabia!";
    return 0;
}
```

Java

```java
public class HelloGabia {
    public static void main(String args[]) {
        System.out.println("Hello, gabia!");
    }
}
```

Python

```python
print("Hello, gabia!")
```

2. 간단한 반복 처리

반복문을 더 쉽고
간결하게 작성할 수 있습니다.

C++

```cpp
#include <iostream>
using namespace std;
int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    for(int i=0; i<5; i++) {
        cout << arr[i] << endl;
    }
    return 0;
}
```

Java

```java
public class HelloGabia {
    public static void main(String args[]) {
        int arr[] = {1, 2, 3, 4, 5};
        for(int i=0; i<5, i++) {
            System.out.println(arr[i]);
        }
    }
}
```

Python

```python
arr = [1, 2, 3, 4, 5]
for el in arr:
    print(el)
```

3. 가독성 높은 코드

들여쓰기를 기반으로 하여
코드 구조가 명확하고
가독성이 뛰어납니다.

C++

```cpp
#include <iostream>
using namespace std;
int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    for(int i=0; i<5; i++) {
        cout << arr[i] << endl;
    }
    return 0;
}
```

Java

```java
public class HelloGabia {
    public static void main(String args[]) {
        int arr[] = {1, 2, 3, 4, 5};
        for(int i=0; i<5, i++) {
            System.out.println(arr[i]);
        }
    }
}
```

Python

```python
arr = [1, 2, 3, 4, 5]
for el in arr:
    print(el)
```

Python의 특징



- **플랫폼** : 프로그램이 실행되는 환경 (운영체제: Windows, MacOS, Linux 등)



- **플랫폼 독립성** : 한 번 작성한 코드가 여러 운영체제에서 실행 가능



- **인터프리터** : 코드를 한 줄씩 번역하며 실행

  → 결과를 바로 확인 가능



- **플랫폼 독립적인 인터프리터 언어**

Python의 특징

사람



사람이 알아본다

의도와 생각을 표현

프로그래밍 언어

```
# Python 예제
name = input("이름이 뭐예요? ")
print(f"안녕하세요, {name}님!")

# 사람이 이해할 수 있는
# 프로그래밍 언어입니다.

안녕하세요, Python님!
```

사람이 이해할 수 있는 언어

Python은 쉬운 문법으로
문제를 표현한다

C, C++

미리 해놓는다
= 컴파일러

번역

그때 그때 번역한다
= 인터프리터

자바스크립트, 파이썬, 루비

기계어

```
01001011 01100101 01110010 01101110
01100101 01101100 00100000 01101110
01100101 01110100 01100011 11101100
00100000 01110101 01101000 01101111
01101001 00100000 01100101 01100010
01110110 01100101 01101010 01100110
01101110 00100000 01101010 01110010
01011111 00100000 01111010 01110010
```

0,1로 이루어진 언어

기계가 이해할 수 있는 언어

기계



기계가 알아본다

명령을 실행한다

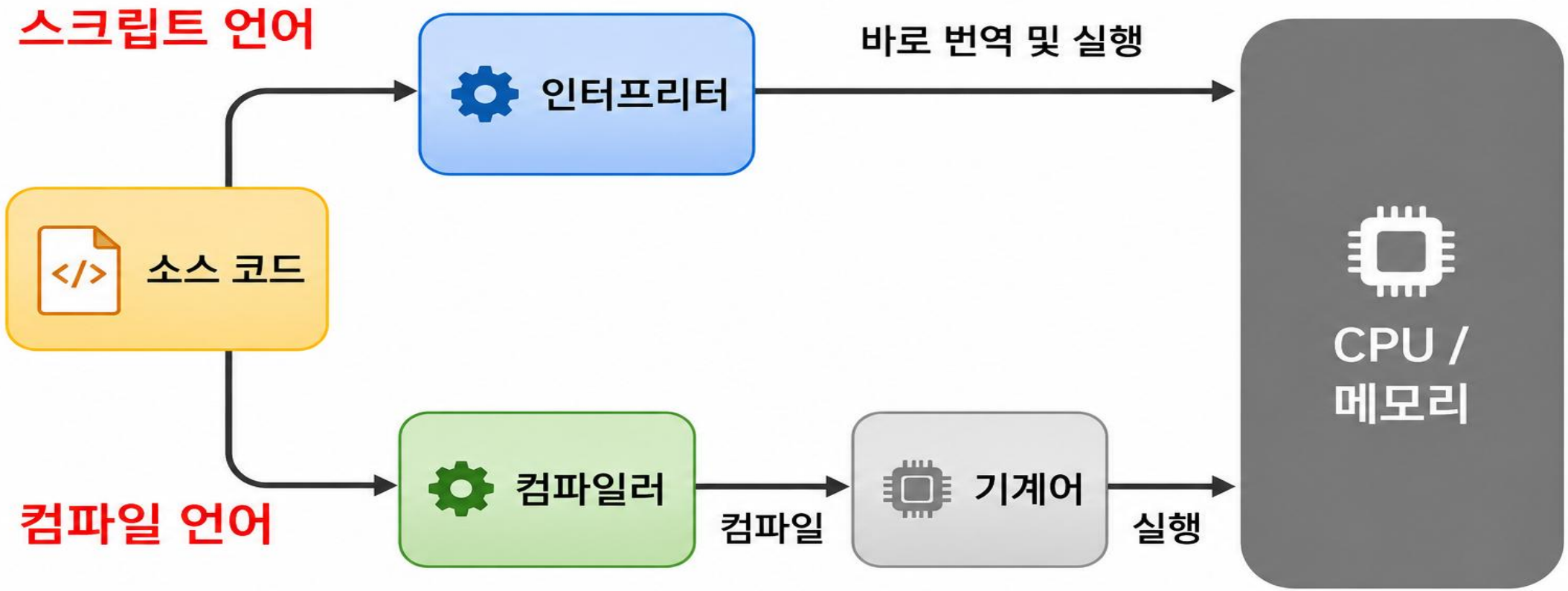


Python은 **인터프리터 방식**으로 동작하여, 쉽게 작성하고 바로 실행할 수 있는 언어입니다.

컴파일러와 인터프리터 비교

특징	컴파일러	인터프리터
번역 방법	프로그램 전체 번역	실행되는 줄(라인) 단위 번역
장점	한 번 컴파일한 후에는 매번 빠른 시간 내에 전체 실행 가능	번역 과정이 비교적 간단하고 대화형 언어에 편리함
단점	프로그램의 일부를 수정하는 경우에도 전체 프로그램을 다시 컴파일해야 함	실행할 때마다 매번 기계어로 바꾸는 과정을 다시 수행해야 하므로 항상 인터프리터가 필요함
출력물	목적 코드	즉시 실행
언어 종류	FORTRAN, COBOL, C 등	BASIC 등

[참고] 프로그램의 동작 과정



Python의 특징

객체지향 언어

객체 = 속성 + 행동



속성	행동
색상	달리다

동적 타이핑

실행 시점에 타입 결정

10 → 'Hello'

컴퓨터와 인간 중 어디에 가까운가?



Machine code





Java



Ruby



저수준 언어
Low-level

Assembly language



C++



Python

고수준 언어
High-level

저수준 언어
Low-level

- 하드웨어에 가까움
- 실행 속도 빠름
- 사용이 어렵고 복잡함

- 사람의 언어에 가까움
- 개발 생산성 높음
- 실행 속도는 상대적으로 느림

고수준 언어
High-level

자바 VS 파이썬

Dynamic Typing

: Java는 미리 변수 타입을 선언해야 하지만, Python은 실행 시점에 자동으로 결정

예제 1 (숫자 연산)

Java

```java
public class Main {
  public static void main(String[] args) {
    int a = 10, b = 20;
    int result = a + b;
    System.out.println("The result a+b = " + result);
  }
}
```

Python

```python
a = 10
b = 20

print(f"The result a+b = {a+b}")
```

Output:

```
The result a+b = 30
```

```
The result a+b = 30
```

예제 2 (문자열 출력)

Java

```java
public class Main {

  public static void main(String[] args) {

     System.out.println("Hello World");

  }

}
```

Python

```python
print("Hello World")
```

Output:

```
Hello World
```

```
Hello World
```

자바 (JAVA)

: 정적 타이핑 언어, 변수 타입을 미리 선언해야 함,
C는 절차지향 + 정적 타이핑.

파이썬 (Python)

: 동적 타이핑 언어, 변수 타입을 선언하지 않고
실행 시 자동 결정

파이썬 활용도

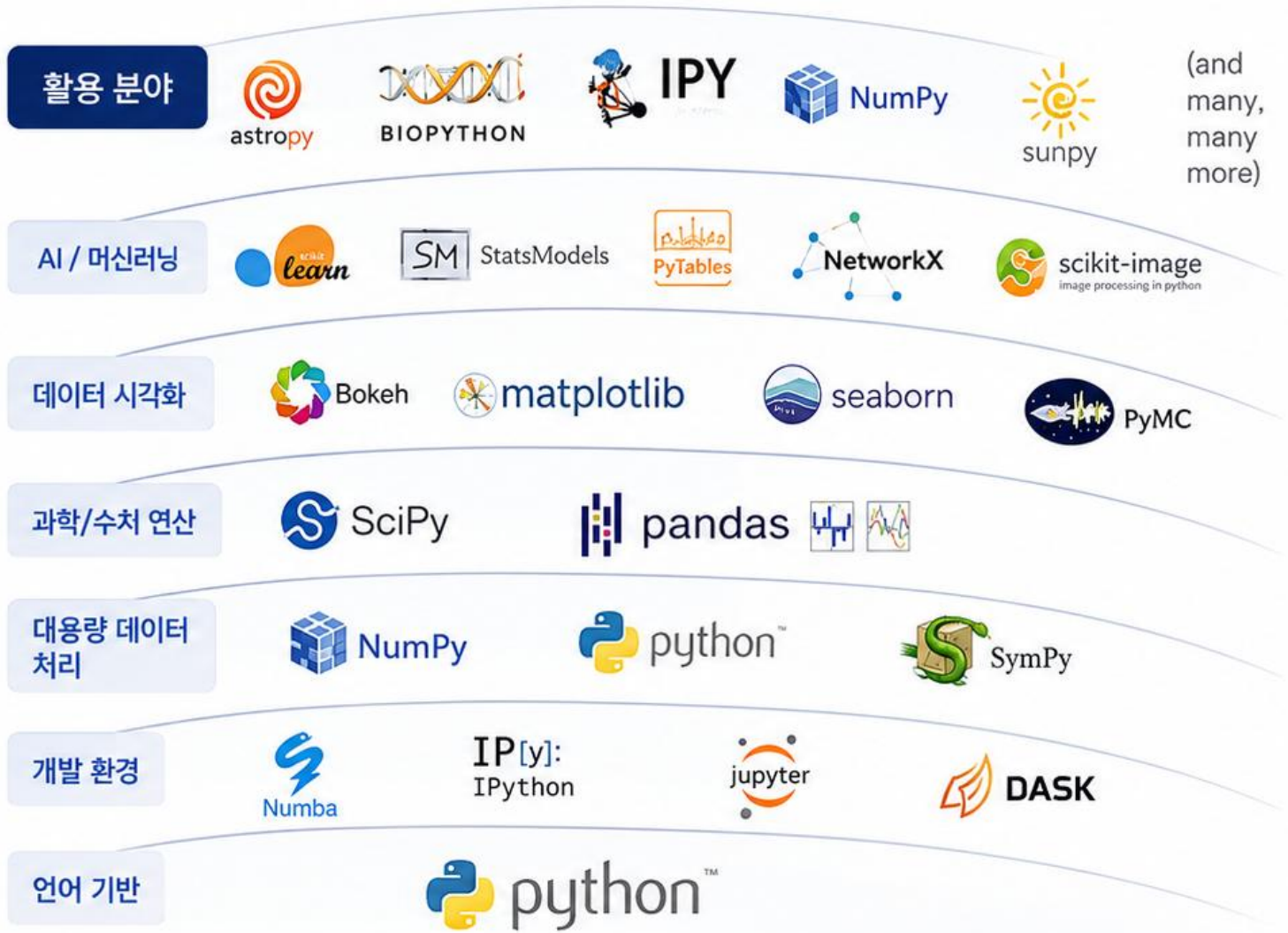
데이터분석
: pandas, numpy ···

데이터시각화
: matplotlib, seaborn ···

AI
: sklearn, tensorflow, pytorch ···

대용량 데이터처리
: pyspark, pandas ···

백엔드 개발
: django, flask, fastapi ···



개발 환경 개요

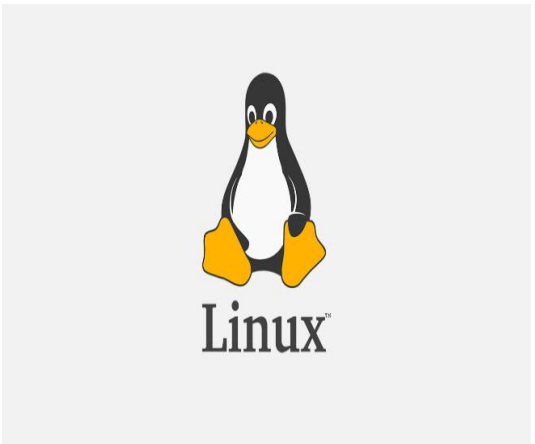
- **프로그램**을 **작성**하고, **실행** 시키는 환경
- 일반적으로 **<코딩 환경>** 이라고 부름
- 개발환경을 **결정**

1. **운영 체재** (Operating System, OS)
   컴퓨터 하드웨어를 관리하고 프로그램 실행을 지원
2. **Python interpreter**
   파이썬 코드를 해석하고 실행하는 프로그램
3. **코드 편집기** (Editor)
   코드를 작성하고 저장하는 도구



운영 체제 - Operation System





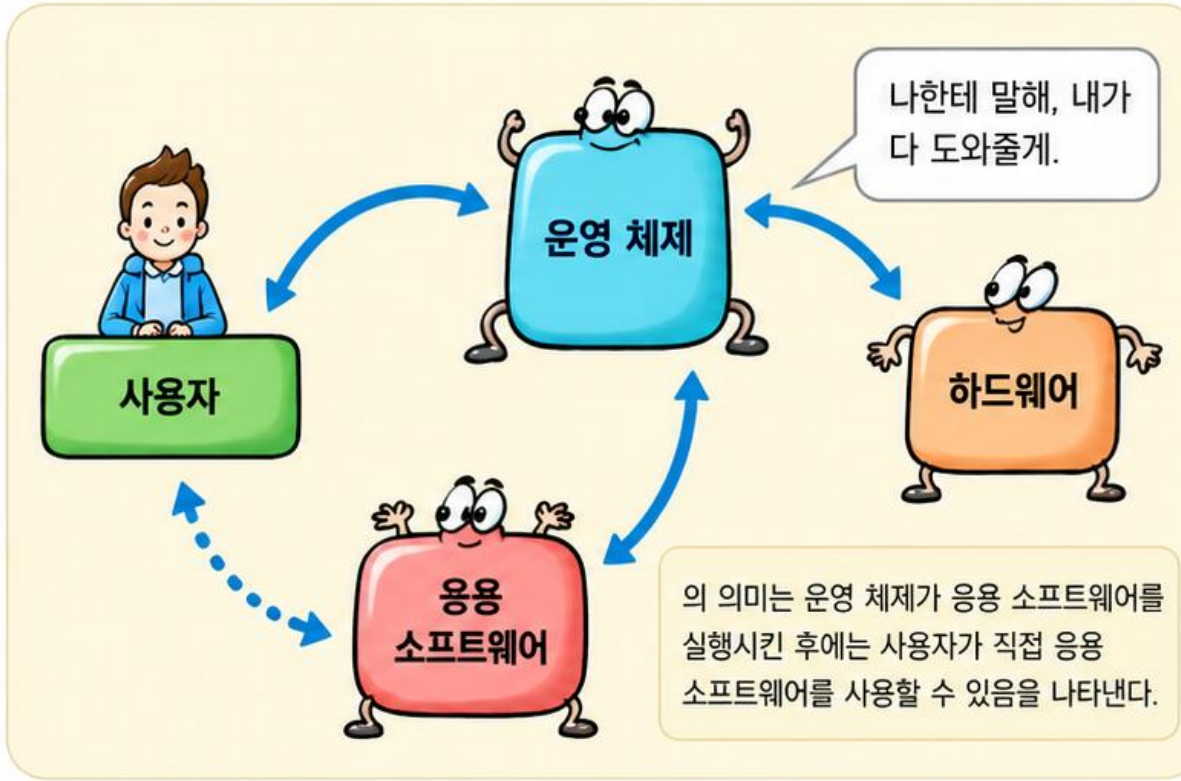


장점	친숙함 초기에 쉬움	모듈 설치 쉬움 무료, 참고문서 많음	모듈 설치 쉬움 참고문서 많음
단점	모듈 설치 어려움 / 유료	OS 자체 사용이 어려움	비쌈

운영 체제 - Operation System

하드웨어 vs 소프트웨어





하드웨어

- 컴퓨터 본체, 모니터, 마우스, 키보드, 하드디스크, CPU, RAM 등

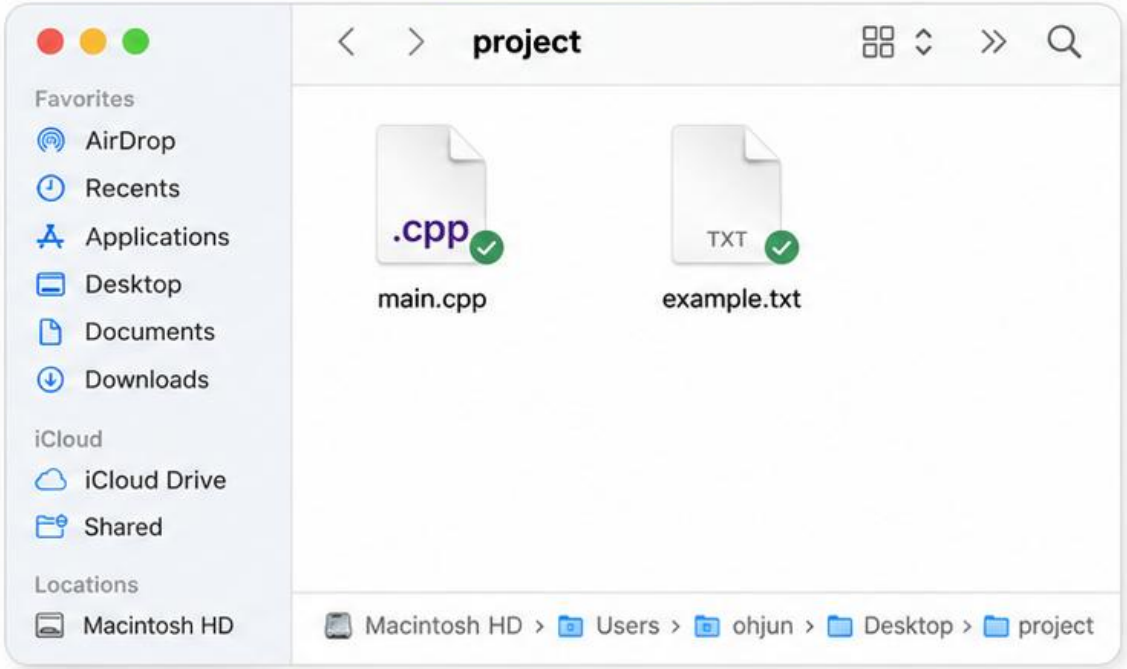
소프트웨어

- 한글, 엑셀, 파워포인트, 크롬 등

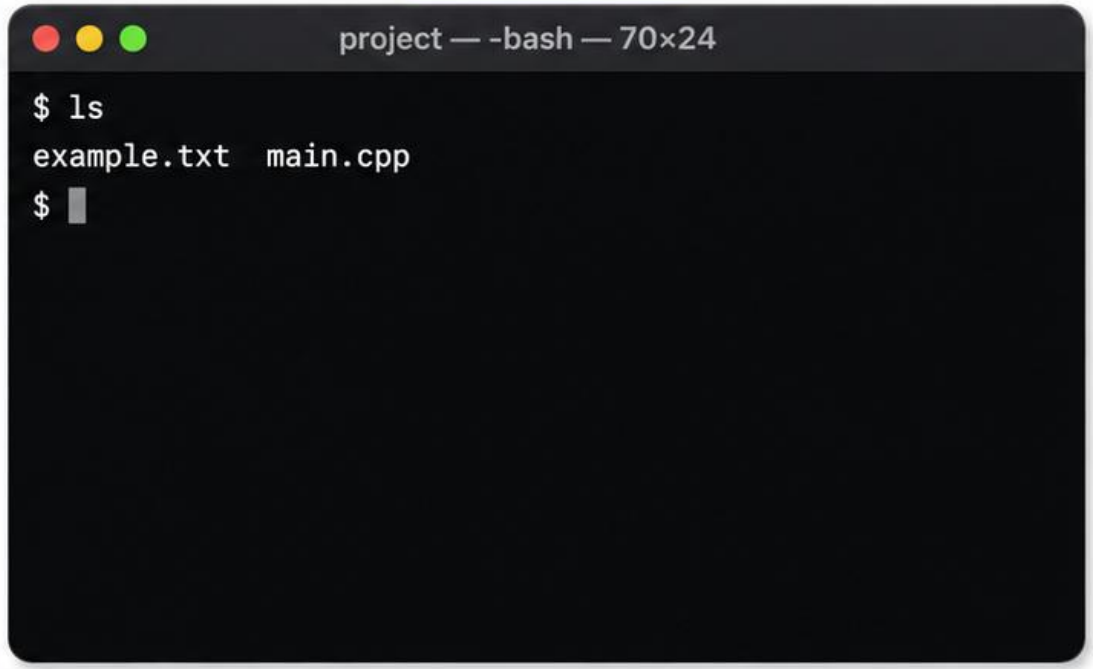
Terminal

- 터미널 - Mouse가 아닌 키보드로 명령을 입력 프로그램 실행

GUI



CLI



Graphical User Interface

마우스와 아이콘 등을 사용하여
직관적으로 작업을 수행하는 환경

Command Line Interface

명령어를 직접 입력하여
컴퓨터와 상호작용하는 환경

Command Line Interface

Graphic User Interface(GUI)와 달리 Text를 사용하여 컴퓨터에 명령을 입력하는 인터페이스 체계

Windows

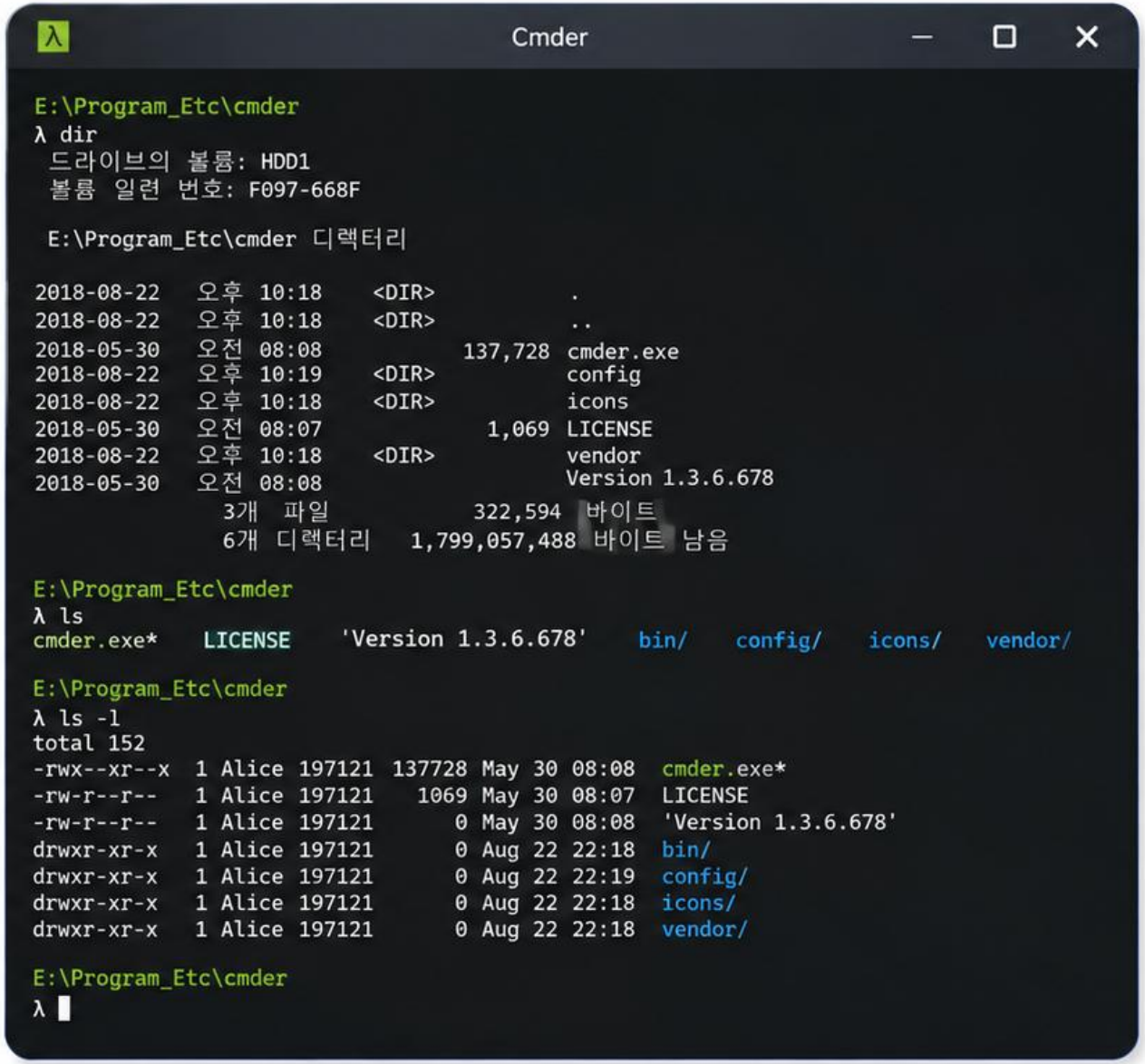
- CMD Window, Windows Terminal

Mac, Linux

- Terminal

원도우 cmder도 권장

https://cmder.net/



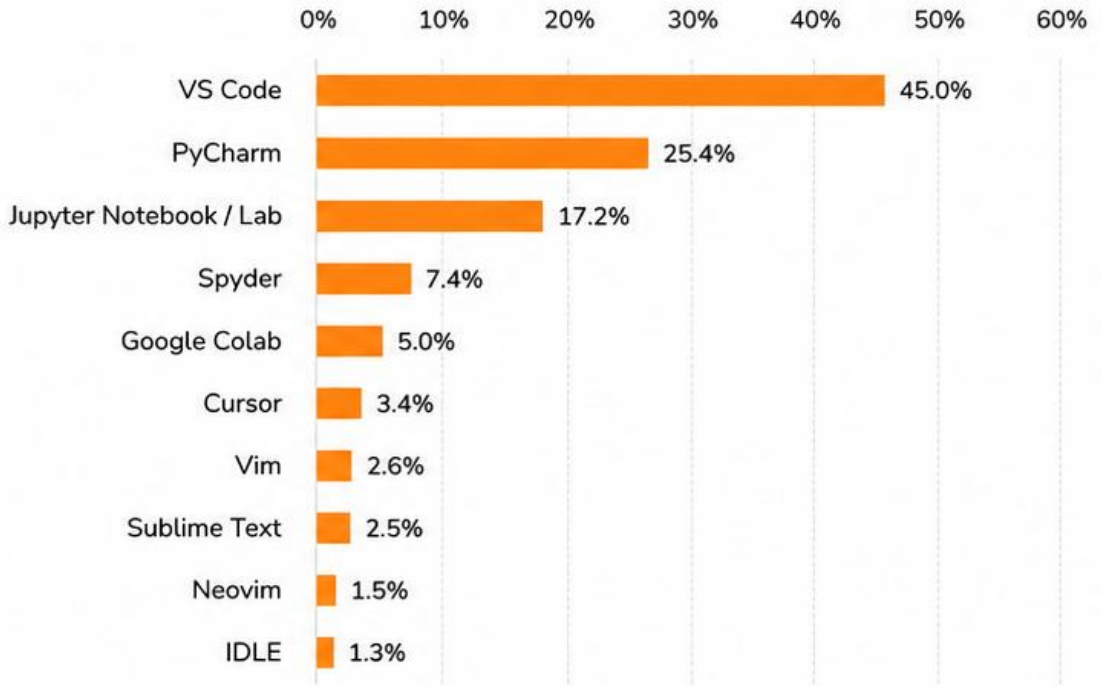
코드 편집기(Editor)

- **파이썬 코드도 일종의 문서**
  - 한글, 워드처럼 코드를 입력할 문서 편집기가 필요함.
  - text 타입의 문서를 저장하는 모든 편집기를 사용가능.

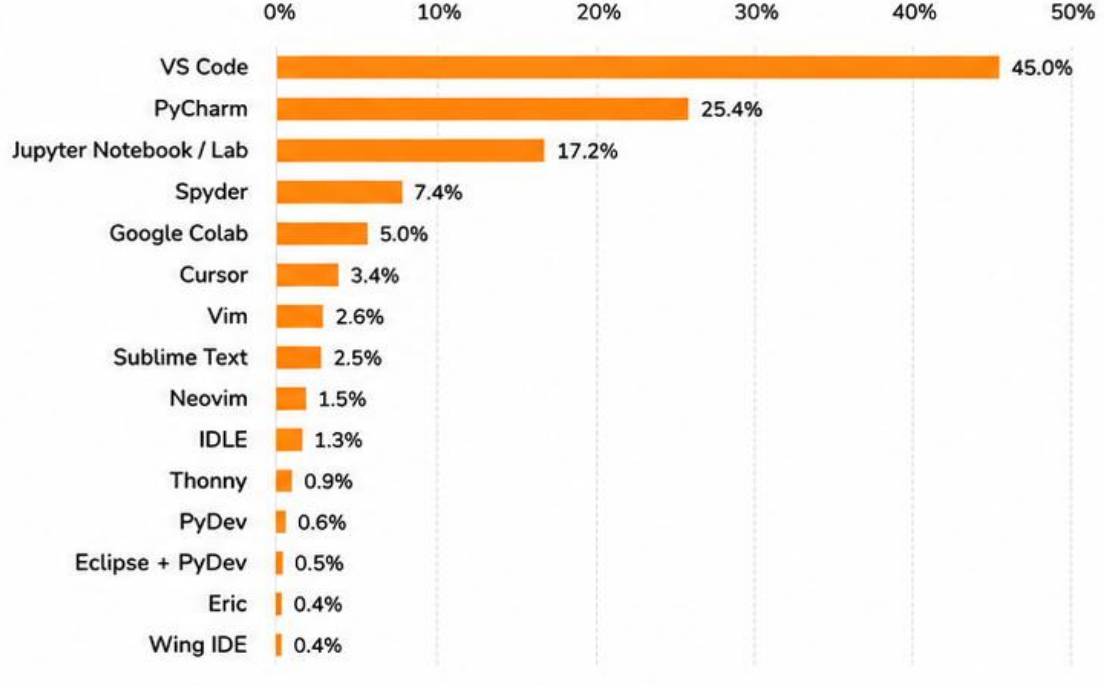
종류	설명	다운로드
메모장	원도우의 기본 문서 편집도구	시작 -> 메모장(Windows)
VI editor	리눅스의 기본 문서 편집도구	https://www.vim.org/download.php
Sublime Text	프로그래밍에 특화된 문서 편집도구	https://www.sublimetext.com/
Atom		https://atom.io/
VS code		https://code.visualstudio.com/
Pycharm	다양한 기능을 갖춘 파이썬 전용 개발 도구	https://www.jetbrains.com/pycharm/

파이썬 개발환경

2025년 기준, 파이썬 개발자들이 가장 많이 사용하는 IDE 및 에디터



※ Source: JetBrains Developer Ecosystem Survey 2025



※ Source: JetBrains Developer Ecosystem Survey 2025

VS Code가 가장 높은 점유율을 차지하고 있으며, PyCharm과 Jupyter Notebook이 그 뒤를 잇고 있습니다.

파이썬 개발환경



- 손쉬운 패키지 설치
- 가상 환경을 통한 패키지 버전관리가 용이

파이썬 개발환경



- web을 통한 접근 가능
- Coding 결과를 실시간으로 확인 가능
- 자동완성 기능
- 다양한 언어 지원
- Markdown을 통해 문서화 가능

ANACONDA 설치

https://www.anaconda.com/products/individual

Anaconda Installers

Windows

Python 3.8

64-Bit Graphical Installer (466 MB)

32-Bit Graphical Installer (397 MB)

MacOS

Python 3.8

64-Bit Graphical Installer (462 MB)

64-Bit Command Line Installer (454 MB)

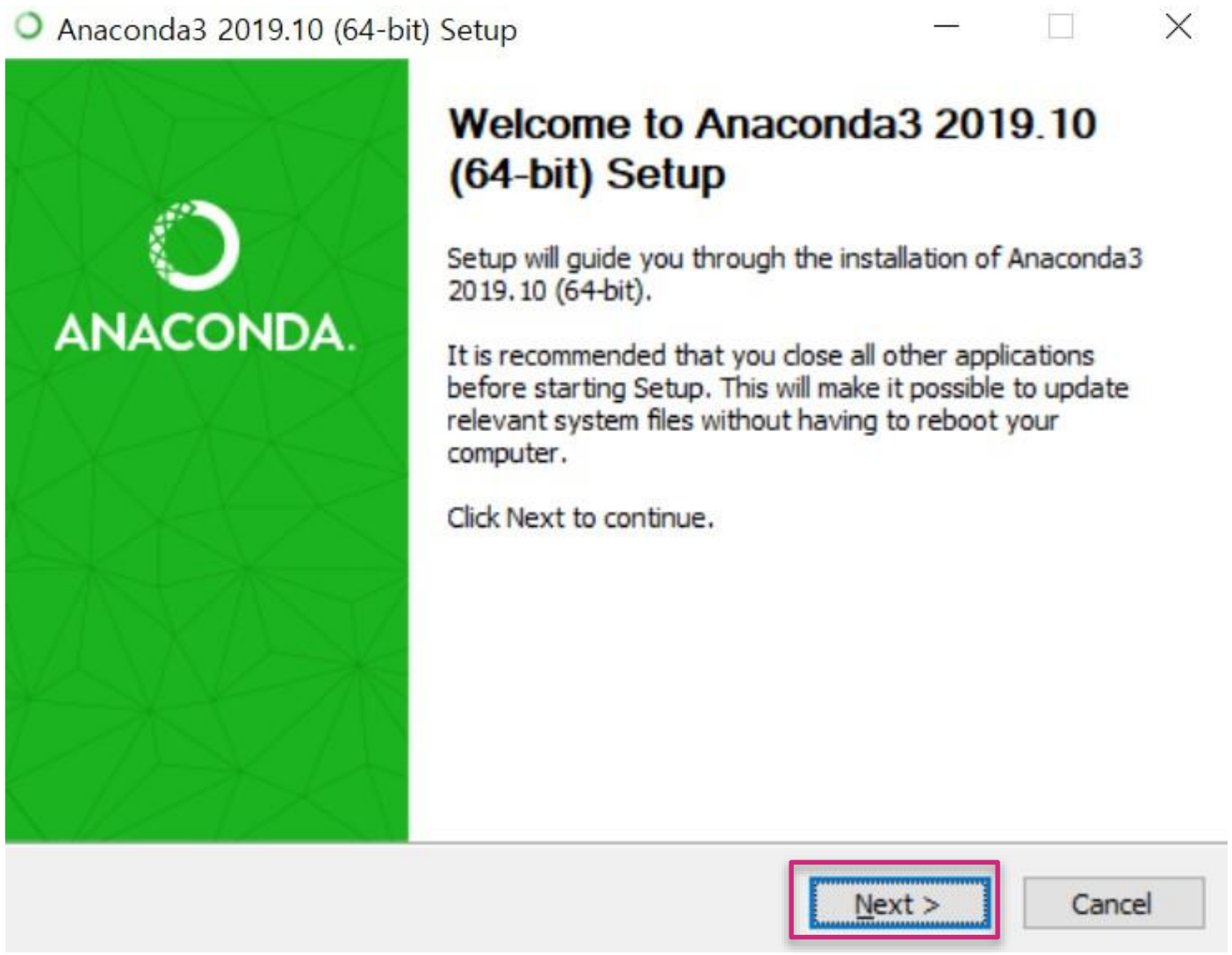
Linux

Python 3.8

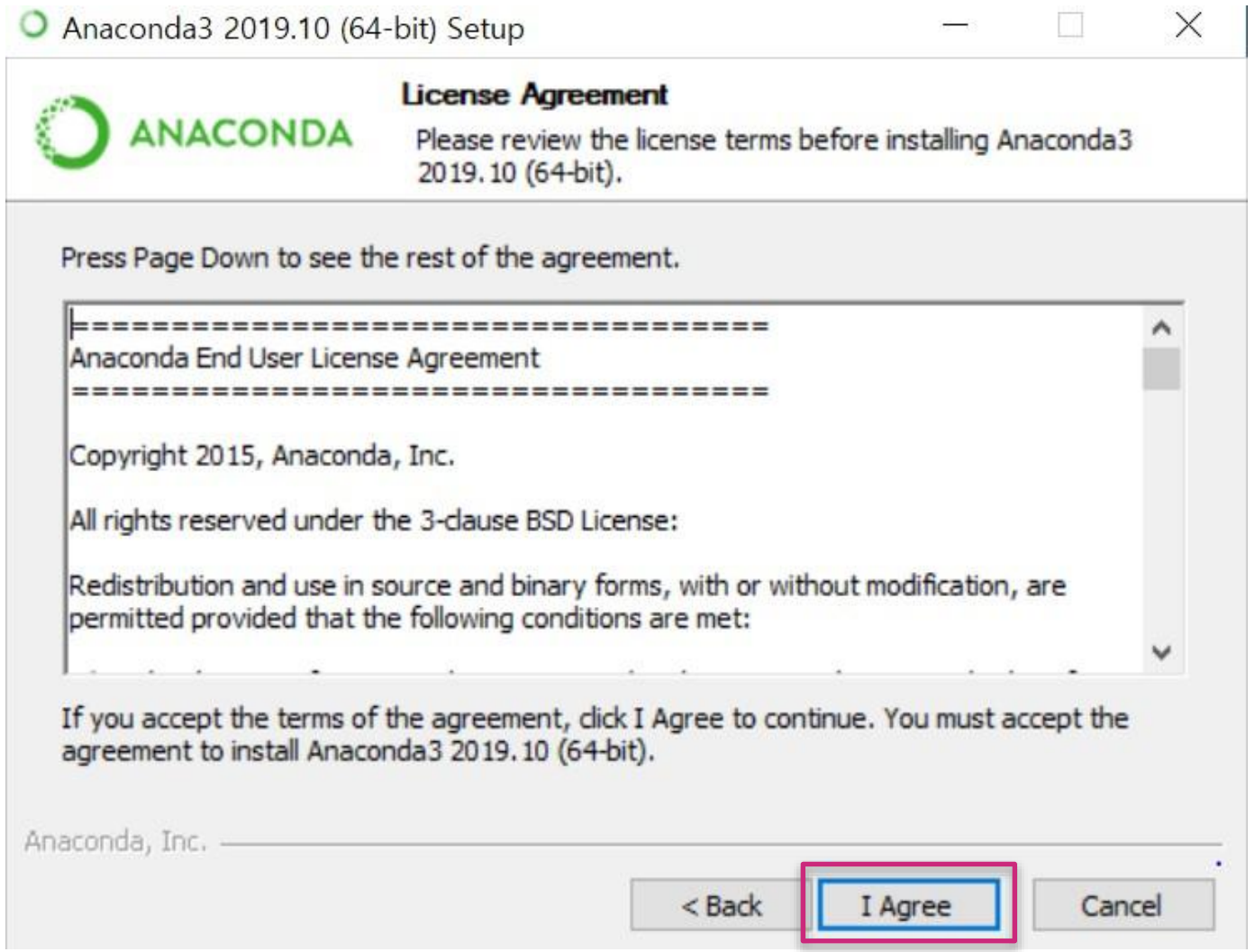
64-Bit (x86) Installer (550 MB)

64-Bit (Power8 and Power9) Installer (290 MB)

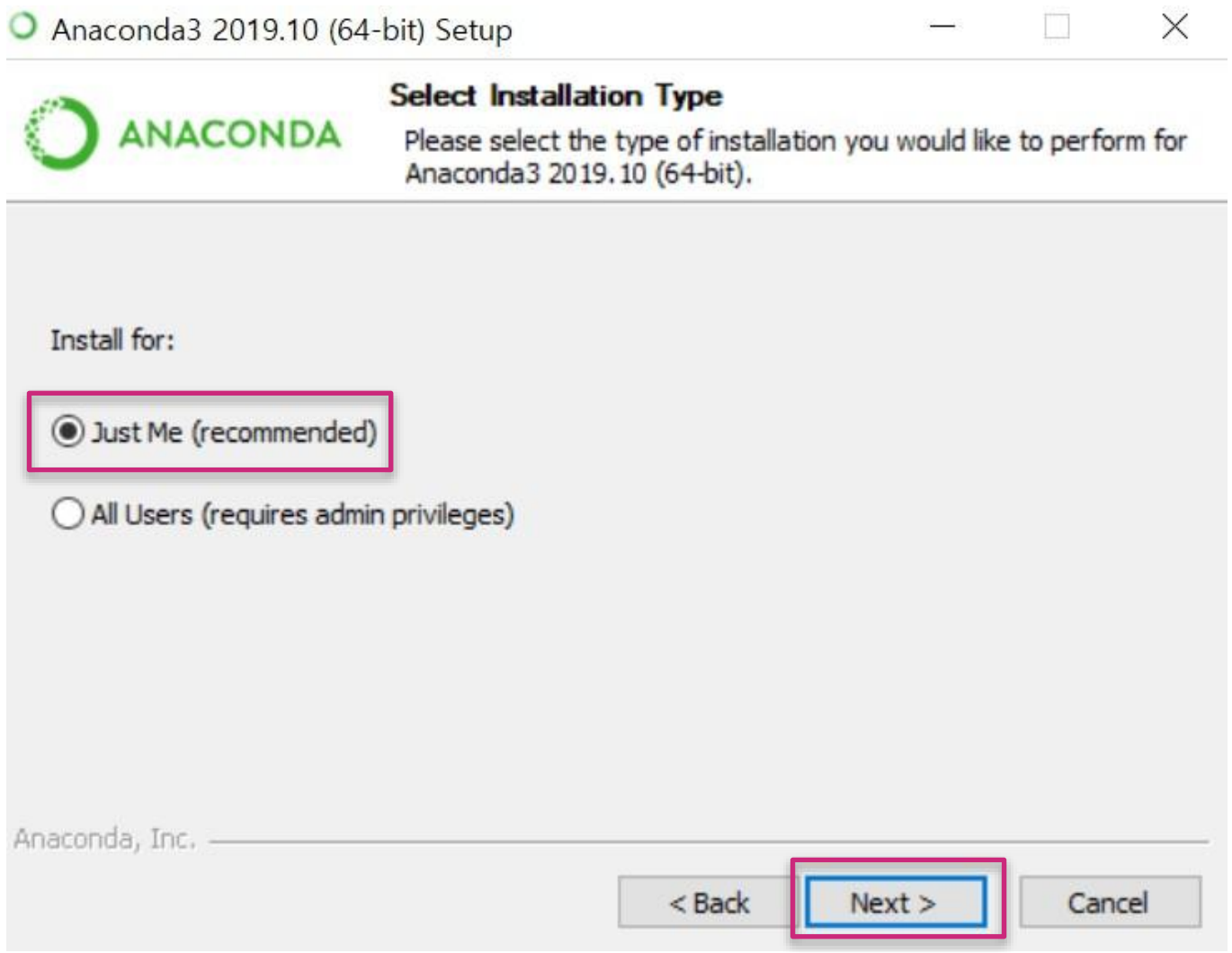
ANACONDA 설치


Anaconda3 2019.10 (64-bit) Setup
ANACONDA.
Welcome to Anaconda3 2019.10 (64-bit) Setup
Setup will guide you through the installation of Anaconda3 2019.10 (64-bit).
It is recommended that you close all other applications before starting Setup. This will make it possible to update relevant system files without having to reboot your computer.
Click Next to continue.
Next >
Cancel

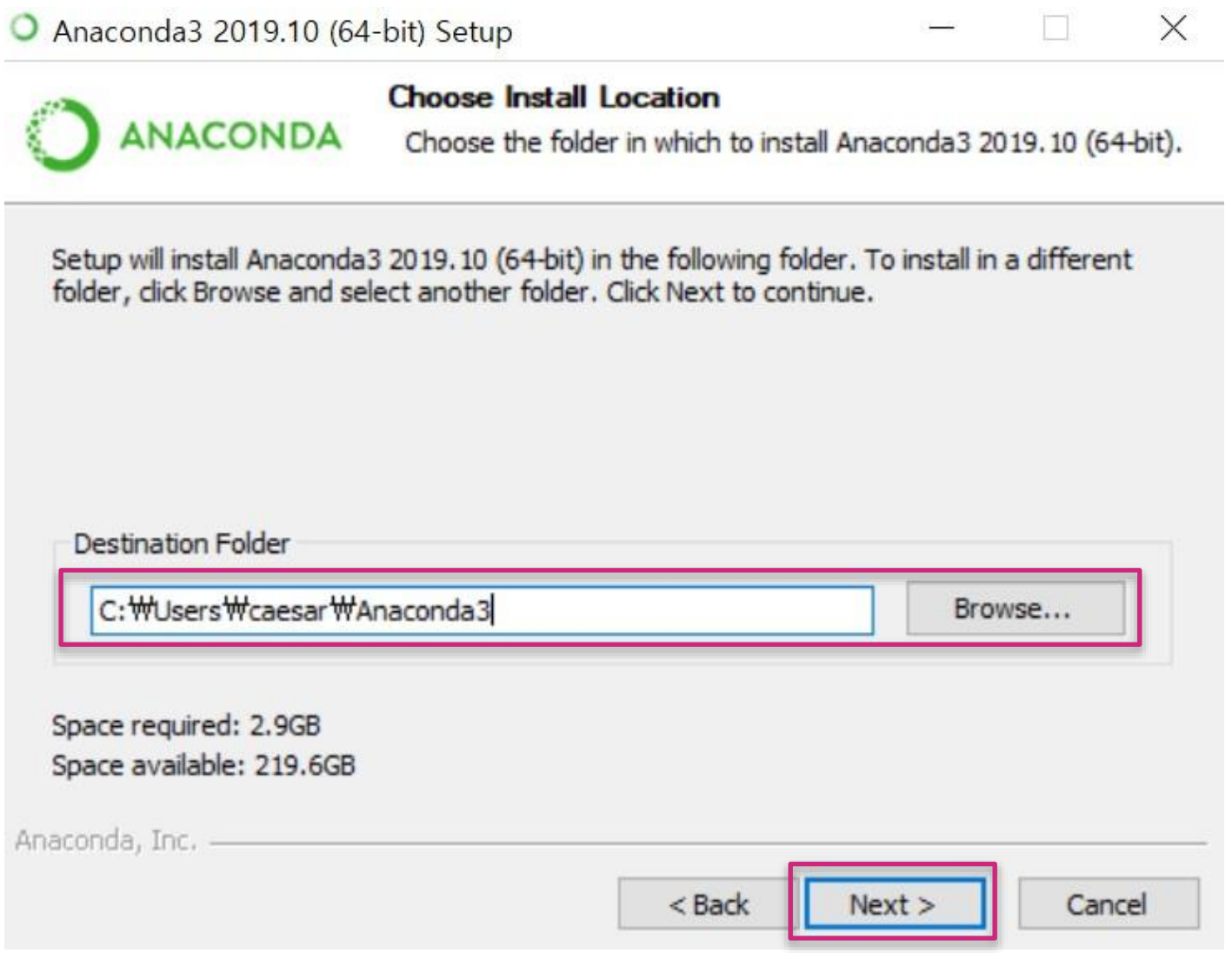
ANACONDA 설치


Anaconda3 2019.10 (64-bit) Setup
ANACONDA
License Agreement
Please review the license terms before installing Anaconda3 2019.10 (64-bit).
Press Page Down to see the rest of the agreement.
Anaconda End User License Agreement
Copyright 2015, Anaconda, Inc.
All rights reserved under the 3-clause BSD License:
Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:
If you accept the terms of the agreement, click I Agree to continue. You must accept the agreement to install Anaconda3 2019.10 (64-bit).
Anaconda, Inc.
< Back
I Agree
Cancel

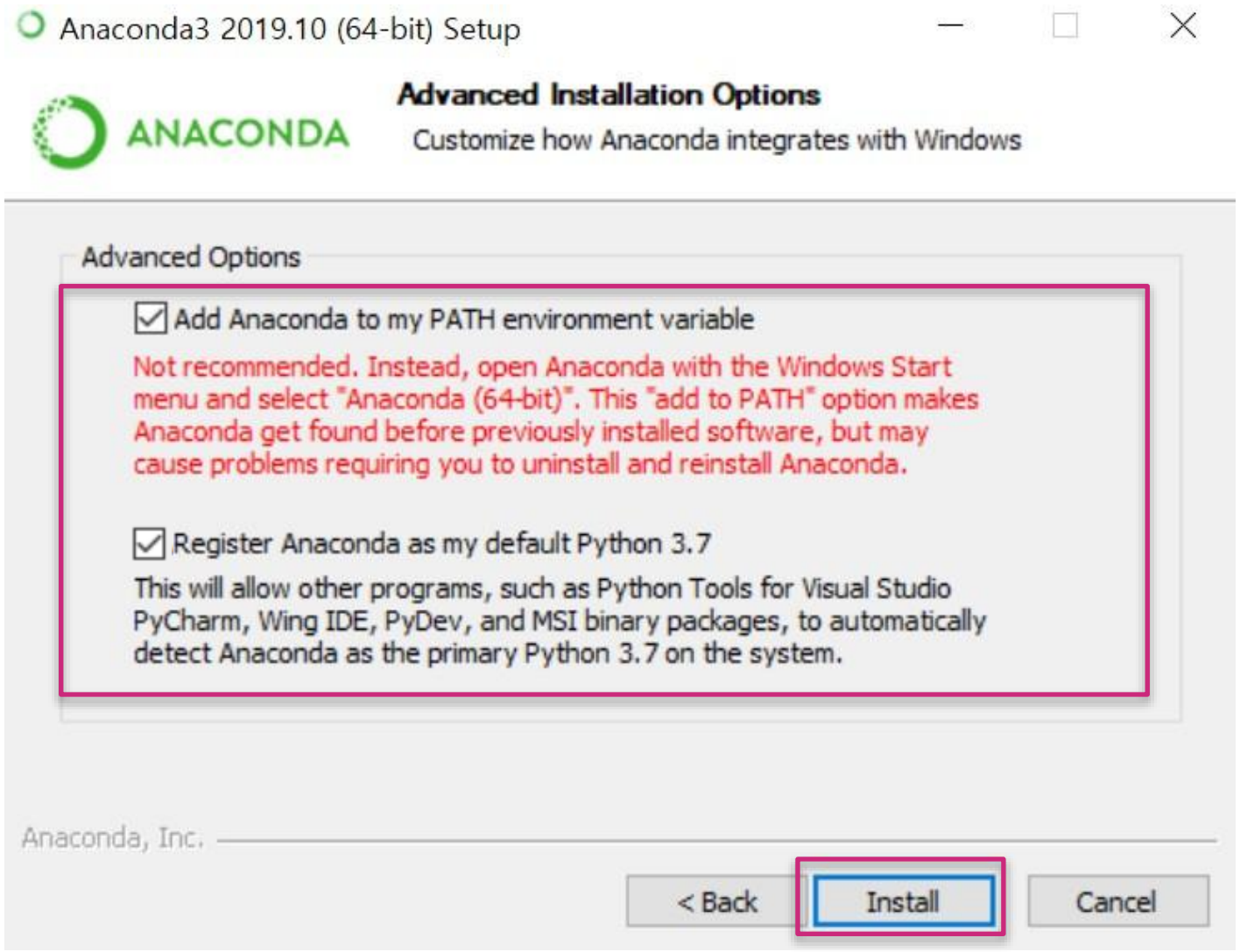
ANACONDA 설치


Anaconda3 2019.10 (64-bit) Setup
ANACONDA
Select Installation Type
Please select the type of installation you would like to perform for Anaconda3 2019.10 (64-bit).
Install for:
Just Me (recommended)
All Users (requires admin privileges)
Anaconda, Inc.
< Back
Next >
Cancel

ANACONDA 설치


Anaconda3 2019.10 (64-bit) Setup
ANACONDA
Choose Install Location
Choose the folder in which to install Anaconda3 2019.10 (64-bit).
Setup will install Anaconda3 2019.10 (64-bit) in the following folder. To install in a different folder, click Browse and select another folder. Click Next to continue.
Destination Folder
C:₩Users₩caesar₩Anaconda3
Browse...
Space required: 2.9GB
Space available: 219.6GB
Anaconda, Inc.
< Back
Next >
Cancel

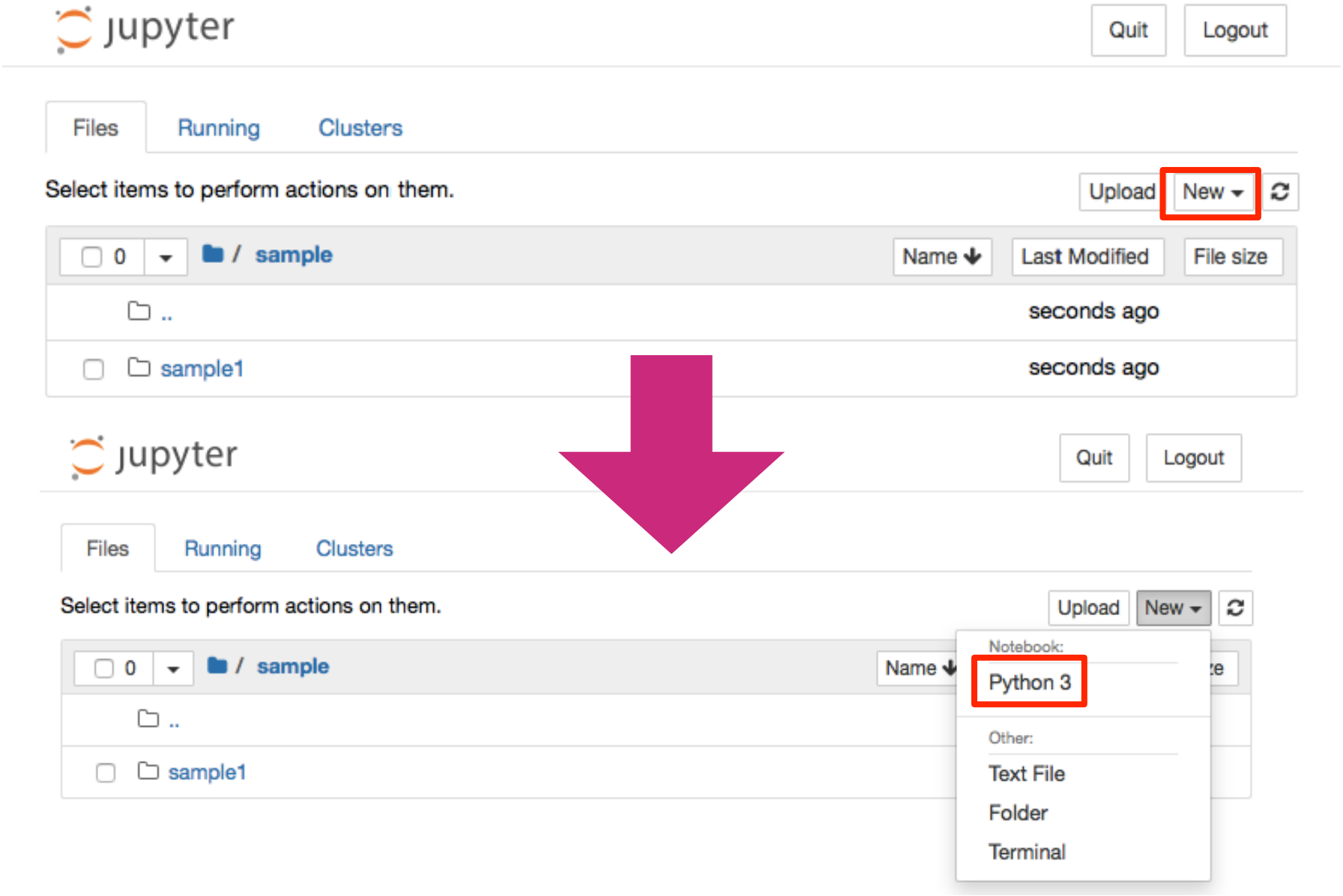
ANACONDA 설치


Anaconda3 2019.10 (64-bit) Setup
ANACONDA
Advanced Installation Options
Customize how Anaconda integrates with Windows
Advanced Options
Add Anaconda to my PATH environment variable
Not recommended. Instead, open Anaconda with the Windows Start menu and select "Anaconda (64-bit)". This "add to PATH" option makes Anaconda get found before previously installed software, but may cause problems requiring you to uninstall and reinstall Anaconda.
Register Anaconda as my default Python 3.7
This will allow other programs, such as Python Tools for Visual Studio PyCharm, Wing IDE, PyDev, and MSI binary packages, to automatically detect Anaconda as the primary Python 3.7 on the system.
Anaconda, Inc.
< Back
Install
Cancel

Jupyter Notebook 사용하기



Jupyter Notebook 사용하기



Jupyter Notebook 사용하기

유용한 단축키

In [ ]:
```
# Edit mode : Enter
# Command mode : ESC
```

In [ ]:
```
# 셀 추가 (Command mode에서)
# 현재 셀 위로 : a
# 현재 셀 아래로 : b
```

In [ ]:
```
# 셀 삭제 (Command mode에서) : dd
```

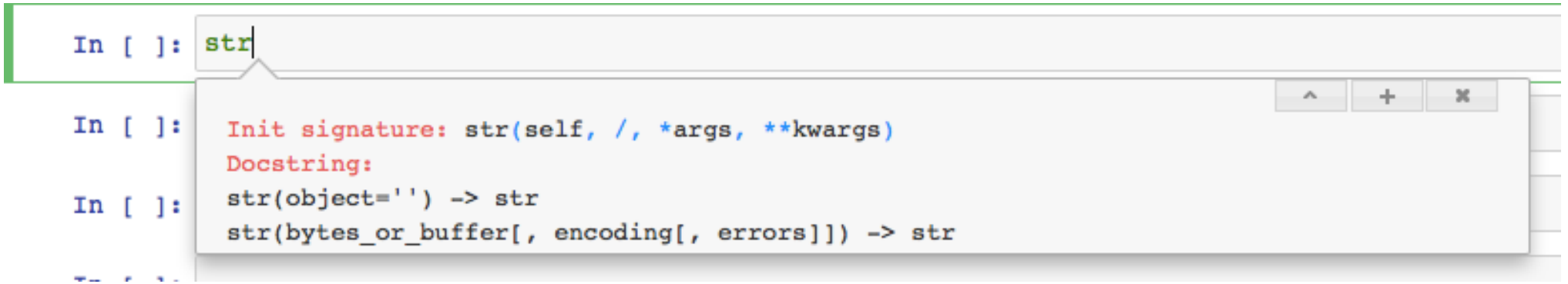
In [ ]:
```
# 선택한 셀을
# 복사하기 : c
# 잘라내기 : x
```

In [ ]:
```
# 셀 붙여넣기
# 선택한 셀 위로 : Shift + v
# 선택한 셀 아래로 : v
```

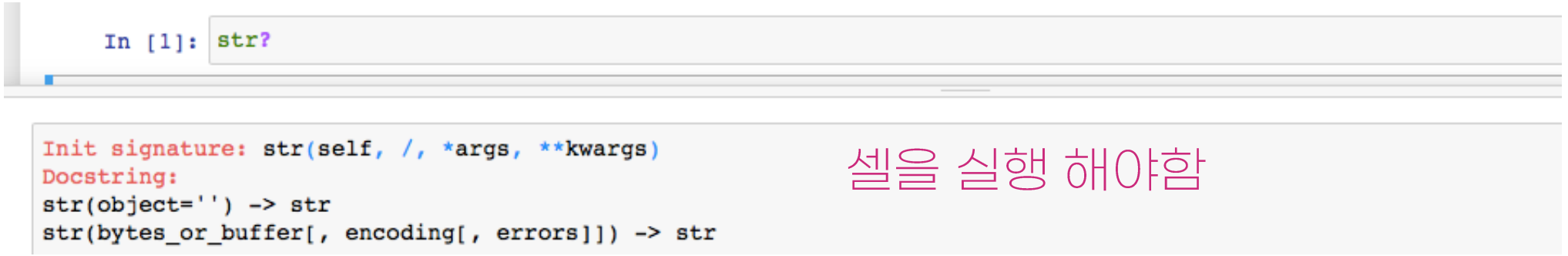
In [ ]:
```
# 셀에 라인 추가하기 (Command mode에서) : l
```

Jupyter Notebook 사용하기

- 함수나 변수에 대한 설명 1) : Shift + Tab



- 함수나 변수에 대한 설명 2) : ?



셀을 실행 해야함

Jupyter Notebook 사용하기

Jupyter 맛보기

markdown

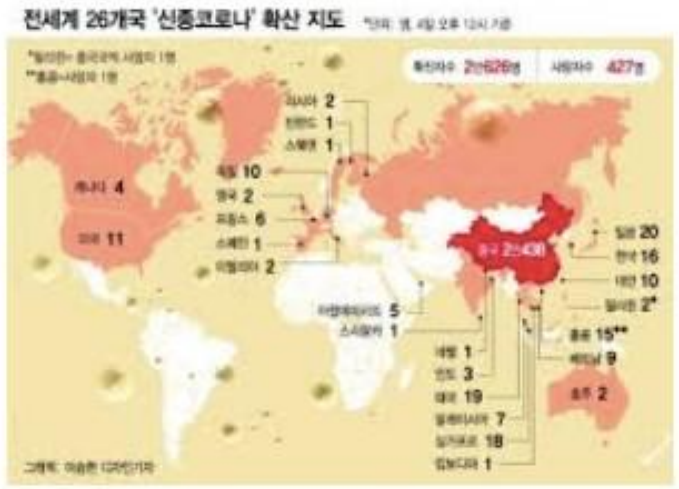
문서처럼 작성할 수도 있습니다.

In [2]:
```
print("당연히 코딩도 할 수 있습니다.")
```

당연히 코딩도 할 수 있습니다.

In [7]:
```
from IPython.display import Image
Image(url="
```

Out[7]:



Google Colab 사용법

1. Google Colab이란?

풀 네임은 Google Colaboratory(Google Colab)

2. Google Drive + Jupyter Notebook

- Google Drive처럼 협업 가능(동시에 수정 가능)
- https://colab.research.google.com/ 로 접속 시 사용 가능

3. 컴퓨터 사양 (2025년 기준)

OS	Ubuntu 20.04 LTS
CPU	2 vCPU (Xeon 계열), 2.2 ~ 2.3GHz 수준
메모리	약 13GB
GPU	T4, V100, L4 등 무작위 제공
GPU/TPU	무료로 사용 가능 (보장되지 않고 자원 상황에 따라 제한)

4. 세션 시간

세션 최대 약 12시간

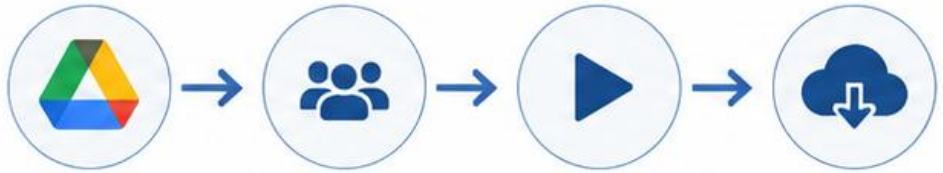
(유휴 시 더 빨리 종료될 수 있음)

5. 요금제

pro/pro+ 요금제 사용 시

더 긴 세션, 더 강력한 GPU 이용 가능

Google Colab 사용 흐름



1. Google Drive에서 새 노트북 생성
2. 실시간으로 협업 및 수정
3. 코드 실행
4. 결과 저장 및 공유

TIP

무료로 강력한 GPU/TPU를 사용할 수 있어 머신러닝, 딥러닝 학습에 매우 유용합니다.

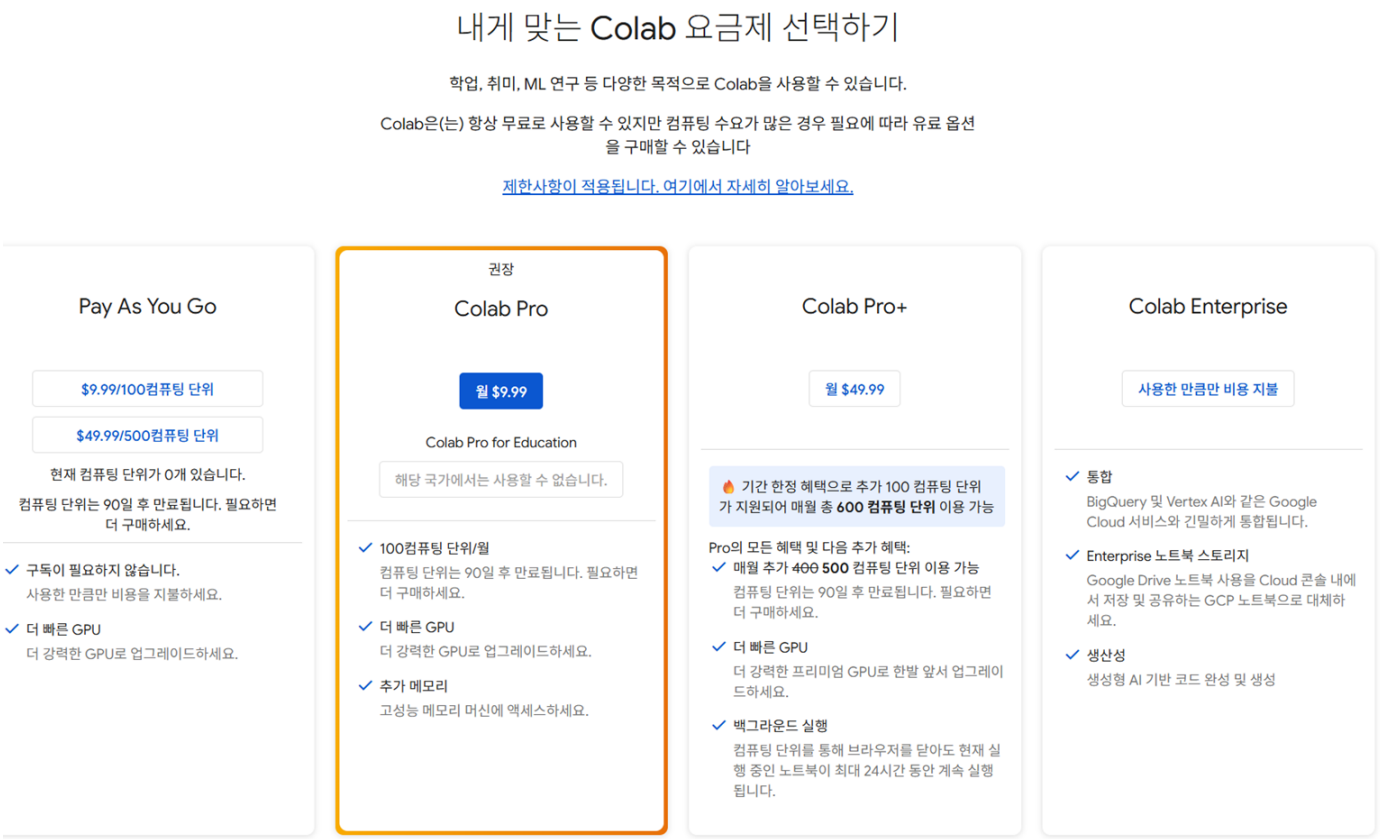
Google Colab

OS 확인

- !cat /etc/issue.net

하드웨어 사양

1. CPU
   - !cat /proc/cpuinfo
2. Memory
   - !cat /proc/meminfo
3. Disk
   - !df -h
4. GPU
   - !nvidia-smi



Google Colab 런타임 방지

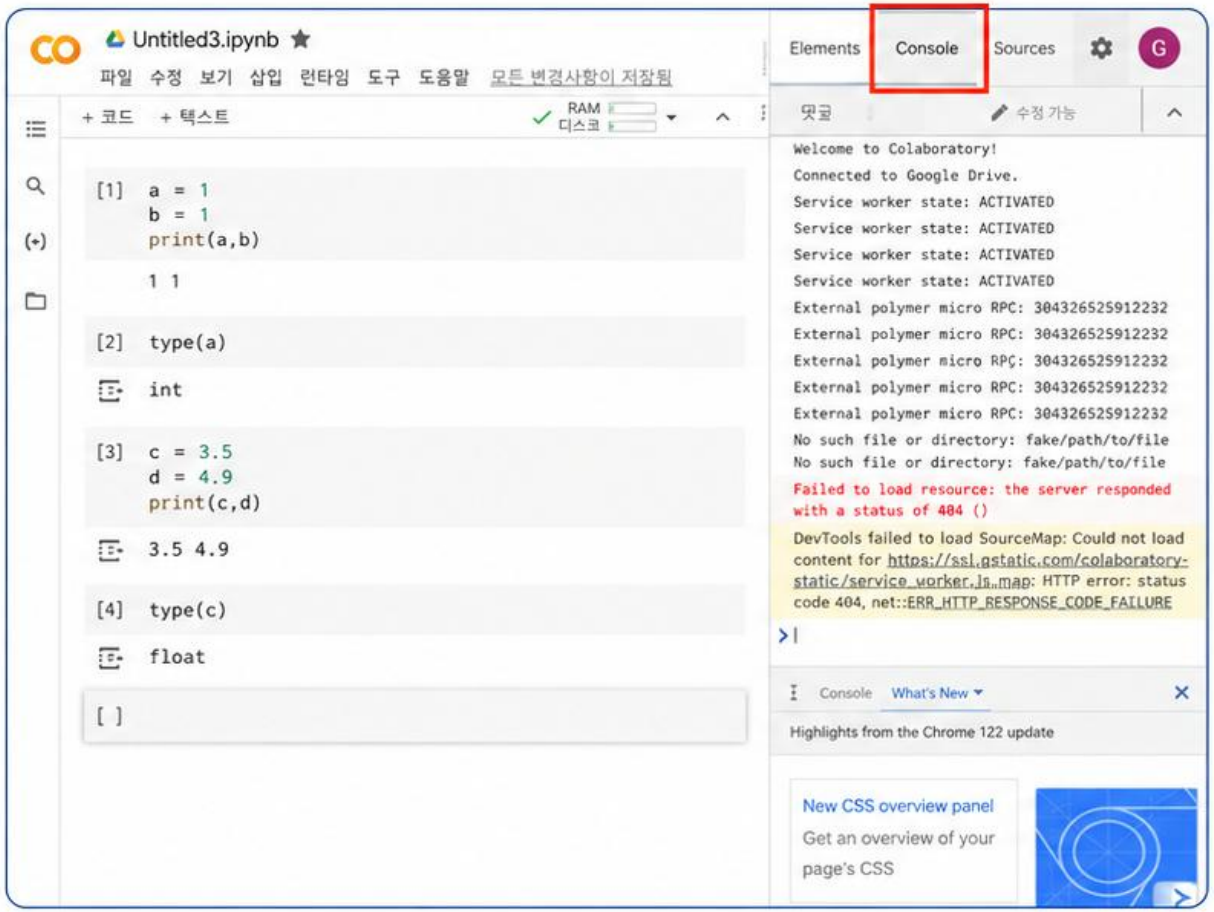
Google Colab의 전체 **세션 유지 시간은 12시간**이고, 90분 이상 비 활성화 되어 있으면 자동으로 끊기고,
몇 분 동안 사용을 안하면 자동으로 세션이 끊긴다.

방법

크롬 브라우저에서 F12(혹은 Ctrl+shift+I)를 눌러 개발자 도구 창을 열고
console 창에서 아래 코드를 입력해 주면 된다.

```
function ClickConnect(){
    var buttons
    =  document.querySelectorAll("colab-
    dialog.yes-no-dialog paper-button#cancel");
    buttons.forEach(function(btn){ btn.click(); });
    console.log("1분마다 자동 재연결");
    document.querySelector("colab-toolbar-button#connect").click(); }
setInterval(ClickConnect,1000*60);
```

현재는 되지 않음(2024.02)



Python IDE VS code 이란?

VS Code의 장점

1. MS의 친절한 관리

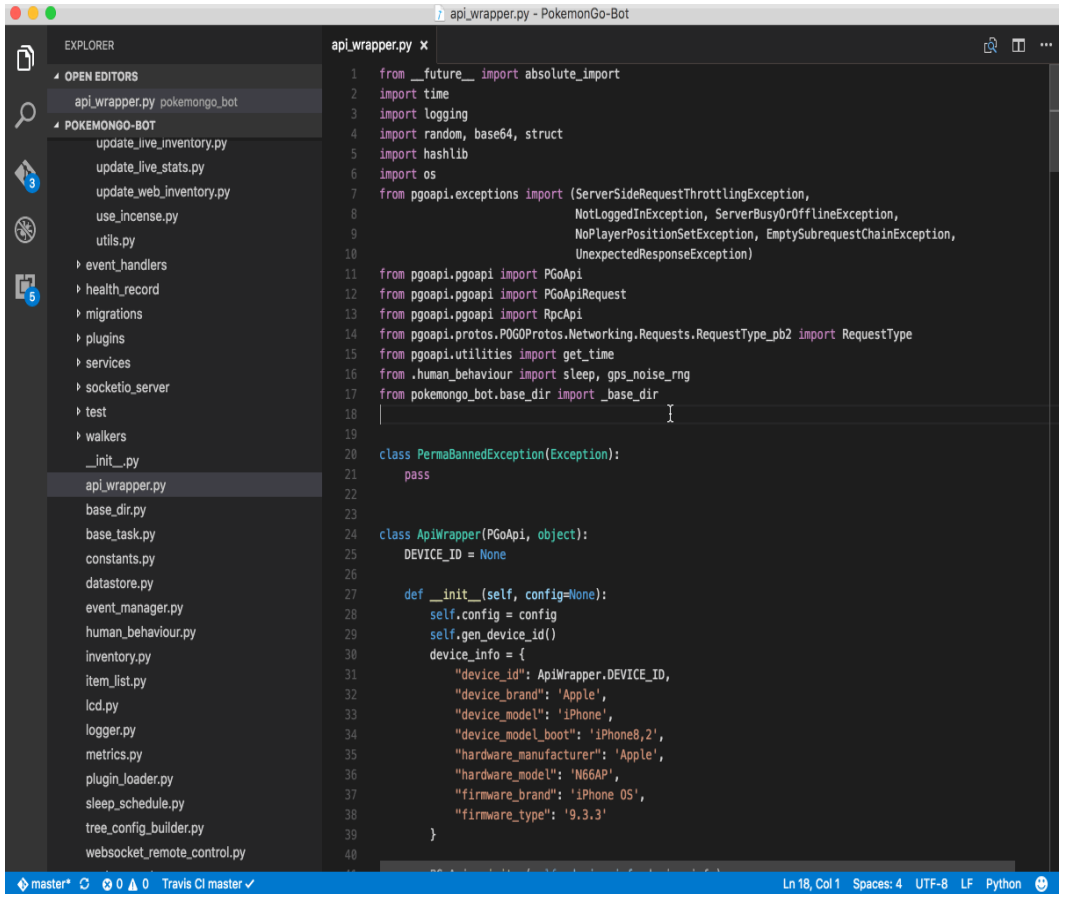
   - MS의 프로그램은 비싸지만 그만큼 친절함.

2. 무료 정책

3. 점유율 1위

4. 마이크로 소프트가 직접 배포하는 확장성

5. 언어 확장성이 좋음(C,C++,JAVA 등 지원함)

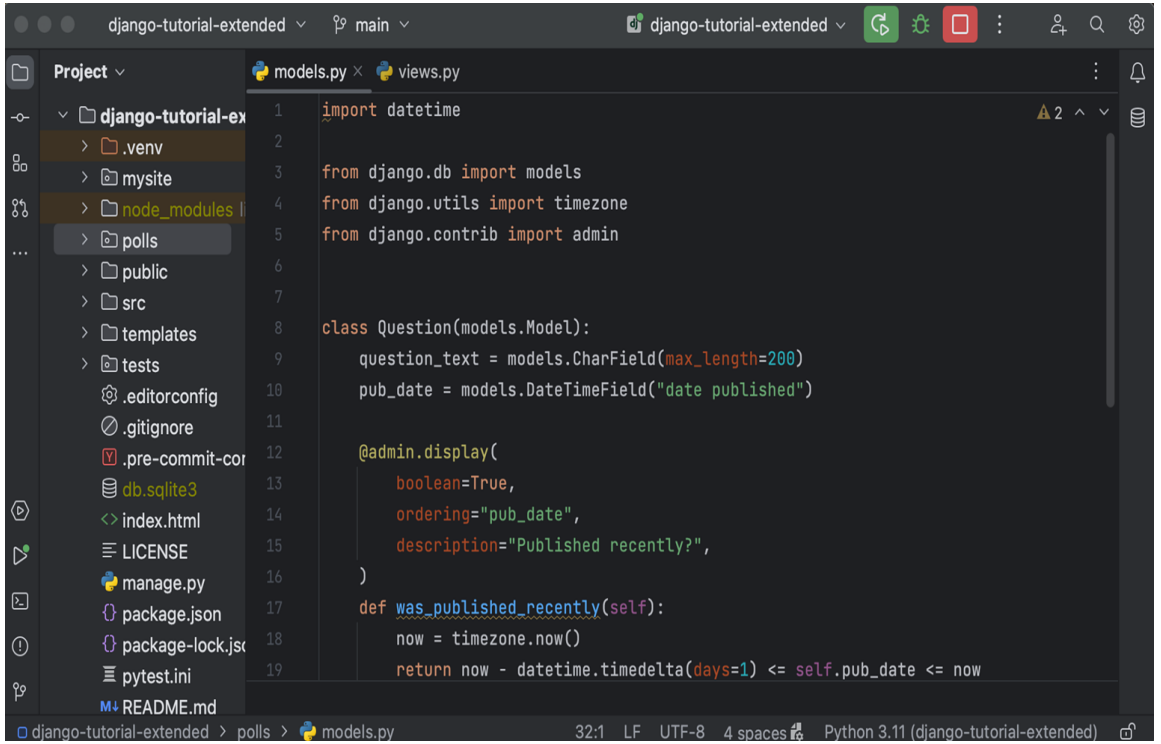


Python IDE Pycharm 이란?

- Pycharm은 Python 언어에 사용되는 JetBrains사에 의해 개발된 IDE입니다.
- IDE는 컴파일, 코드 편집기,디버그, 배포 등의 모든 작업을 하나의 프로그램에서 할 수 있도록 해주는 통합 개발 환경

Pycharm 의 특징

1. 코드 분석 및 코딩 지원
2. 프로젝트 및 코드 탐색
3. 파이썬
4. 웹 프레임 워크 지원(Professional 전용, 유료버전)
5. 통합 파이썬 디버그
6. 라인 단위 테스트
7. Google App Engine python(Professional 전용, 유료버전)
8. 과학적 도구 지원(Professional 전용, 유료버전)



Python 속도

“2~5배 더 빠르게”… 귀도 반 로섬, 파이썬 속도 개선한다

Paul Krill | InfoWorld

파이썬(Python) 창시자 귀도 반 로섬이 파이콘 2021(Pycon 2021)에서 진행된 파이썬 랭귀지 서밋(Python Language Summit)에서 파이썬 속도를 2배에서 5배까지 더 빠르게 만들기 위한 단기 및 장기 계획을 발표했다.



ⓒGetty Images

파이썬에는 이미 파이파이(PyPy)와 같은 대체 런타임부터 C/C++로 작성된 랩핑 모듈까지 더 빠르게 실행할 수 있는 여러 방법이 있다. 하지만 C로 작성된 파이썬 참조 구현체이자 가장 널리 사용되는 언어 버전인 C파이썬(CPython) 자체의 속도를 높이는 방법은 거의 없다는 게 반 로섬의 설명이다.

출처 : https://www.ciokorea.com/news/193909

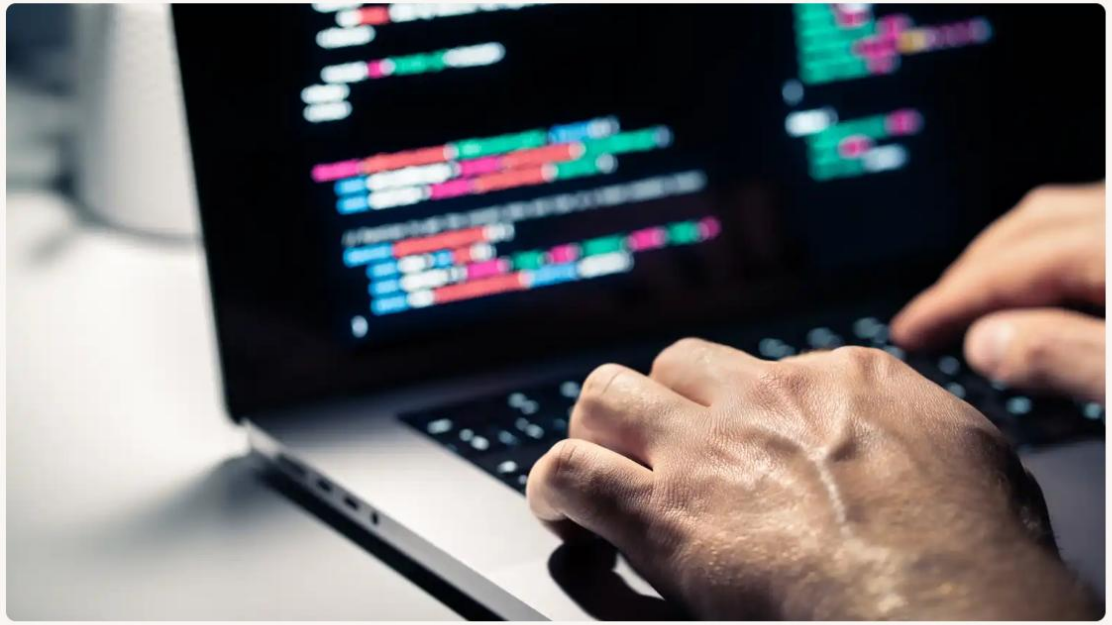
Python 속도

C언어와 속도 격차 좁힌다… 파이썬 새 버전, 인터프리터 개선해 30% 성능↑

뉴스
2025.02.14 • 2분

개발자 프로그래밍 언어

올해 말 출시 예정인 파이썬 3.14에 새로운 인터프리터가 추가되면서, 기존 코드 수정 없이도 최대 30% 성능 향상이 가능해진다. 첫 번째 베타 버전은 2025년 5월 공개될 예정이다.



CREDIT: TERO VESALAINEN / SHUTTERSTOCK

파이썬 재단의 공식 문서에 따르면, 파이썬 3.14 버전에는 '꼬리 호출(tail calls) 기반의 새로운 인터프리터'가 탑재된다. 사실 이 문구는 파이썬 개발 과정을 잘 모르는 사람들에게는 다소 오해를 불러일으킬 수 있는 표현이다. 여기서 말하는 '꼬리 호출'은 CPython 또는 파이썬 언어 자체가 꼬리 호출 최적화를 지원한다는 의미가 아니다. 그보다는 C 컴파일러가 CPython 코드에서 수행하는 최적화 기법으로, 인터프리터가 바이트코드를 실행하는 과정을 최적화해 전체 실행 속도를 높인다.



관련 콘텐츠

뉴스

출처 : https://www.cio.com/article/3824874/c%EC%96%B8%EC%96%B4%EC%99%80-%EC%86%8D%EB%8F%84-%EA%B2%A9%EC%B0%A8-%EC%A2%81%ED%9E%8C%EB%8B%A4%C2%B7%C2%B7%C2%B7-%ED%8C%8C%EC%9D%B4%EC%8D%AC-%EC%83%88-%EB%B2%84%EC%A0%84-%EC%9D%B8%ED%84%B0%ED%94%84.html

Python in HTML



출처 : https://www.youtube.com/watch?v=3DuyJf_XPtM

프론트 엔드 VS 백엔드

	프론트엔드	백엔드
정의	사용자들에게 보여지는 부분 Ex) 레이아웃, 텍스트, 그림 등	사용자들에게 보이지 않는 시스템 안쪽 부분 Ex) 내부적 DB
기술	**HTML&CSS** *HTML : 콘텐츠에 구조와 의미를 주기 위한 것 *CSS : 콘텐츠의 모양과 스타일을 주기 위한 것	**스프링프레임워크** *동적인 웹사이트 개발을 위한 여러가지 서비스 제공
	JAVA script *웹페이지에 기능을 더해 HTML 웹페이지를 동적이고 살아있게 만드는 것	**JAVA** *컴퓨터가 이해할 수 있는 0과 1로 이루어진 기계어
직무	**UI/UX 디자인** *UX디자인 : 사용자가 경험하게 될 전반적 디자인 *UI디자인 : 겉으로 시각화 되는 디자인	**서버코딩** *시스템이 운용되기 위해 필요한 시스템의 관련 기능 구현 등을 담당
	Front-end 개발 *웹상의 표현을 도와주는 여러 기술을 활용하여 UX와 Web 디자인 요소를 구현	**DB 활용** *데이터베이스를 구축하고 운영



출처 : https://www.jobindexworld.com/contents/view/4270 / https://www.facebook.com/coding.comu/posts/372409044728656

2. 파이썬의 기본

- 파이썬의 기본
- 컴퓨터의 구조 - 폰노이만 아키텍처.
- 중앙처리장치
- 파이썬의 기본



파이썬의 기본

- 산술 연산자

1. 더하기 : +

```
[ ] 1 + 2
```

```
3
```

```
[ ] 250.7 + 300.2
```

```
550.9
```

2. 빼기 : -

```
[ ] 5 - 3
```

```
2
```

```
[ ] 250.5 - 100.3
```

```
150.2
```

3. 곱하기 : *

```
[ ] 5 * 3
```

```
15
```

```
[ ] 20.5 * 2
```

```
41.0
```

4. 나누기 : /

```
[ ] 10 / 2
```

```
5.0
```

```
[ ] 7 / 3
```

```
2.3333333333333335
```

- 산술 연산자

▼ 4. 나누기 : /

```
[ ]  10 / 2
```

5.0

```
[ ]  7 / 3
```

2.3333333333333335

▼ 5. //

```
[ ]  10 // 2
```

5

```
[ ]  7 // 3
```

2

/의 연산 결과 : 소수점 표시 → 실수형

//의 연산 결과 : 소수점 없음(나누기의 몫) → 정수형

- 산술 연산자

6. 나머지 : %

```
[ ]  10 % 2
```

0

```
[ ]  7 % 3
```

1

7. 그 밖에 연산자

제곱 : **

```
[ ]  2 ** 3
```

8

```
[ ]  5 ** 3
```

125

파이썬의 기본

- 연산자의 우선순위

사칙연산의 연산자 우선순위와 같음

괄호 () 가 있다면 가자 우선 처리

```
[ ]  5 + 3 * 4
```

```
17
```

```
[ ]  (8 - 3) * 2
```

```
10
```

연습문제 1

1번

10의 제곱을 출력해보자

```
]:  ▶|  # 정답을 적어주세요
```

```
Out[1]: 100
```

```
]:  ▶|  # 정답을 적어주세요
```

```
Out[3]: 100
```

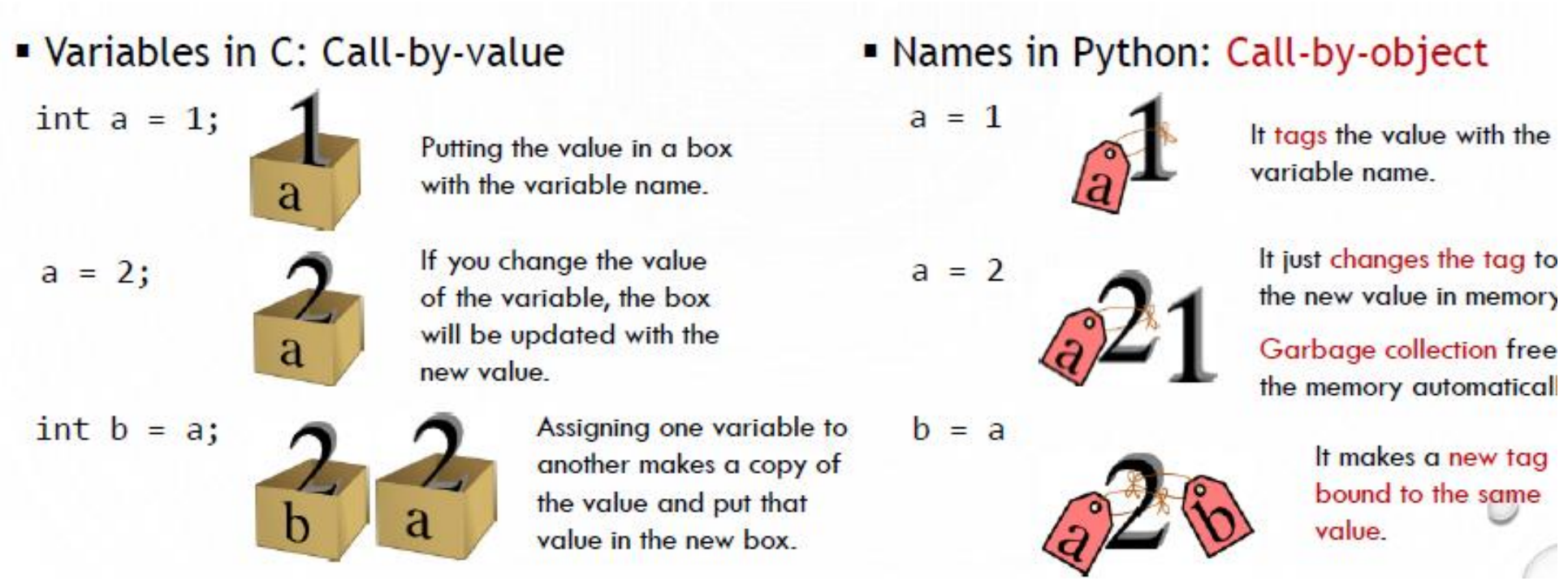
파이썬의 기본

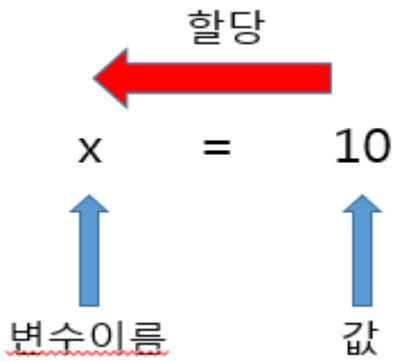
- 가장 기초적인 프로그래밍 문법 개념
- 데이터(값)을 저장하기 위한 메모리 공간의 프로그래밍 상 이름
- 변수는 메모리 주소를 가지고 있고, 변수에 들어가는 값을 메모리 주소에 할당됨.
- 할당 연산자(Assignment) : =

변수 = 연산 / 조건/ 함수

- 할당 연산자 우측을 연산을 한 후, 그 결과를 좌측에 있는 변수에 할당(저장)

- 할당 연산자(Assignment) : =





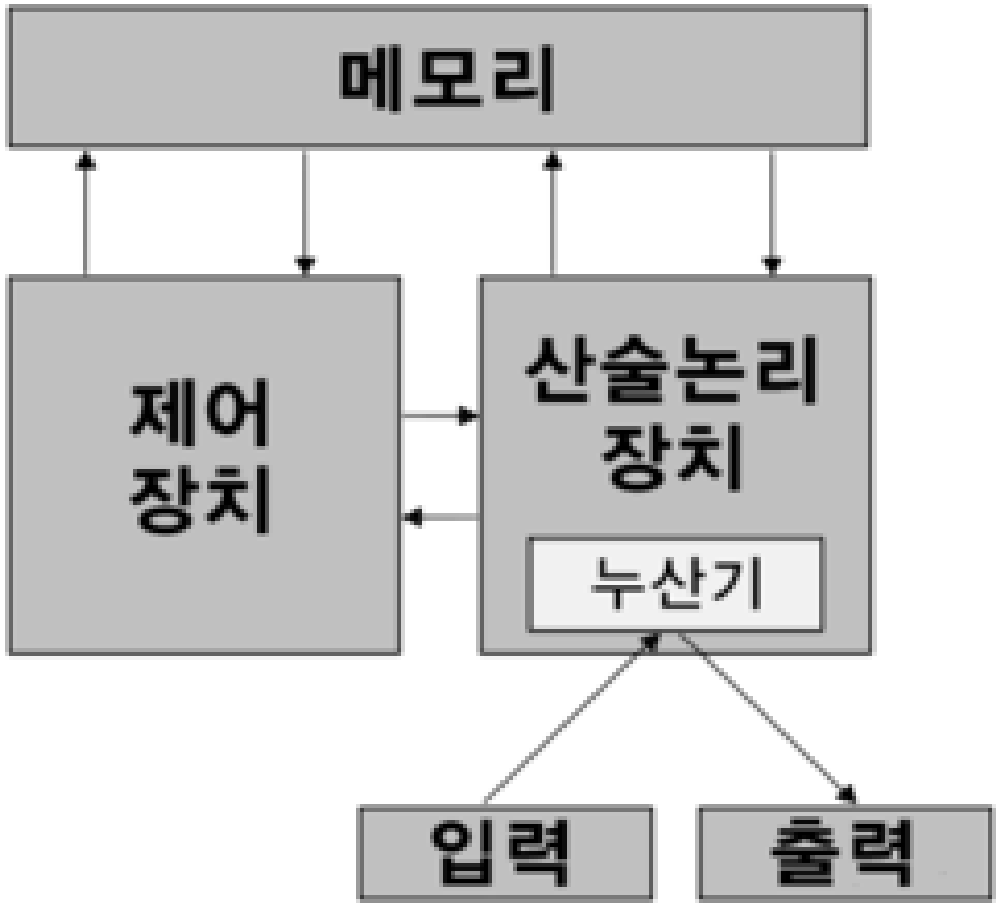
영문, 숫자 사용가능

대소문자 구분

숫자부터 시작할 수 없음.

특수문자 사용 불가(+, -, *, /, $ 등)

컴퓨터의 구조 - 폰노이만 아키텍처



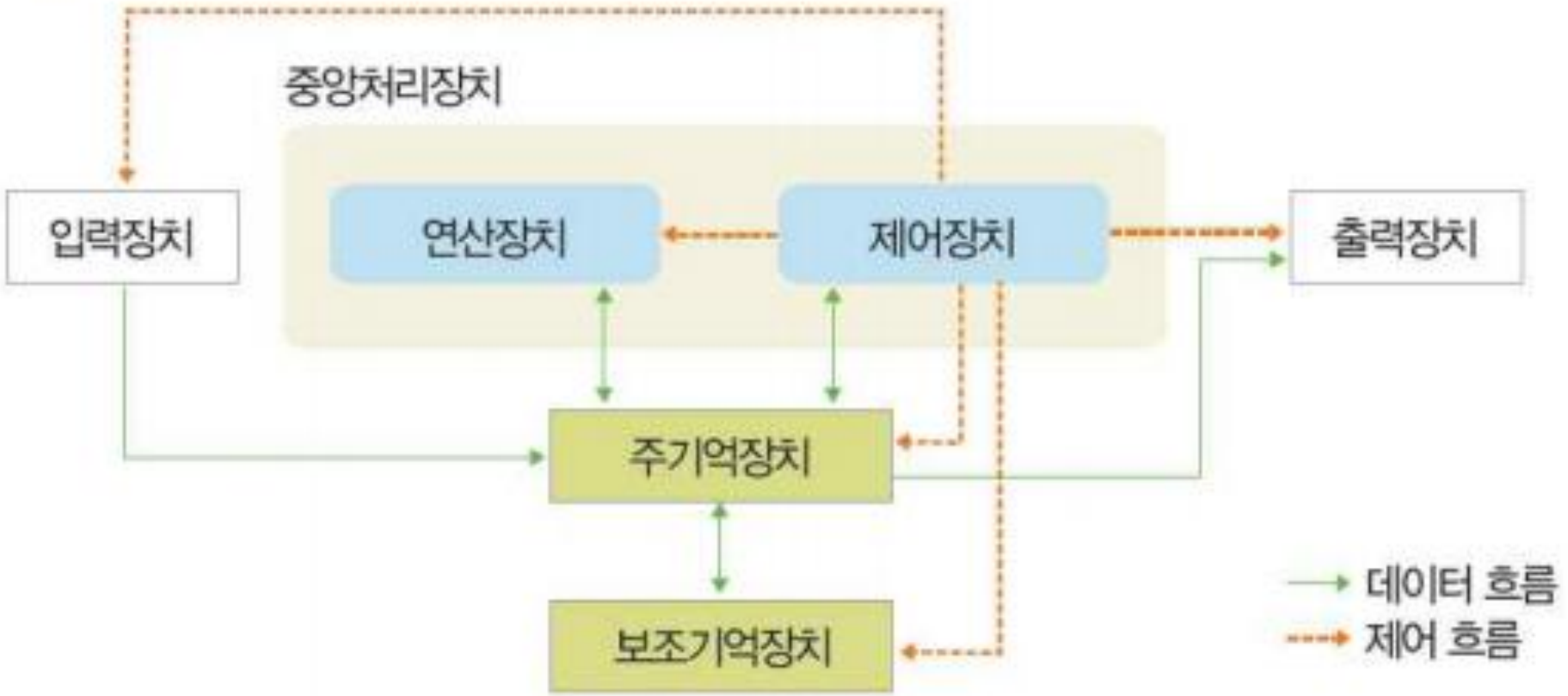
폰 노이만 아키텍처에서는 사용자가 컴퓨터에 값을 입력 하거나 프로그램을 실행하는 경우, 그 정보를 먼저 메모리에 저장시키고 CPU가 순차적으로 그 정보를 해석하고 계산하여 사용자에게 결과값 전달

- 변수는 프로그램에서 사용하기 위한 특정한 값을 저장하는 공간
- 선언 되는 순간 메모리 특정영역에 물리적인 공간이 할당됨

출처 : https://codingisgame.tistory.com/7

중앙처리장치

그림 3-2 하드웨어의 구성



출처 : http://pds21.egloos.com/pds/201504/09/88/computer03.pdf

- 할당 연산자(Assignment) : =

```
In [ ]: a = 3

In [ ]: b = 5

In [ ]: print(a+b)

In [ ]: print("a+b")

In [ ]: a=8

In [ ]: print(a+b)
```

할당 연산자 우측을 연산을 한 후, 그 결과를 좌측에 있는 변수에 할당(저장)

문제) Teacher = 'Youngpyo Ryu'의 의미는?

1. Teacher의 이름은 Youngpyo Ryu이다.
2. Teacher의 Youngpyo Ryu이다.
3. Teacher와 Youngpyo Ryu는 같다.
4. Teacher에 Youngpyo Ryu를 넣어라.

4. Teacher에 Youngpyo Ryu 를 넣어라

-> 정확히는 Teacher라는 변수에 'Youngpyo Ryu'라는 값을 넣으라는 의미.

- 할당 연산자(Assignment) : =

```
In [3]: a += 1
        a
Out[3]: 4

In [5]: a = a+1
        a
Out[5]: 4
```

  - 산술 연산자와 할당 연산자를 함께 쓸 수 있음
  - 코드를 더욱 간결하게 쓸 수 있음

파이썬의 기본

- 할당 연산자(Assignment) : =

```
In [6]: a +=1
        a

Out[6]: 5
```

```
In [7]: a -=5
        a

Out[7]: 0
```

```
In [10]: a *= -1
         a

Out[10]: 0
```

```
In [11]: a /= 5
         a

Out[11]: 0.0
```

```
In [12]: a * = -1
         a

  File "<ipython-input-12-(
    a * = -1
        ^
SyntaxError: invalid syntax
```

```
In [13]: a / = 5
         a

  File "<ipython-input-13-(
    a / = 5
        ^
SyntaxError: invalid syntax
```

파이썬의 기본

- 알파벳, 숫자, 언더스코어(_)로 선언이 가능

  ex) data = 0, a12_ = 2, tt_ = 'afdf'

- 변수명은 의미 있는 단어로 표기하는 것이 좋다.

  Ex) teacher = 'Youngpyo Ryu'

- 변수명은 대소문자가 구분 된다.

  Ex) ABC와 Abc는 같지 않다.

- 특별한 의미가 있는 예약어는 쓰지 않는다.

  Ex) for, if, elif, else 등

파이썬의 기본

파이썬 예약어는 파이썬에서 이미 키워드로 지정되어 있어 변수나 함수의 이름으로 사용할 수가 없다.

```
import keyword
print(keyword.kwlist)
```

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break',
'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for',
'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not',
'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

출처 : https://isss.tistory.com/842

파이썬의 기본

- 할당 연산자(Assignment) : =

  변수간의 연산도 가능

```
[ ]  a = 5
     b = 7
     c = a + b
     print(c)
```

```
12
```

주의 파이썬은 대소문자를 구분함

```
[ ]  c = A + B
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-26-b698ae2737d5> in <module>
----> 1 c = A + B

NameError: name 'A' is not defined
```

파이썬의 기본

- 할당 연산자(Assignment) : =

파이썬은 들여쓰기(Indent)에 굉장히 민감(jupyter에서는 보정)

```
(base) C:₩Users₩USER>python
Python 3.7.3 (default, Apr 24 2019, 15:29:51) [MSC v.1915 64 bit (AMD64)] :: Anaconda, Inc. on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>  1+1
  File "<stdin>", line 1
    1+1
    ^
IndentationError: unexpected indent
>>>
```

연습문제 2-1

1번

사과가 5개 오렌지가 3개 있을 때 총 과일의 갯수를 구해보자
사과는 apple이라는 변수, 오렌지는 orange라는 변수에 할당 한 후, 총 과일의 갯수를 total이라는 변수에 저장해보자

```
# 정답을 적어주세요
```

```
# 이래 주석을 풀고 실행시켜서 원하는 값이 나왔는지 확인하세요
# total
```

[10]: 8

연습문제 2-2

2번

국어는 100점, 영어는 88점, 수학은 94점 일 때, 평균을 구하려고 한다.
각각의 점수를 kor, eng, math라는 변수에 저장한 후 평균을 구해 avg라는 변수에 할당해보자.

[ ]:
```
# 정답을 적어주세요
```

[ ]:
```
# 아래 주석을 풀고 실행시켜서 원하는 값이 나왔는지 확인하세요
# avg
```

Out[13]: 94.0

3. 파이썬의 자료형

- 숫자(int, Float)
- 숫자의 자료형
- 문자(String)
- 리스트(list)
- 튜플(tuple)
- 딕셔너리(dict)
- 셋(set)
- 불(bool)



1. 숫자의 자료형



데이터 형식	설명	크기(Byte)	값의 범위
bool	논리 형식	1(8bit)	true, false
sbyte	signed byte 정수	1(8bit)	-128 ~ 127
byte	부호 없는 정수	1(8bit)	0 ~ 255
short	정수	2(16bit)	-32,768 ~ 32,767
ushort	unsigned short 부호 없는 정수	2(16bit)	0 ~ 65,535
char	유니코드 문자	2(16bit)	U+0000 ~ U+ffff
int	정수	4(32bit)	-2,147,483,648 ~ 2,147,483,647
uint	unsigned int 부호 없는 정수	4(32bit)	0 ~ 4,294,967,295
float	단일 정밀도 부동 소수점 형식	4(32bit)	-3.402823e38 ~ 3.402823e38
long	정수	8(64bit)	-9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807
ulong	unsigned long 부호 없는 정수	8(64bit)	0 ~ 18,446,744,073,709,551,615
double	복수 정밀도 부동 소수점 형식	8(64bit)	-1.79769313486232e308 ~ 1.79769313486232e308
decimal	29자리 데이터를 표현할 수 있는 소수 형식	16(128bit)	±1.0×10e-28 ~ ±7.9×10e28

출처 : 고등학교 상) 수의 체계 / https://mhchoi8423.tistory.com/5

1. 숫자(int, float)

- int와 float

```
[1]  a = 1
     b = 1
     print(a,b)
```

```
1 1
```

```
[3]  type(a)
```

```
int
```

```
[3]  c = 3.5
     d = 4.9

     print(c,d)
```

```
3.5 4.9
```

```
[4]  type(c)
```

```
float
```

int는 정수, float는 실수

type(변수) : 변수의 자료형을 알려주는 함수

Print(변수) : 모니터 화면에 결과물을 출력해주는 함수

1. 숫자(int, float)

- int와 float

```
[4]   d = int(c)
      print(d)
      type(d)
```

```
3
int
```

숫자는 형 변환을 할 수 있음

```
[8]   a = 10
      type(a)
```

```
int
```

```
[9]   float(a)
```

```
10.0
```

Quiz) 현재 a의 자료형은?

Ans) 현재 a는 int이다. 그 이유는 형 변환 해주지 않음

```
[11]  a = float(a)

      type(a)
```

```
float
```

Quiz)

- 5.3과 5.7 정수형으로 형 변환 후 덧셈하면 결과값은?

```
a = 5.3
b = 5.7

a = int(a)
b = int(b)

print(a+b)
```

```
10
```

※ round() : 반올림

2. 문자(String)

- string

‘ ’(작은 따옴표) 혹은 “ ”(큰 따옴표)를 이용하여 문자형(string)임을 표시

```
[11] f = '문자형 예시입니다.'
     print(f)
     type(f)
```

```
문자형 예시입니다.
str
```

2. 문자(String)

```
[12] 따옴표 = '문장안에 따옴표를 쓰고 싶다면? "" 번갈아 쓰면 됩니다.'
     print(따옴표)
```

```
문장안에 따옴표를 쓰고 싶다면? "" 번갈아 쓰면 됩니다.
```

```
[13] escape = "escape 문자인 \" \\ \"(역슬래시 혹은 원화표시)도 사용가능"
     print(escape)
```

```
escape 문자인 " \ "(역슬래시 혹은 원화표시)도 사용가능
```

2. 문자(String)

```
print('''실패란 넘어지는 것이 아니라,
넘어진 자리에 머무는 것이다.
- 도서 '프린세스, 라 브라바!' 中''')
```

```
실패란 넘어지는 것이 아니라,
넘어진 자리에 머무는 것이다.
- 도서 '프린세스, 라 브라바!' 中
```

```
print('₩t탭₩n다음줄')
```

```
        탭
다음줄
```

2. 문자(String)

```
[21] g = 3
     h = '5'
```

Quiz) g,h의 자료형은?

```
[22] type(g)
```

```
int
```

```
[23] type(h)
```

```
str
```

2. 문자(String)

```
[21] g = 3
     h = '5'
```

```
[24] i = str(g)
     print(i)
     type(i)
```

```
3
str
```

```
[ ]  j = int(h)
     print(j)
     type(j)
```

```
5
int
```

str와 int도 형 변환이 가능

2. 문자(String)

- string의 연산

```
[26] str_1 = '문자'
     str_2 = '의 연산'
     result = str_1 + str_2
     result
```

```
'문자의 연산'
```

```
[27] str_3 = '더하기'
     result += str_3
     result
```

```
'문자의 연산더하기'
```

더하기를 하면 두 문자열을 붙여줌

2. 문자(String)

- string의 연산

```
[28] str_4 = '곱하기'
     print(str_4 * 5)
```

```
곱하기곱하기곱하기곱하기곱하기
```

```
[29] str_5 = '반복'
     print((str_4 + str_5)*2)
```

```
곱하기반복곱하기반복
```

곱하기는 그 수만큼 문자열을 반복함

2. 문자(String)

- string의 연산

단, 자료형이 다르다면 연산이 불가

```
int_1 = 8
str_6 = '문자'

error = str_6+int_1
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most
<ipython-input-1-9ac98546a9bd> in <module>
      2 str_6 = '문자'
      3
----> 4 error = str_6+int_1

TypeError: can only concatenate str (not "int") to str
```

2. 문자(String)

- string의 indexing(인덱싱)

  인덱싱 ? 순서가 있는 변수의 특정 위치에 접근 할 수 있게 해줌

변수[위치]

(*주의*) 파이썬의 인덱싱은 0부터 시작

```
[47] str_5 = '인덱싱을 하기 위한 string입니다.'
     len(str_5)
```

len(변수)는 변수의 길이를 재는 함수

```
21
```

```
[48] str_5[2]
```

```
'싱'
```

2. 문자(String)

- string의 slicing(슬라이싱)

슬라이싱 ? 변수의 일정 부분에 접근하여 잘라옴

변수[시작 : 끝 : 간격]

```
[49] str_5 = '인덱싱을 하기 위한 string입니다.'
     str_5[2:7]
```

```
'싱을 하기'
```

(*주의*)

- 슬라이싱은 시작부터 끝 -1까지 가져옴
- 간격이 1이라면 생략해도 됨

2. 문자(String)

- string의 slicing(슬라이싱)

슬라이싱 ? 변수의 일정 부분에 접근하여 잘라옴

변수[시작 : 끝 : 간격]

```
[50] str_6 = '안녕하세요. 홍길동 고객님!'
```

```
[52] str_6[7:10]
```

```
'홍길동'
```

2. 문자(String)

- string의 slicing(슬라이싱)

```
[54] str_6 = '안녕하세요. 홍길동 고객님!'
     print(str_6[:2])
```

```
안녕
```

시작점이 없다면 처음부터

```
[57] print(str_6[11:])
```

```
고객님!
```

끝이 없다면 끝까지

```
[58] print(str_6[:])
```

```
안녕하세요. 홍길동 고객님!
```

시작과 끝이 없다면 전체

2. 문자(String)

- string의 slicing(슬라이싱)

  파이썬은 마이너스 인덱싱을 지원함

```
str_6 = '안녕하세요. 홍길동 고객님!'
```

```
print(str_6[-1])
```

```
!
```

-1은 끝에서부터 거꾸로

```
print(str_6[-4:])
```

```
고객님!
```

```
print(str_6[:-2])
```

```
안녕하세요. 홍길동 고객
```

2. 문자(String)

- string의 slicing(슬라이싱)

  파이썬은 마이너스 인덱싱을 지원함

```
[65] str_6 = '안녕하세요. 홍길동 고객님!'
     print(str_6[::-1])
```

```
!님객고 동길홍 .요세하녕안
```

[::-1]은 str 전체를 뒤집음

2. 문자(String)

- string의 method

  method? 파이썬의 객체가 가지고 있는 고유의 함수 . 을 이용하여 사용

  1. 대소문자 변환 : lower, upper

```
[66] str_7 = 'Alphabet'
     print(str_7.lower())
```

```
alphabet
```

```
[67] print(str_7.upper())
```

```
ALPHABET
```

2. 문자(String)

- string의 method

2. 해당 문자의 개수 세기 : count

```
[68] str_7 = 'Alphabet'
     str_7.count('a')
```

```
1
```

count는 대소문자를 구분함

Quiz) str_7에서 모든 것을 소문자 바꾼 다음에 a 문자의 개수를 세어보려면?

```
[71] str_7.lower().count('a')
```

```
2
```

2. 문자(String)

- string의 method

3. 문자열의 위치 찾기

1) find 해당하는 문자열의 위치(인덱스)를 반환

```
[72] str_7 = 'Alphabet'
     str_7.find('t')
```

```
7
```

2) index

```
[73] str_7.index('t')
```

```
7
```

2. 문자(String)

- string의 method

3. 문자열의 위치 찾기

1) find

```
[74] str_7 = 'Alphabet'
     str_7.find('z')
```

```
-1
```

문자열에 없으면 -1 반환

2) index

```
[75] str_7.index('z')
```

```
---------------------------------
ValueError: substring not found
```

문자열에 없으면 error

2. 문자(String)

- string의 method

4. 문자열 바꾸기 : replace

```
[82] str_8 = 'Life is C between B and D'
     str_8.replace('C', 'Chicken')
```

```
'Life is Chicken between B and D'
```

5. 문자열 나누기 : split

```
[83] str_8.split(' ')
```

따옴표 ‘ ’ 사이에 기준이 되는 문자를 넣기

생략 시 기준은 띄어쓰기

```
['Life', 'is', 'C', 'between', 'B', 'and', 'D']
```

2. 문자(String)

- string의 method

6. 문자열 삽입 : join

```
[84] str_9 = 'abcd'
     ','.join(str_9)
```

따옴표‘ ’사이에 삽입할 문자를 넣기

```
'a,b,c,d'
```

7. join와 split

```
[85] str_10 = str_8.split()
```

split한 결과에 join을 사용하면 다시 문자열로 만들 수 있음

```
[86] ' '.join(str_10)
```

```
'Life is C between B and D'
```

2. 문자(String)

- string의 method

  더 많은 메소드는 Python 공식문서나 아래와 같은 방법으로 확인 가능

```
[21] print(dir(str_8))
```

```
['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__ge
tattribute__', '__getitem__', '__getnewargs__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__iter__', '_
_le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr__', '_
_rmod__', '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'capitalize', 'casefold', 'center',
'count', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isascii'
, 'isdecimal', 'isdigit', 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper', 'jo
in', 'ljust', 'lower', 'lstrip', 'maketrans', 'partition', 'replace', 'rfind', 'rindex', 'rjust', 'rpartition', 'rspl
it', 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate', 'upper', 'zfill']
```

3. 리스트(list)

- 시퀀스 자료형, 여러 데이터들의 집합
- int, float 같은 다양한 데이터 타입 포함

[ ]을 이용하여 리스트(list)임을 선언

```
[1]  list_1 = [3,2,5]
     list_2 = list()
```

```
[2]  type(list_1)
```

```
list
```

3. 리스트(list)_특징

- 리스트는 모든 자료형을 담을 수 있음
- 리스트는 삽입, 수정, 삭제가 자유로움
- 리스트는 순서가 있는 자료형 → 인덱싱과 슬라이싱이 가능

```
[4] list_3 = [1, 2, '문자', ['이중 리스트', '가능'], ('리스트 속', '튜플')]
```

```
[5] list_3[3]
```

```
['이중 리스트', '가능']
```

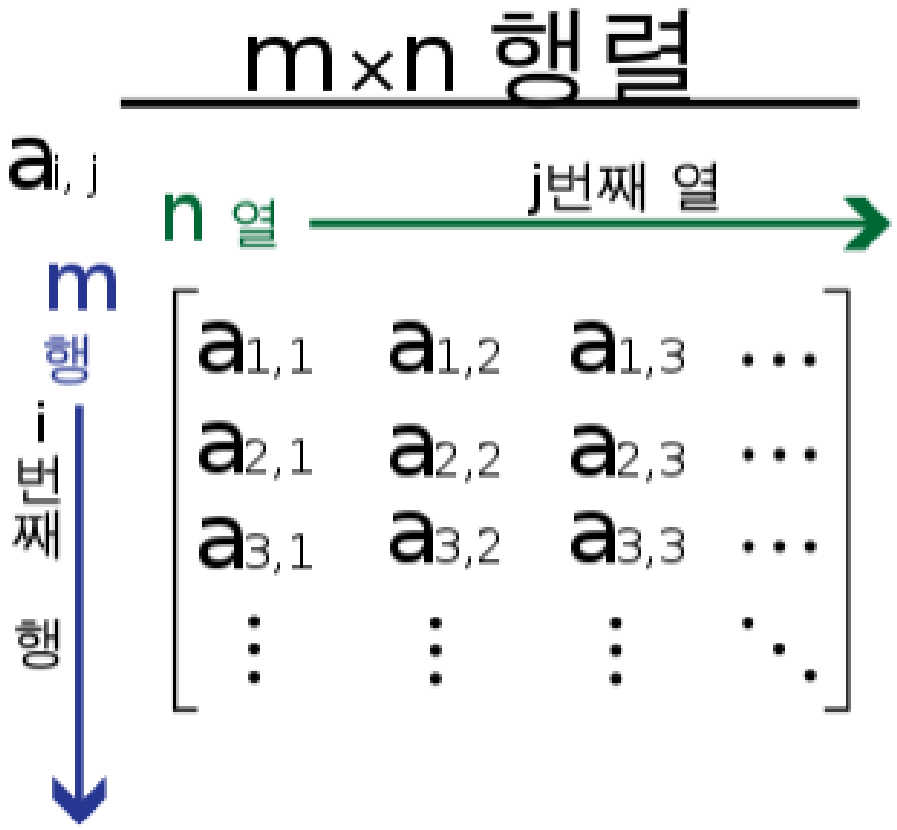
```
[6] list_3[4][1]
```

```
'튜플'
```

주의 python의 인덱스는 0부터 시작

3. 리스트(list)

- 2차원 리스트는 가로X세로 형태로 이루어져 있으며,
  행(row)과 열(column) 모두 0부터 시작



출처 : https://blog.naver.com/cindyvelyn/222126373034

3. 리스트(list)

- 2차원 리스트는 리스트 안에 리스트를 넣어서 만듦.
- , (콤마)로 구분
- 리스트 = [[값, 값],[값, 값],[값, 값]]
- 리스트 [세로인덱스] [가로인덱스]

```
a = [[10,20],[30,40],[50,60]]
a
```

```
[[10, 20], [30, 40], [50, 60]]
```

```
a[0][0]
```

```
10
```

```
a[1][0]
```

```
30
```

3. 리스트(list)

- list의 인덱싱

```
[7]  list_4 = [[1,2,3,4],
               [2,4,6,8],
               [3,6,9,12],
               [4,8,12,16],
               [5,10,15,20]]
```

가장 바깥쪽부터 괄호부터

```
[9]  list_4[1][3]

     8
```

앞에서부터 순서대로

3. 리스트(list)

- list의 인덱싱

```
[ ]  list_5 = [[[1,2,3],[4],[5,[6,7,8,9]]],
               [10,11,[12,13,[14]]]]
```

```
[ ]  list_5[1][2][2]
```

```
[14]
```

3. 리스트(list)

- list의 인덱싱

```
[7]  list_4 = [[1,2,3,4],
               [2,4,6,8],
               [3,6,9,12],
               [4,8,12,16],
               [5,10,15,20]]
```

```
[ ]  list_4[1][:3]
```

```
[2, 4, 6]
```

```
[ ]  list_4[:2][1]
```

```
[2, 4, 6, 8]
```

3. 리스트(list)

- list의 인덱싱

```
[9]  list_5 = [[[1,2,3],[4],[5,[6,7,8,9]]],
               [10,11,[12,13,[14]]]]
```

```
[13] list_5[:2][0]
```

```
     [[1, 2, 3], [4], [5, [6, 7, 8, 9]]]
```

```
[14] list_5[0][:2]
```

```
     [[1, 2, 3], [4]]
```

3. 리스트(list)

- list의 연산

```
[15] list_6 = ['a', 'b', 'c']
```

```
[16] list_7 = ['가', '나', '다']
```

```
[17] list_6 + list_7
```

```
['a', 'b', 'c', '가', '나', '다']
```

```
[18] list_6 * 3
```

```
['a', 'b', 'c', 'a', 'b', 'c', 'a', 'b', 'c']
```

```
[15] list_6 = ['a','b','c']
```

```
[16] a in list_6
```

```
False
```

3. 리스트(list)

- 메소드

  리스트는 삽입, 수정, 삭제가 자유로움

1. 삽입 : append, insert

```
[19] list_8 = ['파이썬', 'C', 'java']
```

```
[20] list_8.append('R')
     list_8
```

```
['파이썬', 'C', 'java', 'R']
```

→ append는 리스트 제일 뒤에

```
[21] list_8.insert(2, 'C++')
     list_8
```

```
['파이썬', 'C', 'C++', 'java', 'R']
```

```
[22] list_8.insert(5, 'C')
     list_8
```

```
['파이썬', 'C', 'C++', 'java', 'R', 'C']
```

3. 리스트(list)

- 메소드

  2. 수정 : 덮어쓰기

```
[23] list_8[0] = 'Python'
     list_8
```

인덱스나 슬라이싱을 통해 수정

```
['Python', 'C', 'C++', 'java', 'R', 'C']
```

2-1) 확장

```
[23] list_8.extend(['MATLAB','Julia'])
     list_8
```

```
['Python', 'C', 'C++', 'java', 'R', 'C', 'MATLAB', 'Julia']
```

3. 리스트(list)

- 메소드

  3. 삭제

```
[24] list_8

     ['Python', 'C', 'C++', 'java', 'R', 'C']
```

```
[25] list_8.remove('C')
     list_8

     ['Python', 'C++', 'java', 'R', 'C']
```

→ remove는 지우고자 하는 요소를 명시해야함

→ 가장 앞에 있는 요소가 **삭제**됨

- remove(x) -> 리스트 안에서 값 x를 찾아서 지움
- del list[i] -> 리스트에서 i번째를 지움
- del list -> 리스트 변수 자체를 없앰

```
[34] del list_8[6]
     list_8

     ['Python', 'C', 'C++', 'java', 'R', 'C']
```

```
del list_8
print(list_8)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
/tmp/ipython-input-1175381942.py in <cell line: 0>()
      1 del list_8
----> 2 print(list_8)

NameError: name 'list_8' is not defined
```

3. 리스트(list)

- 메소드

  4. 꺼내오기

```
[26] list_8.pop()

     'C'
```

```
[28] list_8

     ['Python', 'C++', 'java', 'R']
```

→ pop은 괄호( )안 인덱스에 해당하는 요소를 **반환**

→ 인덱스를 생략하면 가장 마지막 요소를 꺼내옴

3. 리스트(list)

- 메소드

  5. 정렬 : sort

```
[29] list_9 = [1,5,7,2,6]
```

```
[30] list_9.sort()
     list_9
```

리스트 자체를 정렬

```
[1, 2, 5, 6, 7]
```

list.sort(key=None, reverse=False)

- key : 정렬해주고 싶은 기준, 기본적으로 < 비교에 의해 정렬(오름차순)
- reverse : True로 설정하게 되면 > 비교에 의해 정렬(내림차순)

3. 리스트(list)

- 메소드

  5. 정렬 : sort

```
[32] list_10 = [1,47,12,6]
     list_10.sort(reverse=True)
     list_10
```

```
[47, 12, 6, 1]
```

- 메소드

  6. 뒤집기 : reverse

```
[37] list_9
```

```
[1, 2, 5, 6, 7]
```

```
[38] list_9.reverse()
     list_9
```

```
[7, 6, 5, 2, 1]
```

→ 현재 리스트 요소 순서를 뒤집음
→ 리스트 **자체**를 뒤집음

3. 리스트(list)

- 메소드

  6. 뒤집기 : reverse

```
[32] list_10 = [1,47,12,6]
     list_10.sort(reverse=True)
     list_10
```

```
[47, 12, 6, 1]
```

```
[41] list_10.reverse()
     list_10
```

```
[1, 6, 12, 47]
```

```
sorted(list_10)
```

```
[1, 6, 12, 47]
```

sorted() : 원본 리스트를 바꾸지 않고, 정렬된 "새 리스트"를 만들어 변환하는 함수

: 원본을 변경하지 않음

3. 리스트(list)_참고

```
a = [10,9,8,7,6]
b = [5,7,8,9,10]
```

```
print(id(a),id(b))
```

```
140486843016208 140486843685856
```

```
b=a
```

```
print(id(a),id(b))
```

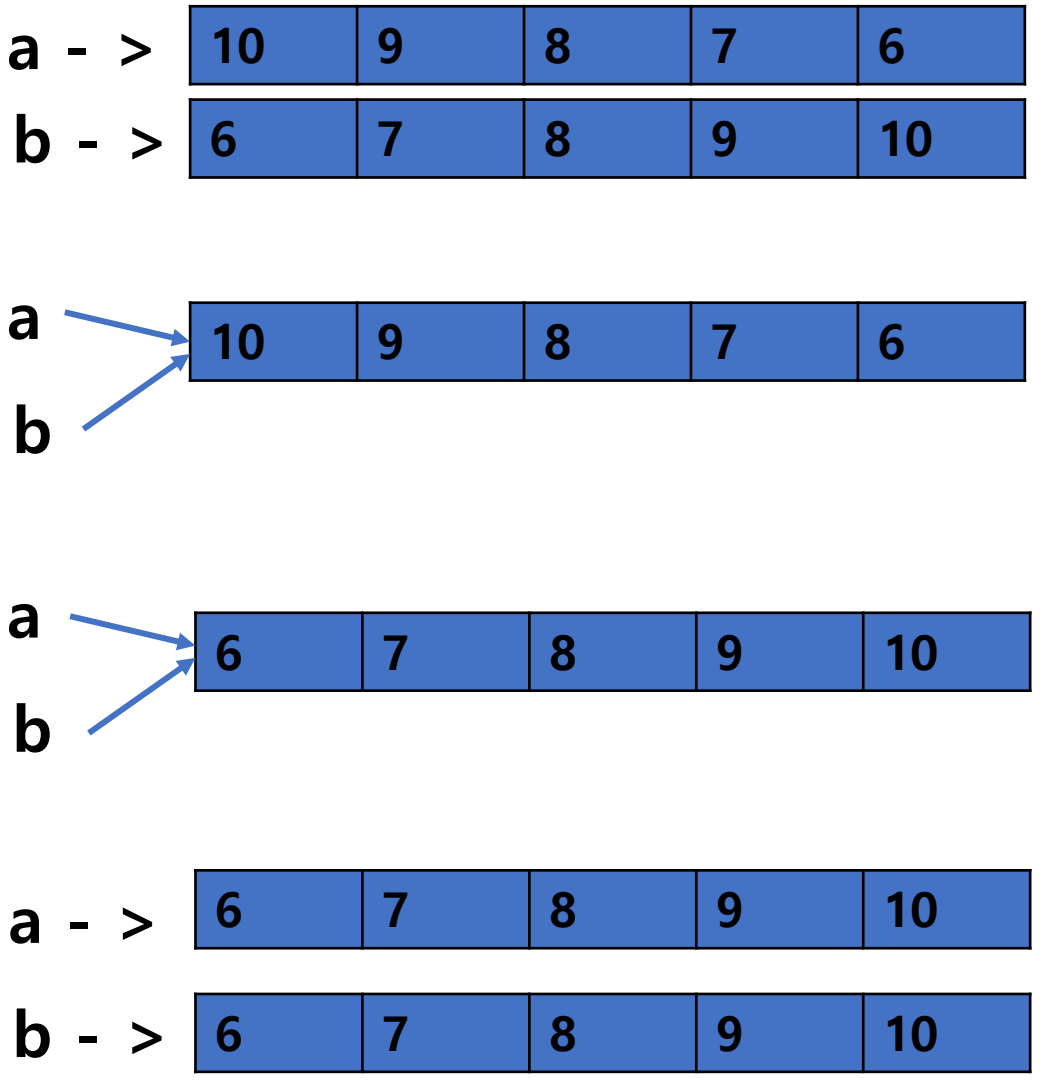
```
140486843016208 140486843016208
```

```
a.sort()
```

```
print(b)
```

```
[6, 7, 8, 9, 10]
```

얕은 복사



3. 리스트(list)_참고

같은 값은 복사하고 싶으면?

```
a = [5,4,3,2,1]
```

```
b=a[:]
```

```
print(id(a),id(b))
```

```
140486843034272 140486843033392
```

```
b
```

```
[5, 4, 3, 2, 1]
```

```
a.sort()
```

```
a
```

```
[1, 2, 3, 4, 5]
```

```
b
```

```
[5, 4, 3, 2, 1]
```

```
a = [5,4,3,2,1]
```

```
b = a.copy()
```

```
print(id(a),id(b))
```

```
140486843511744 140486843484752
```

```
a.sort()
a
```

```
[1, 2, 3, 4, 5]
```

```
b
```

```
[5, 4, 3, 2, 1]
```

```
a
```

```
[1, 2, 3, 4, 5]
```

```
import copy
a = [5,4,3,2,1]
```

```
b=copy.deepcopy(a)
```

```
print(id(a),id(b))
```

```
140486843203744 140486843046240
```

```
a.sort()
a
```

```
[1, 2, 3, 4, 5]
```

깊은 복사

```
b
```

```
[5, 4, 3, 2, 1]
```

```
a
```

```
[1, 2, 3, 4, 5]
```

3. 리스트(list)_참고

같은 값을 복사할 때, 주의사항.

```
a = [5,4,3,2,1]
```

```
b=a[:]
```

```
print(id(a),id(b))
```

```
140486843034272 140486843033392
```

```
b
```

```
[5, 4, 3, 2, 1]
```

```
a.sort()
```

```
a
```

```
[1, 2, 3, 4, 5]
```

```
b
```

```
[5, 4, 3, 2, 1]
```

```
a = [[1,2],[3,4]]
a
```

```
[[1, 2], [3, 4]]
```

```
b=a[:]
```

```
a[0][1]=100
a
```

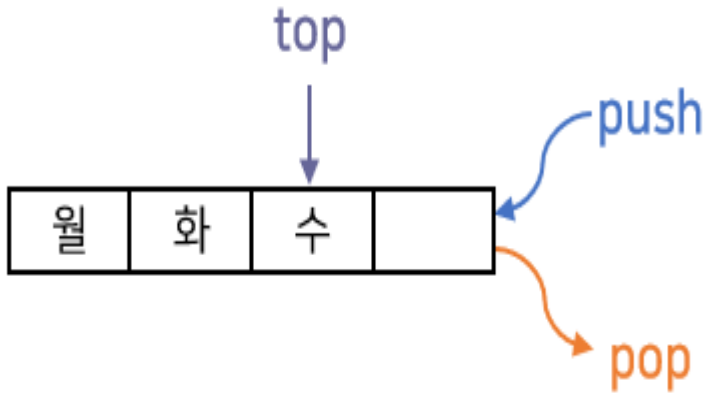
```
[[1, 100], [3, 4]]
```

```
b
```

```
[[1, 100], [3, 4]]
```

3. [자료구조] Stack(스택)

- 나중에 넣은 데이터를 먼저 반환하도록 설계된 메모리 구조
- 스택(Stack)은 데이터를 차곡차곡 쌓아 올린 형태
- 후입선출(LIFO, Last-in First-Out)




```
list_5 = [1,2,3,4,5]
list_5.append(6)
list_5.append(7)
```

```
list_5
```

```
[1, 2, 3, 4, 5, 6, 7]
```

```
list_5.pop()
```

```
7
```

```
list_5.pop()
```

```
6
```

```
list_5
```

```
[1, 2, 3, 4, 5]
```

3. [자료구조] Queue(큐)

- 먼저 넣은 데이터를 먼저 반환하도록 설계된 메모리 주소
- 스택(Stack)과 비슷하지만 다른 자료 구조
- 선입선출(Frist-in First-Out)
- 데이터의 삽입 연산(enQueue),삭제 연산(deQueue)




```
list_5 = [1,2,3,4,5]
list_5.append(6)
list_5.append(7)
```

```
list_5
```

```
[1, 2, 3, 4, 5, 6, 7]
```

```
list_5.pop(0)
```

```
1
```

```
list_5.pop(0)
```

```
2
```

```
list_5
```

```
[3, 4, 5, 6, 7]
```

3. 리스트(list)_참고

- 패킹 : 한 변수에 여러 개의 데이터를 넣는 것
- 언패킹 : 한 변수의 데이터를 각각의 변수로 반환

```
e = [1,2,3] # 1,2,3이라는 변수를 t에 패킹
e
```

```
[1, 2, 3]
```

```
f,g,h  = e # e에 있는 값 1,2,3을 변수 a,b,c에 언 패킹
print(f,g,h)
```

```
1 2 3
```

연습문제 3

1번

다음과 같은 출력값을 만들어보자

In [ ]:

```
a = '아메리카노를'
b = '좋아한다'
c = '많이'
```

In [ ]:

```
# 정답을 적어주세요
```

Out[2]: '아메리카노를많이많이많이좋아한다'

연습문제 3-1

2번

좋아하는 음식 5개를 food라는 리스트에 담아보자

```
: # 정답을 적어주세요
```

3번

리스트 food의 가장 앞에 파이썬이, 가장 뒤에는 집이라는 단어가 오도록 수정해보자

```
In [ ]: # 정답을 적어주세요
```

```
In [ ]: # 아래의 주석을 풀고 실행시켜보세요
        # food
```

4번

pop을 이용하여 파이썬을 삭제해보자

```
In [ ]: # 정답을 적어주세요
```

```
In [ ]: # 아래의 주석을 풀고 실행시켜보세요
        # food
```

5번

remove를 이용하여 집을 삭제해보자

4. 튜플(tuple)

- ( )을 이용하여 튜플(tuple)임을 선언

```
[42] tuple_1 = ()
     tuple_2 = tuple()
```

```
[43] tuple_3 = (1, 2)
     tuple_4 = (3,)
     tuple_5 = (4, 5, (6, 7))
     tuple_6 = 8, 9, 10
```

- 한 개의 요소만 사용 할 때에는 콤마(,)를 반드시 사용해야함
- 괄호가 없어도 튜플로 선언할 수 있음

4. 튜플(tuple)

- 인덱싱과 슬라이싱이 가능
- 리스트와 달리 요소를 삽입/삭제/수정 할 수 없음

```
[44] tuple_3 = (1, 2)
     tuple_3[0] = 3
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-44-c8372825bae8> in <module>()
      1 tuple_3 = (1, 2)
----> 2 tuple_3[0] = 3

TypeError: 'tuple' object does not support item assignment
```

SEARCH STACK OVERFLOW

4. 튜플(tuple)

- tuple의 연산

```
[45] tuple_3 = (1, 2)
     tuple_4 = (3, )
     tuple_3 + tuple_4
```

```
(1, 2, 3)
```

```
[46] tuple_4 * 2
```

```
(3, 3)
```

5. 딕셔너리(dictionary)

- dictionary
  { }을 이용하여 딕셔너리(dictionary)임을 선언

```
[47] dict_1 = {'key' : 'value'}
     dict_2 = dict()
```

- 딕셔너리는 key와 value의 값으로 이뤄진 자료형
- 순서가 있는 자료형이 아니며, key를 통해 value값에 접근이 가능
- key는 고유한 값으로 중복될 수 없음
- value는 중복 가능

5. 딕셔너리(dictionary)

- dictionary 추가/삭제/수정

1. 요소 추가하기 : 새로운 key에 value를 할당

```
[48] dict_3 = {'홍길동' : 100, '홍계월' : 200}
```

주의 인덱스 대신 key 사용

```
[49] dict_3['슈퍼맨'] = 300
     dict_3
```

```
{'슈퍼맨': 300, '홍계월': 200, '홍길동': 100}
```

2. 요소 수정하기 : 덮어쓰기

```
[51] dict_3['홍길동'] = 1
     dict_3
```

```
{'홍계월': 200, '홍길동': 1}
```

5. 딕셔너리(dictionary)

- dictionary 추가/삭제/수정

  3. 요소 삭제하기

```
[50] del dict_3['슈퍼맨']
     dict_3
```

```
{'홍계월': 200, '홍길동': 100}
```

주의 인덱스 대신 key 사용

del : 파이썬 자체의 명령어로 메모리 자체에서 삭제

5. 딕셔너리(dictionary)

- key와 value에 접근하기

1. key에 접근하기 : keys()

```
[1] dict_3 = {'홍길동' : 100, '홍계월' : 200, '슈퍼맨' : 150, '배트맨' : 250}
    print(dict_3.keys())
    print(type(dict_3.keys()))
```

```
dict_keys(['홍길동', '홍계월', '슈퍼맨', '배트맨'])
<class 'dict_keys'>
```

2. value에 접근하기 : values()

```
[2] print(dict_3.values())
    print(type(dict_3.values()))
```

```
dict_values([100, 200, 150, 250])
<class 'dict_values'>
```

5. 딕셔너리(dictionary)

- key와 value에 접근하기

  3. key와 value에 함께 접근하기

```
[3]  print(dict_3.items())
     print(type(dict_3.items()))
```

```
dict_items([('홍길동', 100), ('홍계월', 200), ('슈퍼맨', 150), ('배트맨', 250)])
<class 'dict_items'>
```

주의

리스트 처럼 보이나 실제로는 dict의 개체 이므로 리스트의 메소드를 사용할 수 없음

리스트처럼 사용하고 싶다면 리스트로 형 변환을 해야함

```
[4]  list(dict_3.keys())[0]
```

```
'홍길동'
```

5. 딕셔너리(dictionary)

- key와 value에 접근하기

4. key와 value에 함께 접근하기

1) 인덱싱처럼 키값을 이용하여 바로 접근

2) 메소드 get을 사용

```
[5]  dict_3
```

```
{'배트맨': 250, '슈퍼맨': 150, '홍계월': 200, '홍길동': 100}
```

```
[6]  print(dict_3['배트맨'])
     print(dict_3.get('배트맨'))
```

```
250
250
```

5. 딕셔너리(dictionary)

- key와 value에 접근하기

4. key와 value에 함께 접근하기

```
[7]  print(dict_3['펭수'])
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
<ipython-input-7-43368d90b0e9> in <module>()
----> 1 print(dict_3['펭수'])

KeyError: '펭수'

SEARCH STACK OVERFLOW
```

→ 키 값을 이용하여 접근 할 경우,
해당 키가 없으면 오류 발생

```
[8]  print(dict_3.get('펭수'))
```

→ get을 통해 접근할 경우, None 반환

```
None
```

6. set(집합)

- set

  {}을 이용하여 set을 선언

```
[9]  set_1 = set()
     set_2 = set('Hello')
     set_3 = set([1, 2, 3])
     set_4 = {3,5, 'hi'}
```



```
[10] set_2

     {'H', 'e', 'l', 'o'}
```

```
[11] set_3

     {1, 2, 3}
```

```
[12] set_4

     {3, 5, 'hi'}
```

- 중복을 허용하지 않음
- 순서가 없음

6. set(집합)

- set의 연산

  1. 교집합

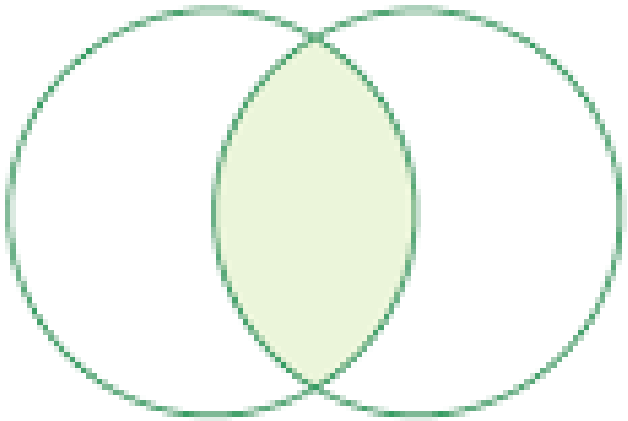
```
[13] set_5 = {1,2,3,4}
     set_6 = {4,5,6,7}
```

```
[14] set_5 & set_6
```

```
{4}
```

```
[15] set_5.intersection(set_6)
```

```
{4}
```



(a) 교집합

6. set(집합)

- set의 연산

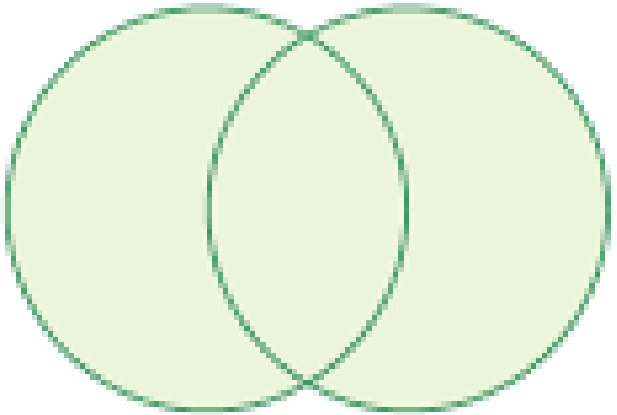
  2. 합집합

```
[16] set_5 | set_6
```

```
{1, 2, 3, 4, 5, 6, 7}
```

```
[17] set_5.union(set_6)
```

```
{1, 2, 3, 4, 5, 6, 7}
```



(b) 합집합

6. set(집합)

- set의 연산

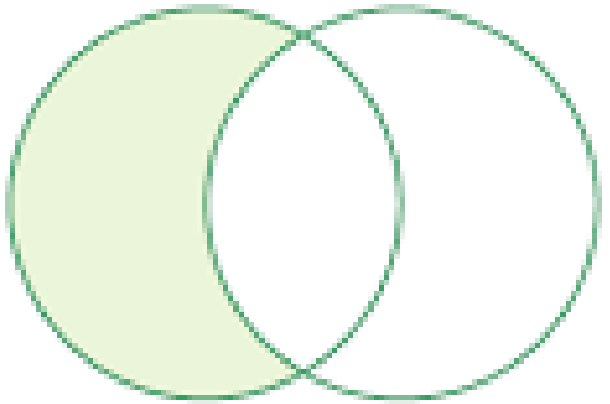
  3. 차집합

```
[18] set_5 - set_6
```

```
{1, 2, 3}
```

```
[19] set_5.difference(set_6)
```

```
{1, 2, 3}
```



(c) 차집합

6. set(집합)

- set의 메소드

1. 한 개 추가하기 : add

```
[22] set_6 = {4, 5, 6, 7}
     set_6.add(8)
     set_6
```

```
{4, 5, 6, 7, 8}
```

2. 여러 개 추가하기 : update

```
[23] set_6.update([1,2,3])
     set_6
```

```
{1, 2, 3, 4, 5, 6, 7, 8}
```

3. 한 개 지우기 : remove

```
[24] set_6.remove(3)
     set_6
```

```
{1, 2, 4, 5, 6, 7, 8}
```

7. 불(bool)

- bool : 참과 거짓을 나타내는 자료형

  참 : True

  거짓 : False

```
[26] True
```

```
True
```

```
[27] type(False)
```

```
bool
```

```
[28] 1 == 1
```

```
True
```

```
[29] 1 > 2
```

```
False
```

비교 연산자	의미
==	같음
!=	같지 않음
>, >=, <, <=	크거나 같음/ 작거나 같음

→ 비교 연산자와 조건문의 반환값으로 불자료형이 사용됨

7. 불(bool)

- bool : 참과 거짓을 나타내는 자료형

자료형에도 참/거짓이 있음

자료	참 / 거짓
'python'	참
''	거짓
[1,2,3]	참
[ ]	거짓
( )	거짓
{ }	거짓
1	참
0	거짓
None	거짓

```
[30] bool(1)
```

```
True
```

```
[31] bool(' ')
```

```
True
```

```
[32] bool('')
```

```
False
```

연습문제 4)

a="20190505chicken19000"는 잘못 쓰여진 변수이다.

2019는 year이라는 변수에, 그 다음 0505를 day라는 변수에,

chicken을 menu라는 변수에, 19000을 money라는 변수에

인덱싱을 사용하여 각각 저장하고 출력하시오.

```
a = "20190505chicken19000"

# code 작성
year = ?
day = ?
menu = ?
money = ?

# code 종료
```

4. 조건문

- 문제해결 프로세스
- 조건문이란 ?
- if / elif / else
- 비교연산자
- 논리연산자 : and / or / not
- 요소인지 파악 하기 : in / not in
- if문 안에 if문
- Pass
- Fluent python



문제해결 프로세스


과학적 문제해결 프로세스
PART1.
문제발생의 원인 규명 과정
PART2.
해법의 수립과 적용 과정
문제 인식
가설 설정
실증
해법 규명

조건문이란 ?

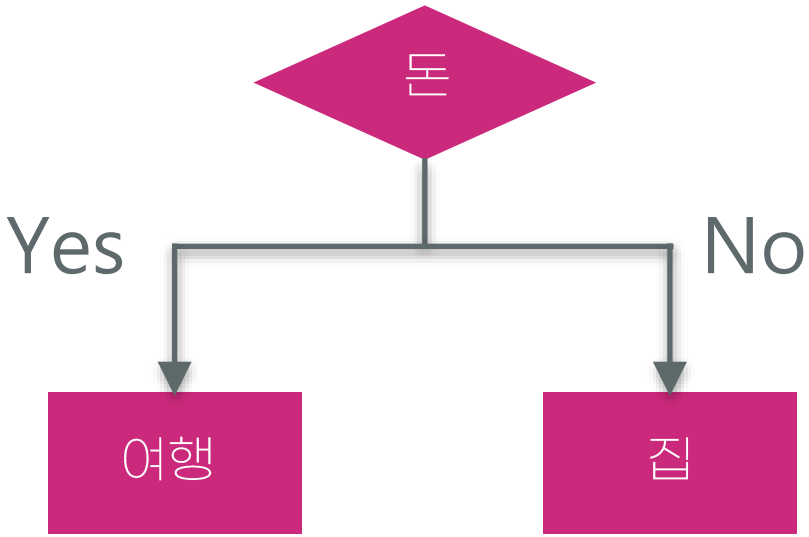
- 조건에 따라 특정한 동작을 하게하는 명령어
- 프로그램 예시 in 생활

  - 자동차 앞차 간격이 100M 이하면 속도를 50km 이하로 낮춰라
  - 사용자가 어린이인 경우에는 VOD를 플레이 하지 마라
  - 패턴이 5회 틀리면 10초 동안 대기 상태로 만들어라

- 조건문은 조건을 나타내는 기준과 실행해야 할 명령으로 구성됨
- 조건의 참, 거짓에 따라 실행해야 할 명령이 수행되거나 되지 않음
- 파이썬은 조건문으로 if, else, elif 등의 명령 키워드를 사용함.

조건문이란 ?

- if / else




```
[ ]  money = True

     if money :
         print("여행을 간다")
     else :
         print("집에서 쉰다")
```

```
여행을 간다
```

if 조건식:

코드

들여쓰기 or 탭

공식 4칸

조건문이란 ?

- if / else

```
[ ]  if 조건 :
         수행 할 문장1
         수행 할 문장2
     else :
         수행 할 문장3
         수행 할 문장4
```

주의

콜론(:)과 들여쓰기에 주의할 것

```
[ ]  money = True

     if money :
         print("여행을 간다")
     else :
         print("집에서 쉰다")
```

```
여행을 간다
```

조건문이란 ?

- if / elif / else

```
[ ]  if 조건1 :
         수행 할 문장1
     elif 조건2 :
         수행 할 문장2
     else :
         수행 할 문장3
```

```
[ ]  money = 20000

     if money < 5000 :
         print('라면을 먹는다')
     elif 5000 < money < 25000 :
         print('치킨을 먹는다')
     elif 25000 < money < 50000 :
         print('삼겹살을 먹는다')
     else :
         print('소고기를 먹는다')
```



치킨을 먹는다

조건문이란 ?

- 비교 연산자

```
[ ]  a = 3
     b = 5

     a < b
```



```
True
```

```
[ ]  c = 3
     d = 3

     c <= d
```



```
True
```

```
[ ]  c != d
```



```
False
```

조건문이란 ?

- 비교 연산자

```
a=10
b=10
```

```
a==b
```

```
True
```

```
a is b
```

```
True
```

```
a=300
b=300
```

```
a==b
```

```
True
```

```
a is b
```

```
False
```

```
a=-5
b=-5
a is b
```

```
True
```

```
a=-6
b=-6
a is b
```

```
False
```

```
a = [1,2,3,4,5]
b=a[:]
```

```
a
```

```
[1, 2, 3, 4, 5]
```

```
b
```

```
[1, 2, 3, 4, 5]
```

```
a is b
```

```
False
```

```
import copy
a = [1,2,3,4,5]
b=copy.deepcopy(a)
```

```
a
```

```
[1, 2, 3, 4, 5]
```

```
b
```

```
[1, 2, 3, 4, 5]
```

```
a is b
```

```
False
```

조건문이란 ?

- 논리연산자 : and / or / not

```
[ ] True and True
```

```
True
```

```
[ ] True and False
```

```
False
```

1) and

비교하는 대상이 모두 참이여야만

True를 반환

```
boolean_list = [True,False,True,False,True]

all(boolean_list)
```

```
False
```

조건문이란 ?

- 논리연산자 : and / or / not

```
[ ] True or True
```

True

```
[ ] True or False
```

True

```
[ ] False or False
```

False

2) or

비교하는 대상 중 하나만 참이여도

True 반환

```
boolean_list = [True,False,True,False,True]

any(boolean_list)
```

True

조건문이란 ?

- 논리연산자 : and / or / not

```
[ ] not True
```

```
False
```

```
[ ] not False
```

```
True
```

3) not

참이면 False, 거짓이면 True을 반환

조건문이란 ?

- 논리연산자 : and

```
a=100

if a>0 and (a%2)==0:
    print('a is even')
```

```
a is even
```

```
5 and 7
```

```
7
```

```
7 and 13
```

```
13
```

- and : 조건이 True/ False 로 반환이 됨.
- & : 비트연산자로 작동 됨.

```
7 & 13 # 0111&1101
```

```
5
```

```
13 & 7# 0111&1101
```

```
5
```

조건문이란 ?

- 논리연산자 : or

```
a=100

if a>0 or (a%2)!=0:
    print('a is even')
```

```
a is even
```

```
5 or 7
```

```
5
```

```
7 or 13
```

```
7
```

- or : 조건이 True/ False 로 반환이 됨.
- | : 비트연산자로 작동 됨.

```
7 | 13 # 0111|1101
```

```
15
```

```
13 | 7# 0111|1101
```

```
15
```

조건문이란 ?

- 요소인지 파악하기 : in / not in

```
[ ] e = [1, 3, 5, 7]
```

```
[ ] 0 in e
```

```
False
```

```
[ ] 1 in e
```

```
True
```

1) x in s

x가 s의 요소인가

```
[ ] 2 not in e
```

```
True
```

```
[ ] 3 not in e
```

```
False
```

2) x not in s

x가 s의 요소가 아닌가

조건문이란 ?

- if문 안에 if문

  조건 안에 조건을 만들 수 있음

```
[ ]  money = 20000
     card = True
```

```
[ ]  if card :
         if money < 30000 :
             print("삼겹살을 먹는다")
         else :
             print("소고기를 먹는다")
     else :
         if money <= 1000 :
             pass
         else :
             print("라면을 먹는다")
```

삼겹살을 먹는다

- pass

  조건을 만족해도 아무일도 일어나지 않게 하려면?

```
[ ]  money = 1000
     card = False
```

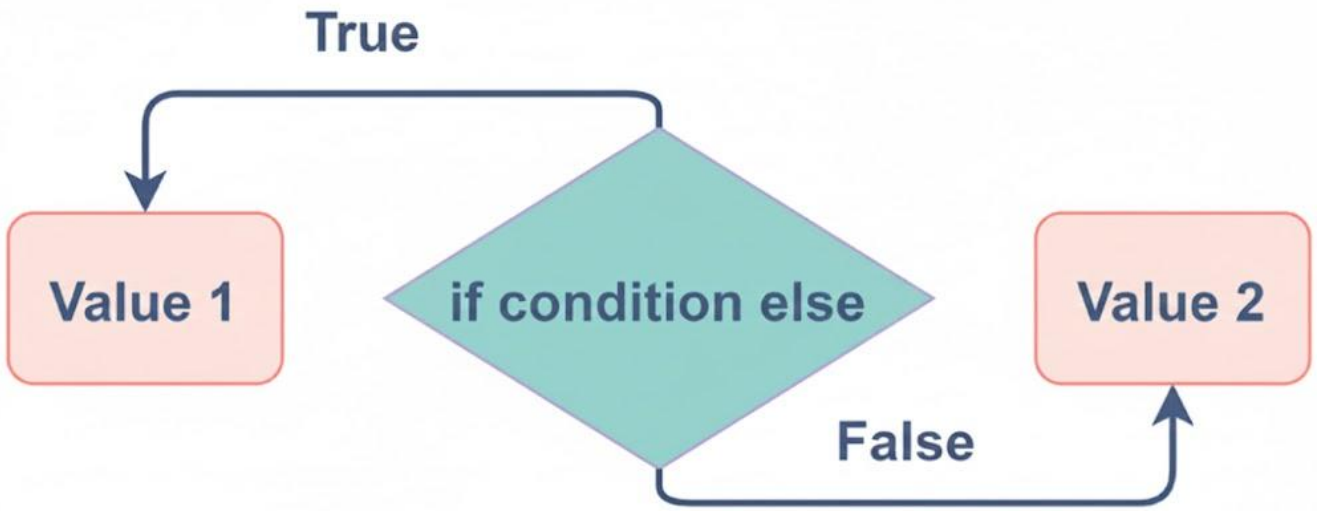
```
[ ]  if card :
         if money < 30000 :
             print("삼겹살을 먹는다")
         else :
             print("소고기를 먹는다")
     else :
         if money <= 1000 :
             pass
         else :
             print("라면을 먹는다")
```

조건문이란 ?

- 삼항 연산자(Ternary Operators)

  - Condition(조건)이 참이면 true_value 값을 취하고 거짓이면 false_value 값을 취함

- [true_value] if [condition] else [false_value]




```
value = 20

is_even = True if value %2 ==0 else False

print(is_even)
```

```
True
```

출처 : https://www.educative.io/edpresso/what-is-the-ternary-operator-in-python

- Formatting

1. %

코드	설명
%d	int
%f	float
%s	string
%c	문자 1개
%%	% 자체

```
[4]  num = 1

[5]  print('문자열 포맷팅 하기 예시 %d' %num)

     문자열 포맷팅 하기 예시 1

[6]  ch = '이름'

[7]  print('내 이름은 : %s' %ch)

     내 이름은 : 이름

[8]  fl = 2.566132

[9]  print('반올림 하기 : %0.1f' %fl)

     반올림 하기 : 2.6
```

- Formatting

1. %

코드	설명
%d	int
%f	float
%s	string
%c	문자 1개
%%	% 자체

괄호로 묶어주면 두 개 이상도 포맷팅 할 수 있음

```
[10] a = 3
     b = 5
```

```
[11] print('%d 와 %d 를 더하면 %d' %(a,b,a+b))
```

```
3 와 5 를 더하면 8
```

- Formatting

2. format

'{ }'.format(포맷팅 할 변수)

```
[12] print('문자열 포맷팅 하기 예시 {}'.format(2))

     문자열 포맷팅 하기 예시 2
```

```
[14] print('내 이름은 : {}'.format(ch))

     내 이름은 : 이름
```

- Formatting

2. format

- 괄호로 묶어주면 두 개 이상도 포맷팅 할 수 있음

```
a = '류'
b = '영표'
```

```
print('성은 {}, 이름은 {}'.format(a,b))
```

```
성은 류, 이름은 영표
```

- 인덱스를 통해 순서를 지정할 수도 있음

```
print('성은 {0}, 이름은 {1}'.format(a,b))
```

```
성은 류, 이름은 영표
```

```
print('Frist name : {1}, Last name : {0}'.format(a,b))
```

```
Frist name : 영표, Last name : 류
```

- Formatting

2. format

  - 이름을 통해서 포맷팅 할 수도 있음

```
[20] print('나는 오늘 {음료}를 {개수} 잔이나 마셨다'.format(음료='커피', 개수=3))

     나는 오늘 커피를 3 잔이나 마셨다
```

  - 이 밖에도 정렬이나 공백 이용하는 등 다양한 포맷팅이 가능

- Formatting

3. f-string(f 문자열 포맷팅)

f '{ }'

```
[28] year = 2020
```

```
[29] f'올해는 {year}'
```

```
'올해는 2020'
```

- Formatting

3. f-string(f 문자열 포맷팅)

- 다른 포맷팅과는 달리 표현식을 지원함

```
[39] drink = '커피'
     nums = 3
```

```
[40] f'나는 오늘 {drink}를 {nums + 1} 잔이나 마셨다'
```

```
'나는 오늘 커피를 4 잔이나 마셨다'
```

```
# 문자열 맨 앞에 f를 붙이고, 출력할 변수, 값을 중괄호 안에 넣습니다.
s = 'coffee'
n = 5
result1 = f'저는 {s}를 좋아합니다. 하루 {n}잔 마셔요.'
print(result1)
```

```
저는 coffee를 좋아합니다. 하루 5잔 마셔요.
```

- Formatting

```
[37] '나는 오늘 %s 를 %d + 1 잔이나 마셨다' %('커피', 3)

     '나는 오늘 커피 를 3 + 1 잔이나 마셨다'
```

```
[38] '나는 오늘 {음료}를 {개수+1} 잔이나 마셨다'.format(음료='커피', 개수=3)

     ---------------------------------------------------------------------------
     KeyError                                  Traceback (most recent call last)
     <ipython-input-38-21356a636e64> in <module>()
     ----> 1 '나는 오늘 {음료}를 {개수+1} 잔이나 마셨다'.format(음료='커피', 개수=3)

     KeyError: '개수+1'

     SEARCH STACK OVERFLOW
```

```
[39] drink = '커피'
     nums = 3
```

```
[40] f'나는 오늘 {drink}를 {nums + 1} 잔이나 마셨다'

     '나는 오늘 커피를 4 잔이나 마셨다'
```

- Padding

  : 여유 공간을 지정하여 글자배열 + 소수점 자릿수를 맞추기

```
print('제품:%8s, 제품가격:%.5f.' %('Peer',3.241)) #8칸을 비우고, 5개의 소수점을 찍어라.
print('제품:{0:8s}, 제품가격:{1:.5f}.'.format('Peer',3.241))
```

```
제품:     Peer, 제품가격:3.24100.
제품:Peer     , 제품가격:3.24100.
```

```
print('제품:%16s, 제품가격:%10.5f.' %('Peer',3.241)) #10칸 공간,
print('제품:{0:<8s}, 제품가격:{1:>.10f}.'.format('Peer',3.241)) #제품 : 8칸을 비우면서 왼쪽으로 정렬,
#제품가격은 10칸을 비우면서 오른쪽으로 정렬.
```

```
제품:             Peer, 제품가격:    3.24100.
제품:Peer     , 제품가격:3.2410000000.
```

연습문제 5-1

b로 a를 나눈 나머지가 3 초과면 실패, 3이면 무승부, 3 미만이면 성공이 출력되도록 만들어 보자

```
a = 34
b = 4
```

```
# 정답을 적어주세요
```

성공

연습문제 5-2

[홀수 짝수 판별기]

사용자로부터 정수를 하나 입력받아

입력한 정수가 홀수인지 짝수인지 판별하여라.

(** 0은 짝수라 하자.)

[출력결과]

```
정수를 입력해주세요: 5
입력하신 5는 홀수입니다.

짝수를 입력해주세요: 10
입력하신 10은 짝수입니다.
```

연습문제 5-3

무슨 학교 다니세요?

태어난 연도를 계산하여 학교 종류를 맞추는 프로그램

```
당신이 태어난 년도를 입력하세요.
2000
학생이 아닙니다.
```

- 나이는 현재년도-태어난 년도+1로 계산
- 26세 이하 20세 이상이면 “대학생“, 20세 미만 17세 이상이면 “고등학생”, 17세 미만 14세 이상이면 “중학생“, 14세 미만 8세 이상이면 “초등학생”
- 그 외의 경우는 “학생이 아닙니다 " 출력

연습문제 5-4

무슨 학교 다니세요?

태어난 연도를 계산하여 학교 종류를 맞추는 프로그램

```
당신이 태어난 년도를 입력하세요.
2000
학생이 아닙니다.
```

- 나이는 현재년도-태어난 년도+1로 계산
- 26세 이하 20세 이상이면 “대학생“, 20세 미만 17세 이상이면 “고등학생”, 17세 미만 14세 이상이면 “중학생“, 14세 미만 8세 이상이면 “초등학생”
- 그 외의 경우는 “학생이 아닙니다 " 출력

연습문제 5-5

<평균과 과락을 함께 판별하기>

변수는 kor, eng, math에 세 과목이 저장되어 있다.

다음 두 조건을 만족하면 "합격", 아니면 "불합격“

- 평균: 70점 이상, 모든 과목이 40점 이상

kor = 80

eng = 75

math = 60

조건문 작성

연습문제 5-6

다음 dictionary에서 나이가 20 이상이고 학생이면 “대학생 또는 성인 학생“ ,
20 미만이고 학생이면 “미성년 학생“ 학생이 아니면 “일반인”을 출력하세요.

person = {"name": "Yuna", "age": 19, "is_student": True}

5. 반복문

- while 문
- for 문
- break, continue



반복문

- 정해진 동작을 반복적으로 수행하게 하는 명령문
- 프로그램 예시 in 생활
  - 100명의 학생에 성적을 산출할 때
  - 검색 결과를 출력해 줄 때
  - 워드에서 단어 바꾸기 명령 실행
- 반복문은 반복 시작 조건, 종료 조건, 수행 명령으로 구성됨
- 반복문 역시 반복 구문은 들여쓰기와 block으로 구분됨
- 파이썬은 반복문으로 for, while 등의 명령 키워드를 사용함

반복문 - while

- 조건문이 참인 동안 반복해서 문장을 수행함

```
[ ]  while 조건 :
         수행할 문장1
         수행할 문장2
```

```
[ ]  # 10이하의 짝수 프린트하기
     i = 1                          -> 조건문에 쓰일 변수

     while i <= 10 :                -> 조건문
         if i % 2 == 0 :
             print(i)
         i += 1                     -> 변수를 증감
```

```
2
4
6
8
10
```

반복문 - while

- break

  반복문에서 빠져나오고 싶다면?

```
[ ]  # 100번째 방문자 찾기
     i = 90

     while i :   -> 변수는 0이 아니라면 항상 참(무한 반복)
         i += 1
         if i == 100 :
             print("축하합니다. %d번째 방문자입니다." % i)
             break
     print("감사합니다. 이벤트가 종료되었습니다.")   -> 반복문이 종료되었으므로 수행됨
```

```
축하합니다. 100번째 방문자입니다.
감사합니다. 이벤트가 종료되었습니다.
```

break를 사용하면 반복문이 더 이상 작동하지 않고 멈춤

반복문 - while

- break

  반복문의 첫 부분으로 돌아가고 싶다면?

```
[ ]  i = 0

     while i < 11 :
         i += 1
         if i == 6 :
             continue
         if i % 2 == 0 :
             print(i)
```

-> 6일 때는 continue를 사용해 반복문의 가장 처음으로 가도록 함

-> 6일 때는 짝수임에도 불구하고 프린트가 안됨

```
2
4
8
10
```

반복문 - for

```
[ ]  for 변수 in range(변수가 속한 자료형, 혹은 변수의 범위) :
         수행해야할 문장1
         수행해야할 문장2
```

주의
콜론(:)과 들여쓰기에
주의할 것.

```
[ ]  for x in range(0,5) :
         print(x)
```

```
0
1
2
3
4
```

반복문 - for

```
[ ]  for x in range(0,5) :
         print(x)
```

```
0
1
2
3
4
```

변수의 범위 지정하기

1) 숫자로 범위주기

2) 자료형으로 범위주기

반복문 - for

- 변수의 범위 지정하기

  1) 숫자로 범위 : range( )

range(시작, 끝, 간격)

- 시작부터 끝 -1 까지
- 간격이 1이라면 생략가능
- 0부터 시작하여 1씩 증가한다면 시작과 간격 생략가능

반복문 - for

- 변수의 범위 지정하기

  1) 숫자로 범위 : range( )

  연습문제) 5에서 0까지 카운트 다운

```
[ ]  # 카운트 다운
     for count in range(5, -1, -1):
         print(count)
```

-> 하나씩 작아지므로 간격은 -1

-> 0까지 출력해야 하므로 -1이 끝 인덱스가 되어야 함

```
5
4
3
2
1
0
```

반복문 - for

- 변수의 범위 지정하기

  2) 자료형

```
[ ]  word = 'Hello!'

     for w in word:
         print(w)
```

```
H
e
l
l
o
!
```

```
[ ]  for a, b in [(2,1), (2,2), (2,3), (2,4)] :
         print(a*b)
```

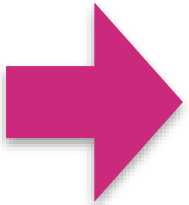
```
2
4
6
8
```

반복문 - for

- 이중 반복문

```
[ ]  # 구구단 2단과 3단을 출력하기

[5]  for i in range(2,4) :
         print('===', i, '단 ===')
         for j in range(1, 10) :
             print(i * j)
```



```
=== 2 단 ===
2
4
6
8
10
12
14
16
18
=== 3 단 ===
3
6
9
12
15
18
21
24
27
```

break, continue

for, while에서 제어흐름을 벗어나기 위해 사용

break

for, while을 완전히 중단

continue

처음으로 돌아가 다음반복 수행

break, continue, pass

for 문에서의 예제

break

```
for i in range(5):
  if (i==3):
    break
  print(i,end = '')
```

012

continue

```
for i in range(5):
  if (i==3):
    continue
  print(i,end = '')
```

0124

```
for i in range(5):
  if (i==3):
    pass
  print(i,end = '')
```

01234

break, continue

while 문에서의 예제

break

```
i=0

while i<30:
  if i==20:
    break
  print(i,end ='')
  i +=1
```

```
012345678910111213141516171819
```

continue

```
i=0

while i<30:
  if i==20:
    continue
  print(i,end ='')
  i +=1
```

```
012345678910111213141516171819
```

Toy example

- 1부터 10까지 합을 구하시오. (range 함수를 이용하시오)

연습문제 1

(1) 사용자로부터 정수를 입력받아, 해당 정수만큼 “안녕”을 출력하세요.

(2) 사용자로부터 정수를 입력받아, 입력된 정수 만큼 별 찍기

(1) [출력결과]

```
정수를 입력해주세요: 5
안녕
안녕
안녕
안녕
안녕
```

(2) [출력결과]

```
정수를 입력해주세요: 5
*
**
***
****
*****
```

연습문제 2

(1) 계단형, n을 입력받아, n 만큼 줄을 만들고 계단 형태로 별찍기

(2) 삼각형, 왼쪽 아래가 직각인 n만큼의 높이를 가지는 직각삼각형

(2)-(1), 오른쪽 아래가 직각인 n만큼의 높이를 가지는 직각삼각형

(3) 역삼각형, 왼쪽 위가 직각인 n만큼의 높이를 가지는 직각삼각형

(3)-(1), 오른쪽 위의 직각인 n만큼의 높이를 가지는 직각삼각형

(4) 피라미드, n만큼의 높이를 가지는 홀수개의 별 피라미드

(1) [출력결과]

```
n:5
*****
 *****
  *****
   *****
    *****
```

(2) [출력결과]

```
n : 5
*
**
***
****
*****
```

(2-1) [출력결과]

```
n : 5
    *
   **
  ***
 ****
*****
```

(3) [출력결과]

```
n : 5
*****
****
***
**
*
```

(3-1) [출력결과]

```
n : 5
*****
 ****
  ***
   **
    *
```

(4) [출력결과]

```
n : 5
    *
   ***
  *****
 *******
*********
```

연습문제 3

(1) x = [3, 6, 9, 20, -7, 5] 의 값의 모든 요소에 10을 곱한뒤 출력하세요

심화 : 출력과 리스트 x 의 값에도 모두 10을 곱해주세요

(2) y = {“math”: 70, “science”: 80, “english”: 20} 의 값의 모든 요소에 10을 더한뒤 출력하세요

심화 : 출력과 딕셔너리 y 의 값에도 모두 10을 더해주세요

(3) 숫자를 입력받고 입력받은 정수의 구구단을 출력하세요

(3) [출력결과]

```
정수를 입력해주세요: 5
5*1 = 5
5*2 = 10
5*3 = 15
...

...
```

연습문제 4

- 1~100 임의의 숫자를 맞추시오.

```
숫자를 맞춰보세요(1~100)
50
숫자가 너무 큽니다.
30
숫자가 너무 큽니다.
10
숫자가 너무 큽니다.
1
숫자가 너무 작습니다.
4
숫자가 너무 작습니다.
5
숫자가 너무 작습니다.
정답입니다. 입력한 숫자는 5입니다.
```

연습문제 5

(1) word = [“school”, “game”, “piano”, “science”, “hotel”, “mountain”] 중 글자수가 6글자 이상인 문자를 모아 새로운 리스트를 생성하세요

(2) 구구단을 1단부터 9단까지 출력하세요

연습문제 6

1-100 까지 숫자중

3과 5의 공배수일경우 “3과 5의 공배수”

나머지 숫자중 3의배수일경우 “3의배수”

나머지 숫자중 5의배수일경우 “5의배수”

모두 해당되지 않을경우 그냥숫자

를 출력하세요

심화 : 1-입력한 숫자까지의 숫자 중

연습문제 7

사용자로부터 숫자를 계속 입력받다가

s or S 를 입력하면 합계출력

(단, 다른 문자는 받을 수 없다.)

값을 입력해주세요30

값을 입력해주세요20

값을 입력해주세요50

값을 입력해주세요40

값을 입력해주세요30

값을 입력해주세요s

합계는 ? 170

연습문제 8

사람 별 평균을 구하라.

- kor_score = [39,69,20,100,80]
- math_score = [32,59,85,30,90]
- eng_score = [49,70,48,60,100]
- midterm_score = [kor_score, math_score, eng_score]

	A	B	C	D	E
국어점수	39	69	20	100	80
수학점수	32	59	85	30	90
영어점수	49	70	48	60	100

6. 함수

- 함수란?
- 함수 정의하기

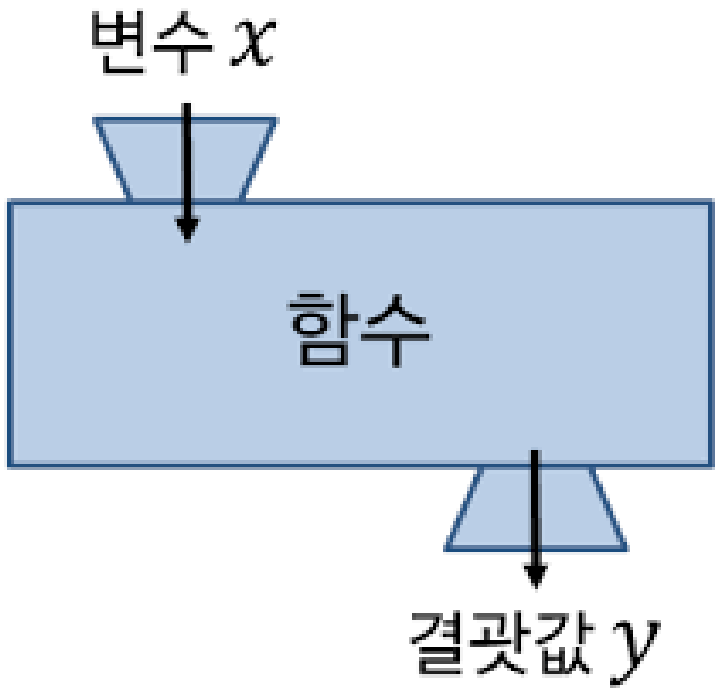
- 함수(Function)
- 연습문제

- 더 알아보기
- 연습문제
- 함수와 변수



함수(Function)

- 함수란?




```
[ ]  def 함수이름(함수인자) :
         수행할 문장1
         수행할 문장2
         return 반환값
```

- 일정한 프로세스를 할 수 있도록 하는 것
- 코드를 논리적인 단위로 분리
- 여러 개의 문장을 하나로 묶어 주는 역할

함수(Function)

```
def 함수이름( 파라미터 ) :
    함수의 내용
    Return 반환값
```

- 함수이름: 사용자가 정의하는 함수 이름, 기존에 사용되는 예약어들을 제외하고 사용
- 파라미터: 함수 안에서 사용할 변수들 (생략 가능)
- return: 함수 안에서 모든 연산을 마친 후 반환할 값(생략 가능)

함수(Function)

pop quiz! 두 개의 input를 받으면 더하기를 하여 값을 돌려주는 함수를 만들어 보자.

```
[ ]  def add(a, b) :
         result = a + b
         return result
```

→ 함수 정의

```
[ ]  c = add(3, 5)
     c
```

```
8
```

→ 함수 호출

함수(Function)

1. 매개변수와 return값이 모두 있는 함수

```
[ ]  def add(a, b) :
         result = a + b
         return result
```

```
[ ]  c = add(3, 5)
     c
```

```
8
```

```
def add(a, b):
    """두 수의 합을 반환"""
    return a + b

def greet(name="student"):
    """기본 인자, 키워드 인자 예시"""
    return f"Hello, {name}"

print(add(2, 3))
print(greet())
print(greet(name="youngpyo"))
```

```
5
Hello, student
Hello, youngpyo
```

2. return값이 비어있는 함수

```
[ ]  def sub(a, b) :
         print('뺄셈의 결과는 %d입니다.' %(a-b))
         return
```

```
[ ]  d = sub(1, 2)
```

```
뺄셈의 결과는 -1입니다.
```

```
[ ]  print(d)
```

```
None
```

3. return이 없는 함수

```
def show_sum(a, b):
    print("덧셈 결과:", a + b)

result = show_sum(3, 2)
print("함수 반환값:", result)
```

```
덧셈 결과: 5
함수 반환값: None
```

```
def f(x):
  print(x+10)
```

```
f(10)
```

```
20
```

```
c = f(10)
```

```
20
```

```
print(c)
```

```
None
```

4. 매개변수와 return값이 모두 없는 함수

```
[ ]  def start():
         print("Hello World!")
```

```
[ ]  start()
```

```
Hello World!
```

함수(Function)

예제) 구구단. 함수를 사용하지 않을 때 VS 함수를 사용할 때

```
for i in range(1,10):
  print(f'==== {i} 단====')
  for j in range(1,10):
    print(i*j)
```

```
==== 1 단====
1
2
3
4
5
6
7
8
9
```

```
def gugu(num):
    for i in range(1, 10):
        print(f'{num} x {i} = {num * i}')
```

```
gugu(3)
```

```
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
3 x 4 = 12
3 x 5 = 15
3 x 6 = 18
3 x 7 = 21
3 x 8 = 24
3 x 9 = 27
```

Python naming rule

- 변수와 Class 명 함수명을 짓는 방식이 존재.
- Snake-case : 단어 사이를 언더바(_)로 이어줌(ex. user_list)

  : 식별자가 마치 뱀처럼 생겼다고 붙혀지 이름
- Camale-case : 단어의 첫글자를 대문자로 사용함. (ex. UserList)

  : 식별자가 모양이 마치 낙타모양 같아서 붙여진 이름

함수의 매개변수 전달 방식

1. 불변(Immutable)
 : 함수 안에서 바꿔도 원본은 그대로
 -> int, str, tuple

```
def bump(x):
    x = x + 1
    return x

n = 10
print(bump(n))
print(n)
```

```
11
10
```

2. 가변(mutable)
 : 함수 안에서 바꾸면 원본도 바뀜
 -> list, dict, set

```
def push_one(lst):
    lst.append(1)
    return lst

arr = [0]
print(push_one(arr))
print(arr)
```

```
[0, 1]
[0, 1]
```

3. 안전 패턴(원본 보존)

```
def safe_push_one(lst):
    new_lst = lst.copy() #얕은 복사
    new_lst.append(1)
    return new_lst

arr = [0]
print(safe_push_one(arr))
print(arr) #원본 보존
```

함수(Function)

3가지 전달 방식이 존재함.

값에 의한 호출 (Call by Value)	참조에 의한 호출 (Call by Reference)	객체 참조에 의한 호출 (Call by Object Reference)
함수에 인자를 넘길 때 값만 넘김	함수에 인자를 넘길 때 메모리 주소를 넘김	객체의 주소가 함수로 전달됨.
함수 내에 인자 값 변경시에 호출자에게 영향을 주지 않음	함수 내에 인자 값 변경이 되면 호출자의 값도 변경됨.	전달된 객체를 참조하여 변경 시 호출자에게 영향을 주나, 새로운 객체를 만들 경우 호출자에게 영향을 주지 않음.

함수(Function)

```
def Sandwich(eggs):
    eggs.append(1) # 기존 객체의 주소값에 [1] 추가
    eggs = [2, 3]

ham = [0]
Sandwich(ham)
print(ham) # [0, 1]
print(eggs)
```

```
[0, 1]
---------------------------------------------------------------
NameError                                 Traceback (most r
<ipython-input-4-15a09db8233a> in <cell line: 8>()
      6 Sandwich(ham)
      7 print(ham) # [0, 1]
----> 8 print(eggs)

NameError: name 'eggs' is not defined
```

```
# 1번 함수
def swap_value(x, y):
    temp = x
    x = y
    y = temp

# 2번 함수
def swap_offset(offset_x, offset_y):
    temp = a[offset_x]
    a[offset_x] = a[offset_y]
    a[offset_y] = temp

# 3번 함수
def swap_reference(list, offset_x, offset_y):
    temp = list[offset_x]
    list[offset_x] = list[offset_y]
    list[offset_y] = temp
```

```
#테스트
a = [1,2,3,4,5]
swap_value(a[1],a[2]) #swap이 발생 안함
print(a)
swap_offset(1,2) #swap이 발생
print(a)
swap_reference(a,1,2) #swap 발생
print(a)
```

연습문제 1

함수를 만들어 보자.

- 이 함수는 두 개의 숫자를 input으로 받으면 앞의 숫자를 뒤의 숫자로 나누는 함수이다.
- 나눗셈을 한 후에는 몫과 나머지 순으로 된 튜플 값을 반환한다.

```
def div(a, b):
# 정답을 적어주세요
```

```
div(10, 3)
```

```
(3, 1)
```

```
div(3, 5)
```

```
(0, 3)
```

```
div(1, 10)
```

```
(0, 1)
```

연습문제 2

함수를 만들어보자.

- 이 함수는 두 개의 숫자를 input으로 받으면 작은 수로 큰 수를 나눈 몫과 나머지를 반환하는 함수이다.
- 반환 값은 튜플로 되어 있으며 몫, 나머지 순으로 되어 있다.
- 단, 0으로 나누는 것은 불가하기 때문에 두 수 중에 작은 수가 0이라면 화면에 '0은 사용할 수 없습니다.'를 출력하고 종료되어야한다.

```
div2(3, 5)
```

```
(1, 2)
```

```
div2(0, 5)
```

```
0은 사용할 수 없습니다.
```

```
div2(10, 1)
```

```
(10, 0)
```

함수(Function)

- 더 알아보기

1. 매개변수의 초기값 미리 설정하기

```
[ ]  def function(a, n=2) :
         print("%d의 제곱은 %d 입니다." %(a, a**n))
```

```
[ ]  function(4)
```

```
4의 제곱은 16 입니다.
```

⚠ 초기값을 설정할 때는 매개변수 순서에 주의.

초기값이 없는 변수 → 초기값이 있는 변수 순으로 배치해야 함.

함수(Function)

- 더 알아보기

2. 매개변수가 몇 개가 필요할지 모를 때

```
[3]  def test(*args) :
       print(args)
```

→ 매개변수에 *args를 붙이면 함수가 받은 input 모두를 tuple으로 묶어줌

```
[4]  test(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
     (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

Quiz! 왜 tuple로 묶어줄까요?

```
[5]  test([1,2],3)
     ([1, 2], 3)
```

*(Asterisk)

- 더 알아보기

  2. 매개변수가 몇 개가 필요할지 모를 때

```
def add(**kwargs):
    print(kwargs)
```

→ 매개변수에 *kwargs를 붙이면 함수가 받은 input 모두를 dict으로 묶어줌

```
add(a =1,b=2, c=3)
```

```
{'a': 1, 'b': 2, 'c': 3}
```

Quiz! 왜 dict로 묶어줄까요?

연습문제 3

어떠한 string을 받으면 일정한 단위로 끊어서 화면에 출력하는 함수를 짜보자.
끊는 단위는 따로 정하지 않으면 2로 설정해보자.

Hint : input을 string 과 unit = 2 로 받고, while을 사용하고, 길이는 len 함수를 사용하도록 하자.

```
: func('테스트를 위한 문장입니다.')

테스
트를
 위
한
문장
입니
다.
```

```
: func('테스트를 위한 문장입니다.', 4)

테스트를
 위한
문장입니
다.
```

연습문제 4

add_all 함수를 짜봅시다

```
add_all(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
```

55

Hint : *args를 input으로 받으세요.

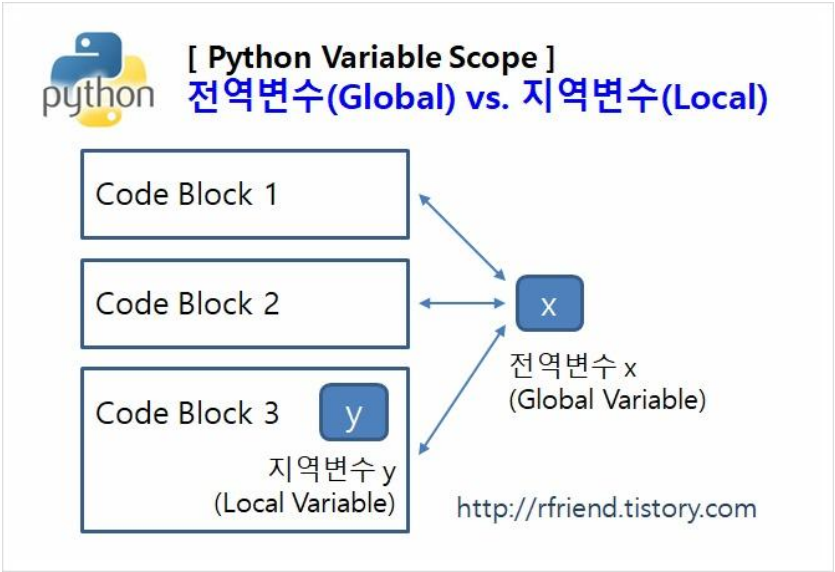
함수(Function)

- 함수와 변수

  Quiz! 함수 안에 선언한 변수를 함수 밖에서도 사용할 수 있을까?

  NO!

  함수 안의 블록은 함수 고유의 영역임.
  따라서, 함수 안에서 선언한 변수는 함수 밖에서 사용할 수 없음.



함수(Function)

- 함수와 변수

  예시 1)

```
[ ]  def myfunction() :
         in_val = '함수 안의 변수'
```

```
[ ]  myfunction()
```

```
[ ]  in_val
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
<ipython-input-30-a23adeea90d3> in <module>
----> 1 in_val

NameError: name 'in_val' is not defined
```

함수(Function)

- 함수와 변수

  예시 2)

```
[ ]  a = 1
```

```
[ ]  def myfunction2(a) :
         a += 1
         print(a)
```

```
[ ]  myfunction2(a)
```

```
2
```

```
[ ]  a
```

```
1
```

함수 실행 시 출력값은 무엇일까요?

a는 어떤 값일까요?

함수(Function)

함수 안의 블록은 함수 고유의 영역임.

따라서, 함수 안에서 선언한 변수는 함수 밖에서 사용할 수 없음.

```
[ ]  def 함수이름(함수인자) :
         수행할 문장1
         수행할 문장2
         return 반환값
```

2. global 사용하기 ← 권장하지 않음

- 함수와 변수

- 함수 안에 있는 변수를 밖에서도 쓰고 싶다면?

1. return 사용하기
2. global 사용하기

함수 안에 있는 변수를 밖에서도 쓰고 싶다면?

1. return 사용하기

```
a = 1

def myfunction3(a):
    a += 1
    print(a)
    return a  # a 값을 반환

a = myfunction3(a)  # 반환된 값을 밖의 변수 a에 할당
print("함수 밖의 a:", a)
```

꼭 값을 할당해야 함

함수(Function)

함수 안에 있는 변수를 밖에서도 쓰고 싶다면?

2. global 사용하기(권장하지 않음)

```
[ ] a = 1

[ ] def myfunction4():
        global a
        a += 1
        print(a)

[ ] myfunction4()

    2

[ ] a

    2
```

```
a=1
print('처음에 있는 전역변수',a)
def myfunction():
    global a
    a+=1
    print('안에 있는 a',a)
myfunction()
print('밖에 있는 a',a)
```

```
처음에 있는 전역변수 1
안에 있는 a 2
밖에 있는 a 2
```

함수 안에 있는 변수를 밖에서도 쓰고 싶다면?

3. 딕셔너리나 리스트를 사용해서 값을 변경하기

```
data = {"a": 1}

def myfunction3(data):
    data["a"] += 1
    print(data["a"])

myfunction3(data)
print("함수 밖의 a:", data["a"])
```

연습문제

- 팩토리얼(Factorial)을 구하시오.

1부터 시작하여 어떤 범위에 있는 모든 정수를 곱하는 것.

Ex) 5! = 120

Function annotation

```
def func_name(params):
  return 0
```

```
def func_name(param1 : int, param2 : str) -> int:
  return 0
```

- 파이썬 3.5부터 추가된 기능(PEP 484 기반)
- 함수의 매개변수와 반환값에 **자료형을 주석처럼 표시**할 수 있음
- 실행 결과에는 영향을 주지 않고, **코드를 읽는 사람과 도구**를 위한 설명

```
def greet(name: str) -> None:
    print("안녕하세요,", name)

greet("철수")   # 정상 실행
greet(123)      # 실행은 되지만, 타입상 권장 X
```

```
안녕하세요, 철수
안녕하세요, 123
```

```
def add(a: int, b: int) -> int:
    return a + b

print(add(2, 3))      # 5
print(add("a", "b"))  # 실행은 되지만 타입상 맞지 않음
```

```
5
ab
```

- PEP 484: 파이썬 3.5부터 적용된 “타입 힌트 문법”을 정의한 공식 문서

Match Case문

- 파이썬 3.10 버전부터 Match와 Case문이 도입이 됨.
- Match문 : 패턴 매칭을 통해 값을 비교하고 실행 코드를 선택하는데 사용.
- Case문 : Match문 내에서 특정 패턴에 대한 처리를 정의하는데 사용.

```
match value:
    pattern_1:
        # pattern_1에 일치하는 경우 실행할 코드
    pattern_2:
        # pattern_2에 일치하는 경우 실행할 코드
    ...
    pattern_n:
        # pattern_n에 일치하는 경우 실행할 코드
    _:
        # 위의 패턴에 일치하지 않는 경우 실행할 코드
```

```
def classify_number(n):
    match n:
        case 0:
            print("Zero")
        case 1 | 2 | 3:
            print("Small number")
        case 4 | 5 | 6 | 7 | 8 | 9:
            print("Big number")
        case _:
            print("Other number")

classify_number(0)  # Zero
classify_number(2)  # Small number
classify_number(7)  # Big number
classify_number(10) # Other number
```

7. 파일 읽기/쓰기

- 파일 쓰기
- 파일 읽기



파일 쓰기

- open

  open은 파일을 여는데 사용하는 함수

open(file_path, mode= ' ', encoding= ' ')

- **file_path** : 파일의 주소
- **mode** : 파일을 열 때 필요한 파일 모드(읽기/쓰기/수정)
- **encoding** : 파일을 열 때 필요한 인코딩 (생략 가능)

```
[ ] new_file = open('/content/drive/My Drive/Colab Notebooks/python/toy_data/new.txt', 'w')

[ ] print(new_file)

<_io.TextIOWrapper name='./drive/My Drive/Colab Notebooks/day2/toy_data/new.txt' mode='w' encoding='UTF-8'>
```

* 주의 * open으로 연 파일은 파일객체를 반환함

파일 쓰기

1. file path

- file path(파일 경로)는 string
- 파일은 두 가지 방식으로 표현할 수 있음
    - 절대 경로 : 파일이 가지고 있는 고유한 경로
    - 상대 경로 : 현재 내 위치에서부터 해당 파일까지의 경로

파일 쓰기

```
import os # Operating System의 약자.
#운영체제에서 사용할 수 있는 기능을 파이썬에서 쓸 수 있게 해주는 파이썬 기본 모듈

# 현재 작업 위치 확인
print("현재 작업 디렉토리:", os.getcwd())

# 1. 파일 생성
with open("sample.txt", "w", encoding="utf-8") as f:
    f.write("Hello, this is a test file.₩n")

print("sample.txt 파일이 생성되었습니다.")

# 2. 상대 경로로 읽기
with open("sample.txt", "r", encoding="utf-8") as f:
    content_relative = f.read()
print("[상대 경로 읽기 결과]")
print(content_relative)
```

```
# 3. 절대 경로로 읽기
abs_path = os.path.abspath("sample.txt")
with open(abs_path, "r", encoding="utf-8") as f:
    content_absolute = f.read()
print("[절대 경로 읽기 결과]")
print(content_absolute)

# 4. 경로 비교 출력
print("상대 경로: sample.txt")
print("절대 경로:", abs_path)
```

```
현재 작업 디렉토리: /content
sample.txt 파일이 생성되었습니다.
[상대 경로 읽기 결과]
Hello, this is a test file.

[절대 경로 읽기 결과]
Hello, this is a test file.

상대 경로: sample.txt
절대 경로: /content/sample.txt
```

파일 쓰기

1. 파일 경로 지정

- file path -> string 형태

2. 윈도우 환경 주의사항

- 윈도우에서는 경로에 ₩u가 포함될 경우, 유니코드 에러로 파일이 열리지 않을 수 있음

```
# ------------------------------------------
# 윈도우 환경 경로 문제 시뮬레이션
# ------------------------------------------
print("₩n[윈도우 환경 경로 문제 체험]")

# Step 6. 잘못된 예시 → SyntaxError 때문에 실행조차 안 됨
print("잘못된 예시: f = open(₩"C:₩₩Users₩₩Guest₩₩test_path.txt₩")")
print("→ SyntaxError 발생 (파이썬이 ₩₩U 를 유니코드 이스케이프 시퀀스로 인식)")

# Step 7. 해결 방법 1: raw string 사용
try:
    f = open(r"C:₩Users₩Guest₩test_path.txt", "r", encoding="utf-8")
    print("[raw string 읽기 결과]")
    print(f.read())
    f.close()
except Exception as e:
    print("[raw string 실행 오류]", e)
```

```
# Step 8. 해결 방법 2: 백슬래시 두 번 사용
try:
    f = open("C:₩₩Users₩₩Guest₩₩test_path.txt", "r", encoding="utf-8")
    print("[백슬래시 두 번 읽기 결과]")
    print(f.read())
    f.close()
except Exception as e:
    print("[백슬래시 두 번 실행 오류]", e)
```

2. mode

파일 모드	설명	비고
r	읽기 모드	파일에 새로운 내용을 쓰거나 수정이 불가능
w	쓰기 모드	파일에 새로운 내용을 덮어씀(기존의 내용 모두 삭제)
a	수정 모드	파일에 새로운 내용이 추가됨(기존의 내용 + 새로운 내용)
rb	바이너리 읽기 모드	바이너리 파일을 읽을 때 사용
wb	바이너리 쓰기 모드	바이너리 파일을 쓸 때 사용

파일 쓰기

- write

파일.write(내용)

- 파일 : open으로 열었던 파일 객체
- 내용 : 파일에 쓰고 싶은 내용

```
# 쓰기 모드 (w): 기존 내용 삭제 후 새로 작성
with open("test.txt", "w", encoding="utf-8") as f:
    f.write("첫 번째 줄₩n")

# 추가 모드 (a): 기존 내용 유지 + 뒤에 이어쓰기
with open("test.txt", "a", encoding="utf-8") as f:
    f.write("두 번째 줄₩n")

# 읽기 모드 (r)
with open("test.txt", "r", encoding="utf-8") as f:
    print(f.read())
```

```
# 파일 열기 (쓰기 모드)
f = open("note.txt", "w", encoding="utf-8")

# 내용 쓰기
f.write("첫 번째 줄입니다.₩n")
f.write("두 번째 줄입니다.₩n")

# 파일 닫기
f.close()
print("note.txt 파일이 저장되었습니다.")
```

```
note.txt 파일이 저장되었습니다.
```

```
f = open("note.txt", "r", encoding="utf-8")
print(f.read())
f.close()
```

```
첫 번째 줄입니다.
두 번째 줄입니다.
```

파일 쓰기

3. encoding

- 지정하지 않으면 운영체재 / 플랫폼 기본값 사용됨

- UTF-8 : 국제 표준, 가장 널리 사용되는 유니코드 방식

- CP949: 한국 윈도우 환경에서 한글 기본 인코딩(명령 프롬프트 등)

  : 과거 호환성 때문에 여전히 사용

파일 쓰기

```
# Step 1. UTF-8로 파일 저장
with open("utf8_file.txt", "w", encoding="utf-8") as f:
    f.write("안녕하세요. UTF-8 테스트입니다.₩n")

print("[생성 완료] utf8_file.txt (UTF-8 인코딩)")

# Step 2. UTF-8로 읽기
with open("utf8_file.txt", "r", encoding="utf-8") as f:
    print("[UTF-8로 읽은 결과]")
    print(f.read())

# Step 3. CP949로 저장 (Colab은 리눅스라 CP949 미지원 → 에러 확인 가능)
try:
    with open("cp949_file.txt", "w", encoding="cp949") as f:
        f.write("안녕하세요. CP949 테스트입니다.₩n")
    print("[생성 완료] cp949_file.txt (CP949 인코딩)")
except Exception as e:
    print("[CP949 저장 오류]", e)
```

```
# Step 4. 잘못된 인코딩으로 읽기 (UTF-8 파일을 CP949로 읽기 시도)
try:
    with open("utf8_file.txt", "r", encoding="cp949") as f:
        print("[UTF-8 파일을 CP949로 읽은 결과]")
        print(f.read())
except Exception as e:
    print("[읽기 오류]", e)
```

파일 읽기

- read / readline / readlines

함수	설명
read	파일 **전체**를 한번에 읽어서 string으로 반환
readline	파일을 **한 줄만** 읽어서 string으로 반환
readlines	파일을 **줄단위**로 모두 읽어서 list로 반환

파일 읽기

- read(): 파일 전체를 문자열로 변환
- readline(): 한 줄씩 읽음
- readlines(): 모든 줄을 리스트로 변환

```
with open("note.txt", "w", encoding="utf-8") as f:
    f.write("첫 번째 줄₩n두 번째 줄₩n세 번째 줄₩n")

# read()
with open("note.txt", "r", encoding="utf-8") as f:
    print("read():", f.read())

# readline()
with open("note.txt", "r", encoding="utf-8") as f:
    print("readline():", f.readline().strip())

# readlines()
with open("note.txt", "r", encoding="utf-8") as f:
    print("readlines():", f.readlines())
```

```
read(): 첫 번째 줄
두 번째 줄
세 번째 줄

readline(): 첫 번째 줄
readlines(): ['첫 번째 줄₩n', '두 번째 줄₩n', '세 번째 줄₩n']
```

- Strip(): 문자열의 양쪽 끝에 있는 공백 문자(또는 지정된 문자)를 제거하여 새로운 문자열을 반환

8. 파이썬 심화

- enumerate
- Zip/Lambda/map
- List comprehension
- Class
- 모듈 /패키지



파이썬 심화

Pop quiz) 문제를 먼저 풀어보자.

- 여기는 동네 유명한 빵집이다. 사람들에게 먼저 온 순서대로 번호표를 나누어주려고 한다.
  - 번호표를 나누어주는 함수를 작성해보자
  - 함수는 사람 이름으로 되어 잇는 리스트를 받아서 "대기번호 x번: 사람이름" 를 화면에 출력하고 (번호표, 사람이름)을 원소로 이루어진 리스트를 반환한다.

- input : 리스트
- output : 리스트

```
people = ['펭수', '뽀로로', '뚝딱이', '텔레토비']
```

```
# func1을 만들어주세요
```

```
lines = func1(people)
```

```
대기번호 1번 : 펭수
대기번호 2번 : 뽀로로
대기번호 3번 : 뚝딱이
대기번호 4번 : 텔레토비
```

파이썬 심화

```
[ ]  people = ['펭수', '뽀로로', '뚝딱이', '텔레토비']
```

```
[ ]  def func1(line) :
         new_lines = []
         i = 1
         for x in line :
             print("대기번호 %d번 : %s" %(i, x))
             new_lines.append((i, x))
             i += 1
         return new_lines
```

대기 번호를 추적하는 변수 i

별도로 업데이트 해야함

```
[ ]  lines = func1(people)
```

```
대기번호 1번 : 펭수
대기번호 2번 : 뽀로로
대기번호 3번 : 뚝딱이
대기번호 4번 : 텔레토비
```

파이썬 심화_enumerate

- 반복가능한 객체의 인덱스와 원소에 함께 접근할 수 있는 함수
- tuple(인덱스, 원소)의 형태로 객체를 반환
- 보통 리스트와 함께 쓰임(tuple도 가능)

```
[ ] lst = ['a','b','c']
    for x in enumerate(lst) :
        print(x)
```

```
(0, 'a')
(1, 'b')
(2, 'c')
```

파이썬 심화_enumerate

```
[ ] st = 'abcd'

    for x in enumerate(st) :
        print(x)
```

```
(0, 'a')
(1, 'b')
(2, 'c')
(3, 'd')
```

```
[ ] dic = {0 : 'p', 1 : 'b' , 2 : 'd'}

    for x in enumerate(dic):
        print(x)
```

```
(0, 0)
(1, 1)
(2, 2)
```

```
[ ] se = {'a','b','c'}
    for x in enumerate(se) :
        print(x)
```

```
(0, 'c')
(1, 'a')
(2, 'b')
```

주의

set은 순서가 없는 자료형

파이썬 심화_enumerate

앞에서 풀었던 퀴즈를 enumerate를 이용해서 다시 작성해보자(빵집 문제)

```
[ ] def func1_with_enu(line) :
        new_lines = []
        for idx, val in enumerate(line):
            print("대기번호 %d번 : %s" %(idx+1, val))
            new_lines.append((idx+1,val))
        return new_lines
```

```
[ ] lines = func1_with_enu(people)
```

```
대기번호 1번 : 펭수
대기번호 2번 : 뽀로로
대기번호 3번 : 뚝딱이
대기번호 4번 : 텔레토비
```

파이썬 심화_zip

- 반복 가능한 객체들을(2개 이상) 병렬적으로 묶어주는 함수
- 각 원소들을 튜플의 형식으로 묶어줌

```
[1] str_list = ['one', 'two', 'three', 'four']
    num_list = [1, 2, 3, 4]

    for i in zip(num_list, str_list) :
        print(i)
```

```
(1, 'one')
(2, 'two')
(3, 'three')
(4, 'four')
```

```
[2] s1 = '123'
    s2 = 'abc'
    s3 = 'ㄱㄴㄷ'
    list(zip(s1,s2,s3))
```

```
[('1', 'a', 'ㄱ'), ('2', 'b', 'ㄴ'), ('3', 'c', 'ㄷ')]
```

파이썬 심화_lambda

- lambda(람다)는 식 형태로 정의되는 익명 함수 -> lambda expression(람다 표현식)
- 간단한 함수 정의에 유용, 코드 간결화에 도움
- Python3에서 사용이 권장되지는 않지만 머신러닝 · 데이터 분석에서 간단한 연산 시 자주 사용

lambda 매개변수: 반환값

```
f = lambda x:x**2
print(f(3))
```

```
9
```

파이썬 심화_lambda

- lambda : 한 줄을 실행한 결과 값이 바로 반환값이 됨.

```
[ ]  def plus_two(num) :
         return num + 2
```

```
[ ]  a = 2
     b = plus_two(a)
     print(b)
```

```
4
```

```
[ ]  lambda x : x + 2
```

```
<function __main__.<lambda>(x)>
```

→ 람다는 함수를 생성함

```
[ ]  func2 = lambda x : x + 2
```

```
[ ]  c = func2(2)
```

```
[ ]  c
```

```
4
```

파이썬 심화_map

- 리스트, 튜플, string 등 자료형 각각의 원소에 동일한 함수를 적용

map (함수, 자료형)

```
items = [1, 2, 3, 4, 5]

squared = []
for i in items :
    squared.append(i*i)

print(squared)
```

```
[1, 4, 9, 16, 25]
```



map

```
squared_map = list(map(lambda x : x**2, items))

print(squared_map)
```

```
[1, 4, 9, 16, 25]
```

연습 문제 1

lambda와 map을 이용하여 items의 요소들을 string(문자)로 바꾸는 것을 짜봅시다.

```
items = [1, 24, 3, 6, 7]
```

```
# 정답을 적어주세요
```

```
print(str_items)
```

```
['1', '24', '3', '6', '7']
```

연습 문제 2

1~10까지의 정수를 항목으로 갖는 리스트 객체에서 map 함수와 람다식을 이용해 항목의 제곱 값을 갖는 리스트를 반환하는 프로그램을 작성하십시오.

파이썬 심화_list comprehension

파이썬만의 독특한 문법으로 간결한 코딩을 할 수 있음

1) for 문

0부터 9까지를 순서대로 가지고 있는 리스트를 만드세요.

```
[ ]  list_1 = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
[ ]  list_2 = []
     for x in range(10) :
         list_2.append(x)
     print(list_2)
```



```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

파이썬 심화_list comprehension

1) for 문

0부터 9까지를 순서대로 가지고 있는 리스트를 만드세요. (한줄로)

```
[ ]  lc_1 = [x for x in range(10)]
     print(lc_1)
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

[저장할 값 **for** 원소 **in** 반복 가능한 객체 **]**

파이썬 심화_list comprehension

1) for 문

Quiz 1) 구구단 2단을 list comprehension을 이용하여 구현하고 리스트를 화면에 출력해보자.

Quiz 2) 다음의 문장을 분석해보자.

"코로나 바이러스를 예방하기 위해 사회적 거리두기를 실천합시다. 마스크를 끼고 손씻기를 생활화합시다."

라는 문장을 띄어쓰기별로 분석하려고 한다. 띄어쓰기별로 문장을 나눈 후 각 요소의 길이를 리스트에 저장하라.

(Hint : 띄어쓰기는 split 함수를 써라.)

파이썬 심화_list comprehension

2) for 문 + if 문

10부터 20 사이의 숫자들 중에서 짝수만을 담은 리스트를 만들어보자.
(List Comprehension을 사용하시오.)

```
[33] list_3 = []
     for x in range(10, 21) :
         if x % 2 == 0 :
             list_3.append(x)
     print(list_3)
```

```
[10, 12, 14, 16, 18, 20]
```

파이썬 심화_list comprehension

2) for 문 + if 문

10부터 20 사이의 숫자들 중에서 짝수만을 담은 리스트를 만들어보자.

```
[34] lc_2 = [x for x in range(10, 21) if x % 2 == 0]
```

```
[35] lc_2
```

```
[10, 12, 14, 16, 18, 20]
```

[저장할 값 for 원소 in 반복 가능한 객체 if 조건]

2) for 문 + if 문

Quiz 1) 1부터 10의 제곱수 중, 50 이하인 수만 리스트에 저장하라.

Quiz 2) 다음의 문장을 분석해보자.

"코로나 바이러스를 예방하기 위해 사회적 거리두기를 실천합시다. 마스크를 끼고 손씻기를 생활화합시다." 라는 문장을 띄어쓰기별로 분석하려고 한다. 띄어쓰기 별로 문장을 나눈 후 각 요소의 길이가 5미만인 것들만 리스트에 저장하라.

파이썬 심화_list comprehension

2) for 문 + if 문

1부터 10까지의 숫자들 중 홀수이면 제곱수를, 짝수이면 세제곱수를 담은 리스트를 만들어보자.

```
[40] list_4 = []
     for x in range(1, 11) :
         if x % 2 == 1 :
             list_4.append(x ** 2)
         else :
             list_4.append(x ** 3)
```

```
[41] list_4
```

```
[1, 8, 9, 64, 25, 216, 49, 512, 81, 1000]
```

[저장할 값 if 조건 else 저장할 값 for 원소 in 반복 가능한 객체]

파이썬 심화_list comprehension

3) for 문 + for 문

```
word_1 = 'Hello'
word_2 = 'World'

result = [i+j for i in word_1 for j in word_2]
result
```

```
['HW',
 'Ho',
 'Hr',
 'Hl',
 'Hd',
 'eW',
 'eo',
 'er',
 'el',
 'ed',
 'lW',
 'lo',
 'lr',
 'll',
 'ld',
 'lW',
 'lo',
 'lr',
 'll',
 'ld',
 'oW',
 'oo',
 'or',
 'ol',
 'od']
```

파이썬 심화_list comprehension

2) For 문 + if문

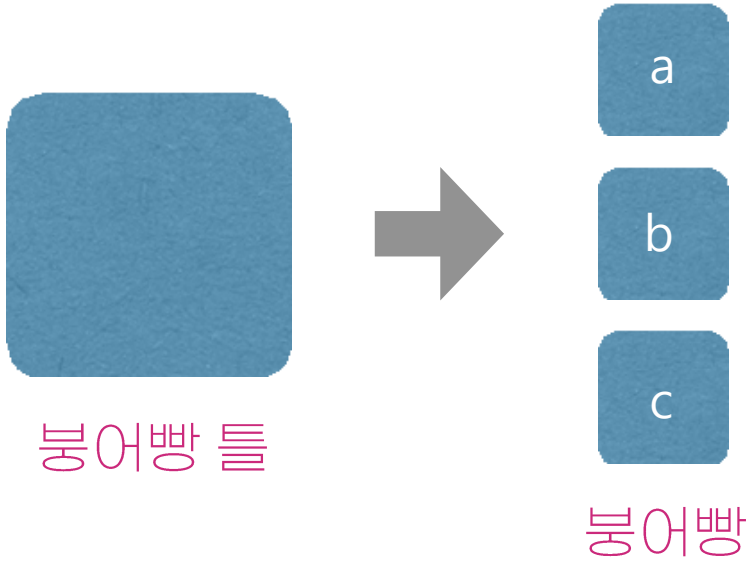
Quiz 1) 40 이하의 숫자는 5를 더하고, 40 초과의 숫자는 41로 바꾸어 리스트로 저장하고,
리스트를 출력하라.

Quiz 2) 컷트라인이 60점일 때, 사람 이름과 통과여부를 리스트로 담아서 출력하라.
이름과 통과 여부는 튜플로 묶여있는 자료이다.

```
[ ]  students = {"보라돌이" : 61,
                 "뚜비" : 35,
                 "나나" : 78,
                 "뽀" : 88}
```

파이썬 심화_class

- 객체(Object): 사람, 자동차, 강아지처럼 실제로 존재하는 사물을 말함.
- 클래스(Class): 객체를 만드는 구조/틀
- 인스턴스(Instance) : 클래스가 실질적으로 객체를 만들었을 때 그 객체를 부르는 용어



a, b, c는 각각 객체

a, b, c는 MyClass의 인스턴스

파이썬 심화_class

- 인스턴스(Instance)

  : 클래스가 실질적으로 객체를 만들었을 때, 그 객체를 부르는 용어

```
[5]  class MyClass():
         pass
```

→ 클래스를 선언

```
a = MyClass()
b = MyClass()
c = MyClass()
```

→ 함수처럼 콜(call)하여 사용

```
print(a,b,c)
```

객체(object) 생성 →

```
<__main__.MyClass object at 0x7ff058193eb8>
<__main__.MyClass object at 0x7ff058193e80>
<__main__.MyClass object at 0x7ff058193ef0>
```

- 인스턴스(Instance)

  클래스가 실질적으로 객체를 만들었을 때, 그 객체를 부르는 용어

  1. Instance variable(인스턴스 변수)

     : 인스턴스 고유의 변수

  2. Instance method(인스턴스 메소드)

     : 인스턴스에 적용되는 함수

- 인스턴스(Instance)

```
[1]  class Account():
         def make_account(self) :
             self.balance = 0

         def deposit(self, money) :
             self.balance += money

         def draw(self, money) :
             self.balance -= money
```

인스턴스

```
[2]  a1 = Account()
```

self : 인스턴스를 위한 placeholder

→ 클래스를 선언

- 인스턴스(Instance)

```
[1]  class Account():
         def make_account(self) :
             self.balance = 0

         def deposit(self, money) :
             self.balance += money

         def draw(self, money) :
             self.balance -= money
```

```
[2]  a1 = Account()
     a1.make_account()
     a1.deposit(1000)
     print(a1.balance)
```

```
1000
```

- 인스턴스.변수
- 인스턴스.메소드

-> 위와 같은 방식으로 클래스 선언시에 만들었던 변수들, 메소드들을 사용할 수 있음

파이썬 심화

- 인스턴스(Instance) 메소드(Instance Method)

주의 인스턴스 메소드의 매개변수 중 가장 첫번째는 인스턴스의 자리

C++	Python

class Person{
  int age;
public :
  Person(){
  this->age = 17;
  }
}
	class person():
	  def __init__(self):
	  self.age = 17
	
	[51] p1 = person()
	
	[52] p1.age
	17

- 인스턴스(Instance) 메소드(Instance Method)

주의 인스턴스 메소드의 매개변수 중 가장 첫번째는 인스턴스의 자리

```
[1]  class TestClass1():
         def __init__(self, in1, in2):
             self.v1 = in1
             self.v2 = in2

[2]  t_cls1 = TestClass1(10, 20)

[3]  t_cls1.v1

     10
```

인스턴스

-> 인스턴스 메소드의 매개변수

-> 파이썬은 인스턴스를 자동으로 넘겨줌

파이썬 심화

- 인스턴스(Instance) 메소드(Instance Method)

  주의 인스턴스 메소드의 매개변수 중 가장 첫번째는 인스턴스의 자리

```
[4]  class TestClass2():
         def __init__(in1, in2):
             self.v1 = in1
             self.v2 = in2
```

```
[5]  t_cls2 = TestClass2(10, 20)
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-5-32e19ab7347c> in <module>()
----> 1 t_cls2 = TestClass2(10, 20)

TypeError: __init__() takes 2 positional arguments but 3 were given

SEARCH STACK OVERFLOW
```

- Instance Method(인스턴스 메소드)

1) __init__(초기화자) : 인스턴스가 생성될 때, 객체의 초기값을 설정하는 메소드로, 자동으로 호출됨

2) __del__(소멸자) : 인스턴스가 소멸될 때, 자동으로 호출되는 메소드

```
[1]  class TestClass1():
         def __init__(self, in1, in2):
             self.v1 = in1
             self.v2 = in2
```

-> 초기화자를 통해서 객체의 초기값을 자동으로 설정

```
[2]  t_cls1 = TestClass1(10, 20)
```

```
[3]  t_cls1.v1
```

```
10
```

- Instance (인스턴스)

```
[1]  class Account():
         def make_account(self) :
             self.balance = 0

         def deposit(self, money) :
             self.balance += money

         def draw(self, money) :
             self.balance -= money
```

인스턴스 변수

인스턴스 메소드

인스턴스

```
[2] a1 = Account()
    a1.make_account()
    a1.deposit(1000)
    print(a1.balance)
```




```
1000
```

통장1에 1000원을 넣음

```
[3] a2 = Account()
    a2.make_account()
    a2.deposit(5000)
    print(a2.balance)
```




```
5000
```

통장2에 5000원을 넣음

- Instance (인스턴스)

```
[1]  class Account():
         def make_account(self) :
             self.balance = 0

         def deposit(self, money) :
             self.balance += money

         def draw(self, money) :
             self.balance -= money
```

인스턴스 변수

클래스 이름

```
Account.balance
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent
<ipython-input-4-35da668b8893> in <module>()
----> 1 Account.balance

AttributeError: type object 'Account' has no attribute 'balance'
```



→ 통장끼리는 잔고를 공유하지 않음

파이썬 심화_class

```
class 클래스 이름:
  변수1
  변수2
  ......필요한 만큼 변수를 제공.....
    def 메소드1(인수):
      ......메소드의 처리.....
    def 메소드2(인수):
      ......메소드의 처리..... .....
      필요한만큼 메소드를 제공.....
```

```python
class MyClass:
    # 클래스 변수: 클래스 전체가 공유하는 값
    class_var = "공유되는 값"

    def __init__(self, name):
        # 인스턴스 변수: 객체마다 따로 가지는 값
        self.name = name
        self.instance_var = 0

    def instance_method(self):
        # 인스턴스 메서드: self(내 객체)의 값을 바꾸는 동작
        self.instance_var += 1
        print(self.name, ":", self.instance_var)


# 사용 예
a = MyClass("A")
b = MyClass("B")

a.instance_method()  # A : 1
a.instance_method()  # A : 2
b.instance_method()  # B : 1

print(a.class_var)   # 공유되는 값
print(b.class_var)   # 공유되는 값

MyClass.class_var = "바뀐 공유 값"
print(a.class_var)   # 바뀐 공유 값
print(b.class_var)   # 바뀐 공유 값
```

- Instance (인스턴스)

```
class test():
  name = '아무나'
  age = 0
  def __init__(self,name,age):
    print('생성자 호출!')
    self.name = name
    self.age = age
  def __del__(self):
    print('소멸자 호출!')
  def info(self):
    print('나의 이름은', self.name, '입니다!')
    print('나이는', self.age,'입니다!')
```

```
r = test('류영표',7)
```

```
생성자 호출!
```

```
r.info()
```

```
나의 이름은 류영표 입니다!
나이는 7 입니다!
```

```
[14] test('류영표',7)
```

```
생성자 호출!
<__main__.test at 0x7ff089935e80>
```

```
[15] test.info()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-15-a72a0779be47> in <module>()
----> 1 test.info()

TypeError: info() missing 1 required positional argument: 'self'
```

- Instance (인스턴스)

```
class ClassExample:
    def __init__(self, name, age):  # 생성자(Constructor) 생성 시 자동 수행
        self.name = name
        self.age = age + 5  # 변수 생성 후 파라미터로 보낸 값 할당
        print("[생성자] self와 변수 비교 " + str(self.age) + " age : " + str(age))
        print("[생성자] 이름 : " + self.name + " 나이 : " + str(self.age))

    def ten_year_call(self, val):  # 함수
        return self.age + 10  # 반환
```

```
[2] a = ClassExample("철수", 20)  # 객체화 (ClassExample의 정보를 a에 담음)
    print(a.age)  # a의 age 출력
    print(a.ten_year_call(50))  # a의 ten_year_call 메소드 수행
```

```
[생성자] self와 변수 비교 25 age : 20
[생성자] 이름 : 철수 나이 : 25
25
35
```

- 객체 슬라이싱과 __getitem__

```
class CustomNumbers:
  def __init__(self):
    self._numbers = [n for n in range(1, 11)]

a = CustomNumbers()
```

```
a._numbers[2:5]
```

```
[3, 4, 5]
```

```
a[2:5]
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
<ipython-input-37-cd92fdba5c05> in <module>
----> 1 a[2:5]

TypeError: 'CustomNumbers' object is not subscriptable
```

- 객체 슬라이싱과 __getitem__

- 인스턴스 변수에 직접 접근하지 말고 객체 자체를 통해서 슬라이싱을 구현하기 위해서는 __getitem__ 특별 메서드를 정의해야 함.

```
class CustomNumbers:
  def __init__(self):
    self._numbers = [n for n in range(1, 11)]

  def __getitem__(self, idx):
    return self._numbers[idx]

a = CustomNumbers()
a[2:7]
# [3, 4, 5, 6, 7]
```

파이썬 심화

- 메소드 오버로딩(method overloading)

```
class Adder{
    static int add(int a,int b)
    {
        return a+b;
    }
    static int add(int a,int b,int c)
    {
        return a+b+c;
    }
}
```

자바는 함수이름이 같아도 허용.

```
class Korea:

    def __init__(self, name,population, captial):
        self.name = name
        self.population = population
        self.capital = captial

    def show(self):
        print(
            """
            국가의 이름은 {} 입니다.
            국가의 인구는 {} 입니다.
            국가의 수도는 {} 입니다.
            """.format(self.name, self.population, self.capital)
        )

    def show(self, abc):
        print('abc :', abc)
```

```
>>> from inheritance import *
>>> a = Korea('대한민국',50000000, '서울')
>>> a.show()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: show() missing 1 required positional argument: 'abc'
```

파이썬 심화

- Class Variable과 Class Method

1. class variable(클래스 변수)

   클래스와 인스턴스 전체가 공유하는 변수

2. class method(클래스 메소드)

   클래스와 인스턴스 전체가 공유하는 함수

   인스턴스와 상관없이 사용(인스턴스가 없어도 사용 가능) 클래스와 인스턴스 전체에 공유됨

파이썬 심화

- Class Variable과 Class Method

1. class variable(클래스 변수)

   클래스와 인스턴스 전체가 공유하는 변수

2. class method(클래스 메소드)

   클래스와 인스턴스 전체가 공유하는 함수

```
[8]  class MyClass2() :
         class_var = '클래스 변수'

         @classmethod
         def class_method(cls):
             print('클래스의 메소드')
```

데코레이터(decorator) : wrapping을 통해 특정 코드를 재사용할 수 있게 해주는 역할

cls : 클래스를 위한 placeholder (self와 유사한 역할)

→ 파이썬은 클래스 또한 자동으로 넘겨줌

- Class Variable과 Class Method

```
[8]  class MyClass2() :
         class_var = '클래스 변수'

         @classmethod
         def class_method(cls):
             print('클래스의 메소드')
```

```
[10] e = MyClass2()
     print(e.class_var)
     print(MyClass2.class_var)
```

```
클래스 변수
클래스 변수
```

```
e.class_method()
MyClass2.class_method()
```

```
클래스의 메소드
클래스의 메소드
```

인스턴스.변수 클래스이름.변수
인스턴스.메소드 클래스이름.메소드

→ 위와 같은 방식으로 클래스 변수와
클래스 메소드를 사용할 수 있음

- Class Variable과 Class Method

```
[6] class Account2() :
        bank = '모두은행'
        total = 0

        @classmethod
        def merge(cls, acc1, acc2) :
            cls.total = acc1.balance + acc2.balance
            print("당신의 재산은 %d" %cls.total)

        def __init__(self) :
            self.balance = 0

        def deposit(self, money) :
            self.balance += money

        def draw(self, money) :
            self.balance -= money
```

클래스 변수

클래스 메소드



은행 내 자산

```
[7] Account2.bank
```

```
'모두은행'
```

통장1

```
[8] b1 = Account2()
    b1.deposit(4000)
    print(b1.balance)
    print(b1.bank)
```




```
4000
모두은행
```

통장2

```
[9] b2 = Account2()
    b2.deposit(7000)
    print(b2.balance)
    print(b2.bank)
```




```
7000
모두은행
```

```
[10] Account2.merge(b1, b2)
```

```
당신의 재산은 11000
```

```
[11] Account2.total
```

```
11000
```

파이썬 심화

```
[22] class Hotel() :
         def __init__(self) :
             self.room = []

         def add_person(self, name):
             self.room.append(name)
```

```
[23] r1 = Hotel()
     r2 = Hotel()

     r1.add_person('뽀로로')
     r2.add_person('펭수')
```

```
[26] Hotel.room
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
<ipython-input-26-b87892c63c8a> in <module>()
----> 1 Hotel.room

AttributeError: type object 'Hotel' has no attribute 'room'
```

파이썬 심화

```
[27] class GuestHouse() :

         guest = []

         def __init__(self) :
             self.room = []

         @classmethod
         def check_in(cls, name):
             cls.guest.append(name)

         def add_person(self, name):
             self.check_in(name)
             self.room.append(name)
```

```
[28] r3 = GuestHouse()
     r4 = GuestHouse()

     r3.check_in('뚝딱이')
     r4.check_in('뿡뿡이')
```

```
[29] GuestHouse.guest
```

```
['뚝딱이', '뿡뿡이']
```

```
[30] r3.room
```

```
[]
```

```
[31] r4.room
```

```
[]
```

```
[32] r3.add_person('홍길동')
     r4.add_person('텔레토비')
```

```
[33] r3.room
```

```
['홍길동']
```

```
[34] r4.room
```

```
['텔레토비']
```

```
[35] GuestHouse.guest
```

```
['뚝딱이', '뿡뿡이', '홍길동', '텔레토비']
```

- 클래스 상속(inheritance)

상속 : 부모의 클래스를 물려받음

-> 부모 클래스가 가지고 있는 함수/변수를 그대로 사용할 수 있음

상속의 장점 : 기존 클래스를 변형하지 않고 추가/ 변경이 가능함

상속받고자 하는 클래스 (부모 클래스(super class)

```
[ ]  class MyPhone2(MyPhone):
         def has_case(self, val=False) :
             self.case = val
```

자식 클래스(sub Class)의 새로운 메소드

- 클래스 상속(inheritance)

```
[ ]  class MyPhone():
         def __init__(self, model, color) :
             self.model = model
             self.color = color

         def set_name(self, name):
             self.user = name
             print("사용자의 이름은 : %s" %self.user)

         def set_number(self, number):
             self.number = number
```

```
[ ]  class MyPhone2(MyPhone):
         def has_case(self, val=False) :
             self.case = val
```

```
[48] p2 = MyPhone2('iphone', 'red')
```

```
[49] p2.set_name("MJ")
```

```
사용자의 이름은 : MJ
```

부모 클래스의 메소드 사용 가능

```
[50] p2.has_case(True)
```

```
[51] p2.case
```

```
True
```

자식 클래스의 새로운 메소드도 사용 가능

- 메소드 오버라이딩(overriding)

➢ 상속을 받은 자식 클래스가 상속을 해준 부모 클래스의 메소드를 변형하는 방법

➢ 원본(부모 클래스)의 소스코드는 그대로 유지한 채 소스코를 확장

➢ 개인화 할 수 있다는 장점이 있음

파이썬 심화

```
class Calculate:
    type = 'low'

    def __init__(self,n1,n2):
        self.n1 = n1
        self.n2 = n2

    def sum(self):
        print(self.n1+self.n2)
```

```
c = Calculate(4,2)
print(c.type)
c.sum()
```

```
low
6
```

```
class Calculate_1(Calculate):
    type = 'high'

    def sub(self):
        print(self.n1-self.n2)

    def mul(self):
        print(self.n1*self.n2)
```

```
c1 = Calculate_1(4,2)
print(c1.type)
c1.sum()
c1.sub()
c1.mul()
```

```
high
6
2
8
```

파이썬 심화

- 메소드 오버라이딩(overriding)

```
[46] class MyPhone():
         def __init__(self, model, color) :
             self.model = model
             self.color = color

         def set_name(self, name):
             self.user = name
             print("사용자의 이름은 : %s" %self.user)

         def set_number(self, number):
             self.number = number
```

```
[52] p1 = MyPhone('iphone', 'red')
     p1.set_number("010-xxxx-xxxx")
```

```
[55] class MyPhone3(MyPhone) :
         def set_number(self, num) :
             self.number = num
             print("이 핸드폰의 번호는 : %s" %self.number)
```

```
[56] p3 = MyPhone3('iphone', 'red')
     p3.set_number("010-xxxx-xxxx")
```

```
이 핸드폰의 번호는 : 010-xxxx-xxxx
```

1번

Human 이라는 클래스를 만들어보자.

- Human 클래스는 아래와 같은 특징이 있다.
- Human 클래스는 인스턴스 생성 시 birth_date, sex, nation을 변수로 가지고 있다.
- give_name은 인스턴스 메소드로 이름을 input으로 받아서 name이라는 인스턴스 변수로 저장하고 화면에 이름을 출력하는 역할을 하는 함수이다.
- can_sing이라는 함수는 True/False값을 input으로 받으며, 참이면 "Sing a song"을 화면에 출력하는 함수이다.

2번

Human 이라는 클래스를 상속하는 Child라는 클래스를 만들어보자.

- Child의 클래스에는 아래와 같은 변수와 함수들이 추가된다.
- 눈동자 색깔을 나타내는 변수 eye를 인스턴스 선언시 사용할 수 있게 추가해보자.
- Child의 클래스는 노래하는 능력이 없다. 따라서 can_sing이라는 메소드가 호출되면 무조건 "Can't Sing"이고 출력하도록 바꿔보자.
- Child는 노래 대신 춤을 출 수 있다. can_dance라는 메소드가 호출되면 "Dance Time!"을 출력하도록 메소드를 작성해보자.

모듈(Module)

- 함수나 변수, 클래스 등을 가진 파일(.py) / jupyter notebook(.ipynb)
- 모듈안에는 함수, 클래스 또는 변수들이 정의되어 있음
- 파이썬은 많은 표준 라이브러리 모듈을 제공함

```
[ ]  import math
```

```
[ ]  math.factorial(4)
```

```
24
```

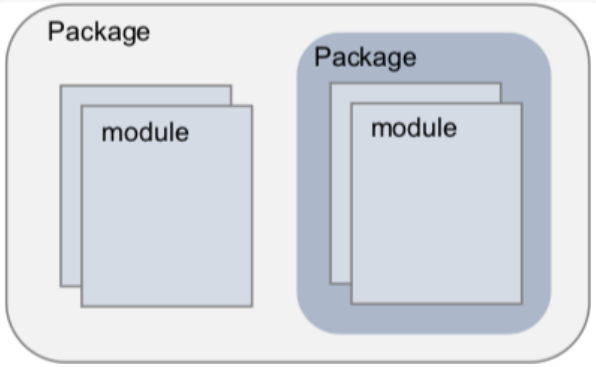
```
[ ]  from math import factorial
```

```
[ ]  factorial(4)
```

```
24
```

```
[ ]  import 패키지.모듈
     import 패키지.모듈.변수
     import 패키지.모듈.함수
     import 패키지.모듈.클래스
```

```
[ ]  from 패키지.모듈 import 변수/함수/클래스
```



예외처리

프로그램의 제어 흐름을 조정하기 위해 사용됨.

1. try : 실행코드(오류가 발생하나 보자!)
2. except : 예외처리 코드(오류가 발생하면 이걸 실행하자!)
3. else : 예외처리할 오류가 없을때 실행되는 코드
4. finally : 오류 발생여부 상관 없이 무조건 실행되는 코드
5. raise : 오류를 일부러 발생시키기

예외처리

```
int('number')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-1-3146e5e02b83> in <module>()
----> 1 int('number')

ValueError: invalid literal for int() with base 10: 'number'
```

SEARCH STACK OVERFLOW

```
try:
    int('number')
except:
    print('Not number!')
```

```
Not number!
```

```
try:
    int('number')
except ValueError as m:
    print(m)
```

```
invalid literal for int() with base 10: 'number'
```

예외처리

```
try:
    int('11')
except ValueError as m:
    print(m)
else:
    print('No Error!')
```

```
No Error!
```

```
for _ in ['11','number']:
    try:
        int(_)
    except ValueError as m:
        print(m)
    else:
        print('No Error!')
    finally:
        print('Finally Code')
    print('₩n')
```

```
No Error!
Finally Code


invalid literal for int() with base 10: 'number'
Finally Code
```

에러를 직접 일으키는 방법

```
# 190이 넘는 학생을 발견하면 반복을 종료한다.
school = {'1반' : [150, 156, 179, 191, 199], '2반' : [150, 195, 179, 191, 199]}

try:
    for class_number, students in school.items():
        for student in students:
            if student > 190:
                print(class_number, '190을 넘는 학생이 있습니다.')
                # break # 바로 상위 for문은 종료되지만 최고 상위 for문은 종료되지 않는다.
                raise StopIteration
                # 예외가 try 문 안에 있지 않으면 에러 발생시 프로그램이 멈춘다.
except StopIteration:
    print('정상종료')
```

```
1반 190을 넘는 학생이 있습니다.
정상종료
```

```
a = 10

# True 인경우
print("1. assert True 인 경우")
assert a == 10, "a is not 10"

print("끝1")
# Fasle 인 경우
print("2. assert False 인 경우")
assert a == 999, "a is not 999"

print("끝2")
```

```
1. assert True 인 경우
끝1
2. assert False 인 경우
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
<ipython-input-11-2b467e8c26a9> in <module>()
     10 # Fasle 인 경우
     11 print("2. assert False 인 경우")
---> 12 assert a == 999, "a is not 999"
     13
     14 print("끝2")

AssertionError: a is not 999
```

연습문제 7

사용자로부터 숫자를 계속 입력받다가
s or S 를 입력하면 합계출력

+다른 문자가 나왔을 때,
예외처리를 해보세요.

값을 입력해주세요30
값을 입력해주세요20
값을 입력해주세요50
값을 입력해주세요40
값을 입력해주세요30
값을 입력해주세요s
합계는 ? 170

Exception Groups & except *

파이썬 3.11+ 다중 예외 처리 핵심 요약

도입 배경

- 기존의 **"한 번에 하나의 예외 전파"** 한계 극복
- 비동기(Async) 작업 시 발생하는 **다중 실패**를 누락 없이 동시 전파
- 서로 무관한 여러 에러를 하나의 컨테이너에 담아 보고

핵심 문법: except*

- **부분 매칭:** 그룹 내 특정 타입만 골라내어 처리하고 나머지는 재전파
- **자동 래핑:** 단일 예외가 발생해도 핸들러에는 항상 **그룹 형태**로 전달
- **혼용 금지:** 기존 except와 같은 try 문에서 섞어쓰기 불가

```
except* ValueError as eg:
  for e in eg.exceptions: ...
```

TaskGroup 활용

- **구조적 병렬성:** 태스크 중 하나만 실패해도 나머지를 안전하게 자동 취소
- **에러 취합:** 모든 종료된 태스크의 실패 사례를 ExceptionGroup으로 반환
- 비동기 코드의 **안정성과 가독성** 비약적 향상

Exception Groups & except *

```
try:
    raise ExceptionGroup(
        "Occurred during processing",
        [
            ValueError("Invalid input data"),
            TypeError("Unexpected data type"),
            IndexError("Out of bounds access"),
        ],
    )

# except* 없이 잡으면: "그룹 전체"를 한 번에 잡는 것만 가능
except ExceptionGroup as eg:
    print("[ExceptionGroup] 처리:", eg)
    print("포함된 예외들:")
    for e in eg.exceptions:
        print(" -", type(e).__name__, ":", e)

print("₩n--- 프로그램 계속 진행 ---")
```

```
try:
    # 여러 예외를 포함하는 ExceptionGroup을 직접 생성합니다.
    raise ExceptionGroup(
        "Occurred during processing",
        [
            ValueError("Invalid input data"),
            TypeError("Unexpected data type"),
            IndexError("Out of bounds access")
        ]
    )
except* ValueError as e:
    print(f"[ValueError] 처리: {e}")
except* TypeError as e:
    print(f"[TypeError] 처리: {e}")
except* IndexError as e:
    print(f"[IndexError] 처리: {e}")

print("₩n--- 프로그램 계속 진행 ---")
```

```
[ValueError] 처리: Occurred during processing (1 sub-exception)
[TypeError] 처리: Occurred during processing (1 sub-exception)
[IndexError] 처리: Occurred during processing (1 sub-exception)

--- 프로그램 계속 진행 ---
```



Youngpyo Ryu. All rights reserved.
Unauthorized reproduction or distribution is prohibited.